

SNCS provides a high performant, scalable, and highly available site-to-site network connection for processing network traffic between a customer's on-premise environment and the CSPI by implementing a robust infrastructure of network elements and computing nodes that are used to provide the secure site to site network connectivity.

20 Claims, 27 Drawing Sheets

Related U.S. Application Data

which is a continuation of application No. 18/078,897, filed on Dec. 9, 2022, now Pat. No. 11,811,593, which is a continuation of application No. 17/515,093, filed on Oct. 29, 2021, now Pat. No. 11,558,245.

- (60) Provisional application No. 63/457,695, filed on Apr. 6, 2023, provisional application No. 63/457,700, filed on Apr. 6, 2023, provisional application No. 63/434,879, filed on Dec. 22, 2022, provisional application No. 63/434,846, filed on Dec. 22, 2022.
- (58) **Field of Classification Search**
USPC 709/220
See application file for complete search history.

(56) References Cited

U.S. PATENT DOCUMENTS

10,777,505	B2	9/2020	Chang et al.	
10,785,158	B1 *	9/2020	Chen	H04L 61/5007
11,372,689	B1	6/2022	Allen	
11,558,245	B1	1/2023	Kreger-Stickles et al.	
11,736,558	B2	8/2023	Kreger-Stickles et al.	
11,777,848	B2	10/2023	Tracy et al.	
11,811,593	B2	11/2023	Kreger-Stickles et al.	
11,968,078	B2	4/2024	Tessmer et al.	
12,355,664	B2	7/2025	Tracy et al.	
2012/0005521	A1	1/2012	Droux et al.	
2012/0124660	A1 *	5/2012	Wang	H04L 63/083 726/3
2014/0168453	A1 *	6/2014	Shoemake	H04L 43/08 348/207.11
2014/0207930	A1	7/2014	Benny	
2015/0301844	A1	10/2015	Droux et al.	
2018/0006877	A1 *	1/2018	Raman	H04L 41/044
2018/0062920	A1	3/2018	Srinivasan et al.	
2018/0183756	A1	6/2018	Bangalore Krishnamurthy	
2018/0284985	A1	10/2018	Bansal et al.	
2018/0287938	A1	10/2018	Han	
2018/0349163	A1	12/2018	Gao et al.	
2019/0036868	A1	1/2019	Chandrashekar et al.	
2019/0102384	A1	4/2019	Hung et al.	
2019/0265996	A1	8/2019	Shevade et al.	
2019/0347406	A1	11/2019	Lev-Ran	
2019/0392150	A1	12/2019	Shevade et al.	
2020/0167175	A1	5/2020	Tsirkin	
2020/0334241	A1 *	10/2020	Muralidhar	G06F 3/067
2020/0334242	A1 *	10/2020	Muralidhar	G06F 16/24557
2020/0382471	A1 *	12/2020	Janakiraman	H04L 61/5007
2020/0409873	A1	12/2020	Kamath et al.	
2020/0410820	A1	12/2020	Ellis et al.	
2021/0374693	A1	12/2021	La Salle	
2022/0263793	A1	8/2022	Baker et al.	
2022/0335764	A1 *	10/2022	Immanuel	G07C 9/22
2022/0342718	A1	10/2022	Iqbal et al.	
2024/0020373	A1	1/2024	Ivanov et al.	

OTHER PUBLICATIONS

International Application No. PCT/US2023/085686, "International Search Report and Written Opinion", May 27, 2024, 14 pages.

International Application No. PCT/US2023/085695, "International Search Report and Written Opinion", May 8, 2024, 14 pages.

International Application No. PCT/US2023/034751, "International Preliminary Report on Patentability", Feb. 2, 2024, 11 pages.

International Application No. PCT/US2023/034753, "International Preliminary Report on Patentability", Feb. 2, 2024, 12 pages.

U.S. Appl. No. 18/051,088, Non-Final Office Action dated Jul. 31, 2024, 15 pages.

U.S. Appl. No. 18/373,698, Notice of Allowance dated Aug. 7, 2024, 8 pages.

U.S. Appl. No. 17/175,569, "Advisory Action", Aug. 4, 2022, 4 pages.

U.S. Appl. No. 17/175,569, "Final Office Action", Apr. 15, 2022, 22 pages.

U.S. Appl. No. 17/175,569, "Non-Final Office Action", Sep. 15, 2022, 24 pages.

U.S. Appl. No. 17/175,569, "Non-Final Office Action", Sep. 27, 2021, 21 pages.

U.S. Appl. No. 17/175,569, "Notice of Allowance", May 17, 2023, 6 pages.

U.S. Appl. No. 17/175,569, "Supplemental Notice of Allowability", Aug. 17, 2023, 3 pages.

U.S. Appl. No. 17/175,573, "Corrected Notice of Allowability", Jul. 26, 2022, 2 pages.

U.S. Appl. No. 17/175,573, "Corrected Notice of Allowability", Sep. 2, 2022, 2 pages.

U.S. Appl. No. 17/175,573, "Non-Final Office Action", Mar. 15, 2022, 19 pages.

U.S. Appl. No. 17/175,573, "Notice of Allowance", Jul. 14, 2022, 9 pages.

U.S. Appl. No. 17/515,087, "Corrected Notice of Allowability", Jul. 14, 2023, 5 pages.

U.S. Appl. No. 17/515,087, "Final Office Action", Sep. 19, 2022, 12 pages.

U.S. Appl. No. 17/515,087, "Non-Final Office Action", Mar. 24, 2022, 10 pages.

U.S. Appl. No. 17/515,087, "Notice of Allowance", Mar. 17, 2023, 8 pages.

U.S. Appl. No. 17/515,093, "Corrected Notice of Allowability", Nov. 10, 2022, 4 pages.

U.S. Appl. No. 17/515,093, "Non-Final Office Action", May 25, 2022, 21 pages.

U.S. Appl. No. 17/515,093, "Notice of Allowance", Sep. 21, 2022, 7 pages.

U.S. Appl. No. 18/051,088, "Final Office Action", Jan. 17, 2024, 18 pages.

U.S. Appl. No. 18/051,088, "Non-Final Office Action", Sep. 20, 2023, 16 pages.

U.S. Appl. No. 18/078,897, "Corrected Notice of Allowability", Aug. 10, 2023, 6 pages.

U.S. Appl. No. 18/078,897, "Non-Final Office Action", Apr. 11, 2023, 12 pages.

U.S. Appl. No. 18/078,897, "Notice of Allowance", Jul. 26, 2023, 7 pages.

Bari, et al., "Data Center Network Virtualization: A Survey", Institute of Electrical and Electronics Engineers Communications Surveys & Tutorials, vol. 15, No. 2, Jan. 2013, pp. 909-928.

Eisenbud, et al., "Maglev: A Fast and Reliable Software Network Load Balancer", 13th USENIX Symposium on Networked Systems Design and Implementation, Mar. 2016, 13 pages.

Firestone, et al., "Azure Accelerated Networking: SmartNICs in the Public Cloud", 15th USENIX Symposium on Networked Systems Design and Implementation, Apr. 2018, 14 pages.

Li, et al., "A Novel Hardware-Assisted Virtualization Approach for Network Interface Card", International Conference on Research Challenges in Computer Science, Dec. 28, 2009, pp. 225-228.

Liebeherr, et al., "Experimental Evaluation of Address Binding Dissemination in Multi-Substrate Overlay Networks", Business, Computer Science, Sep. 2011, 8 pages.

Luo, et al., "Accelerated Virtual Switching with Programmable NICs for Scalable Data Center Networking", Proceedings of the

(56)

References Cited

OTHER PUBLICATIONS

Second Association for Computing Machinery SIGCOMM Workshop on Virtualized Infrastructure Systems and Architectures, Sep. 2010, pp. 65-72.
 Nakajima, et al., "High-Performance vNIC Framework for Hypervisor-Based NFV with Userspace vSwitch", Fourth European Workshop on Software Defined Networks, IEEE, 2015, pp. 43-48.
 International Application No. PCT/US2022/034751, "International Search Report and Written Opinion", Sep. 23, 2022, 13 pages.
 International Application No. PCT/US2022/034751, "Written Opinion", Aug. 8, 2023, 10 pages.
 International Application No. PCT/US2022/034753, "International Search Report and Written Opinion", Sep. 23, 2022, 13 pages.
 International Application No. PCT/US2022/034753, "Written Opinion", Aug. 30, 2023, 22 pages.
 Wang, et al., "Expeditus: Congestion-Aware Load Balancing in Clos Data Center Networks", Institute of Electrical and Electronics Engineers/Association for Computing Machinery Transactions on Networking, vol. 25, No. 5, Oct. 2017, pp. 3175-3188.

U.S. Appl. No. 18/051,088, Notice of Allowance, Mailed On Mar. 14, 2025, 9 pages.
 U.S. Appl. No. 18/233,779, Non-Final Office Action, Mailed On Feb. 12, 2025, 19 pages.
 U.S. Appl. No. 18/393,454, Non-Final Office Action, Mailed On Dec. 2, 2024, 35 pages.
 U.S. Appl. No. 18/051,088, "Corrected Notice of Allowability", May 28, 2025, 2 pages.
 U.S. Appl. No. 18/233,779, Non-Final Office Action, Mailed On May 28, 2025, 23 pages.
 U.S. Appl. No. 18/393,454, Final Office Action mailed Jul. 1, 2025, 9 pages.
 International Application No. PCT/US2023/085686, International Preliminary Report on Patentability mailed Jul. 3, 2025, 11 pages.
 International Application No. PCT/US2023/085695, International Preliminary Report on Patentability mailed Jul. 3, 2025, 11 pages.
 Indian Application No. 202447022236, First Examination Report mailed Jul. 9, 2025, 11 pages.
 Indian Application No. 202447024560, First Examination Report mailed Jul. 9, 2025, 9 pages.

* cited by examiner

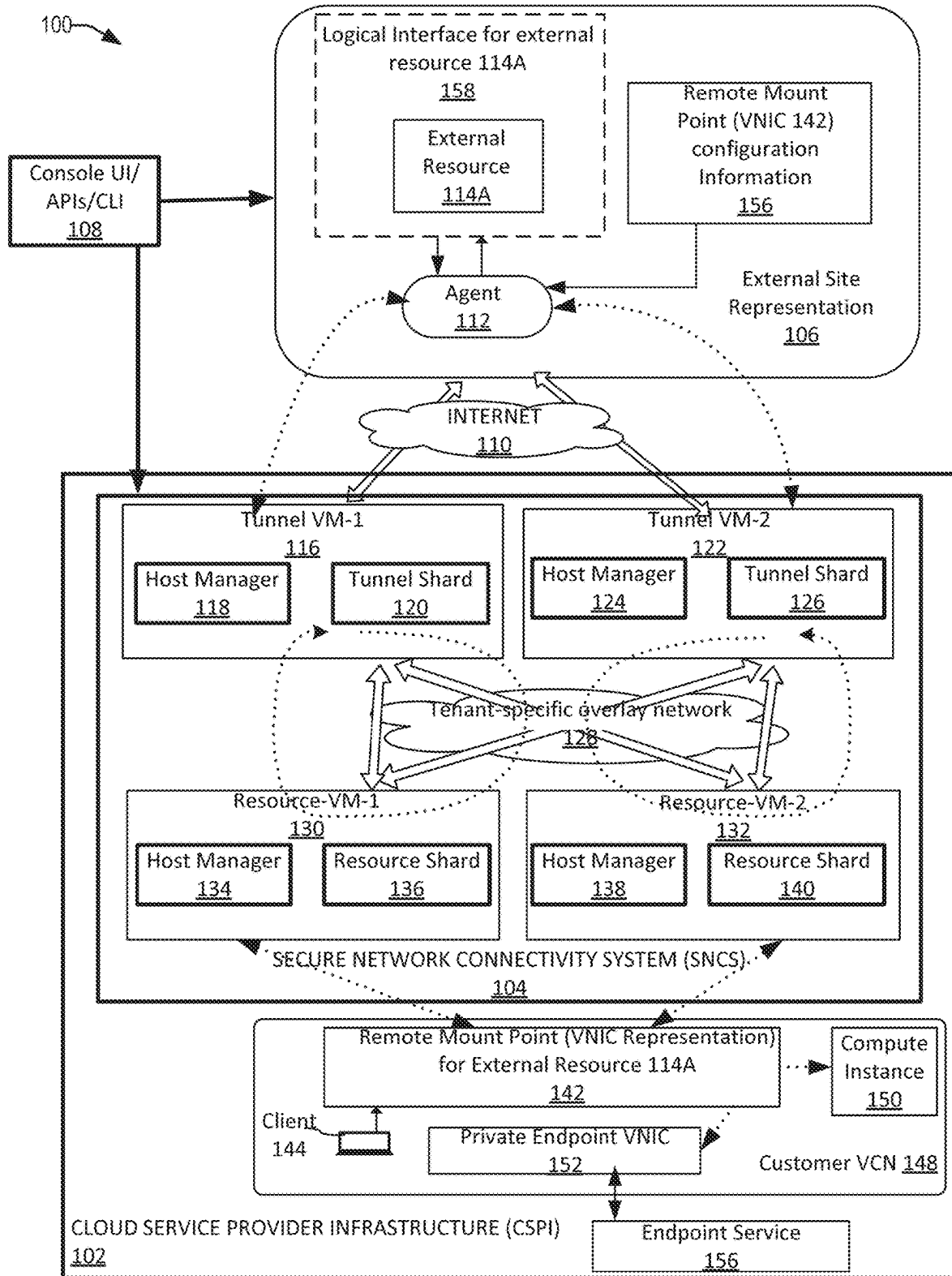
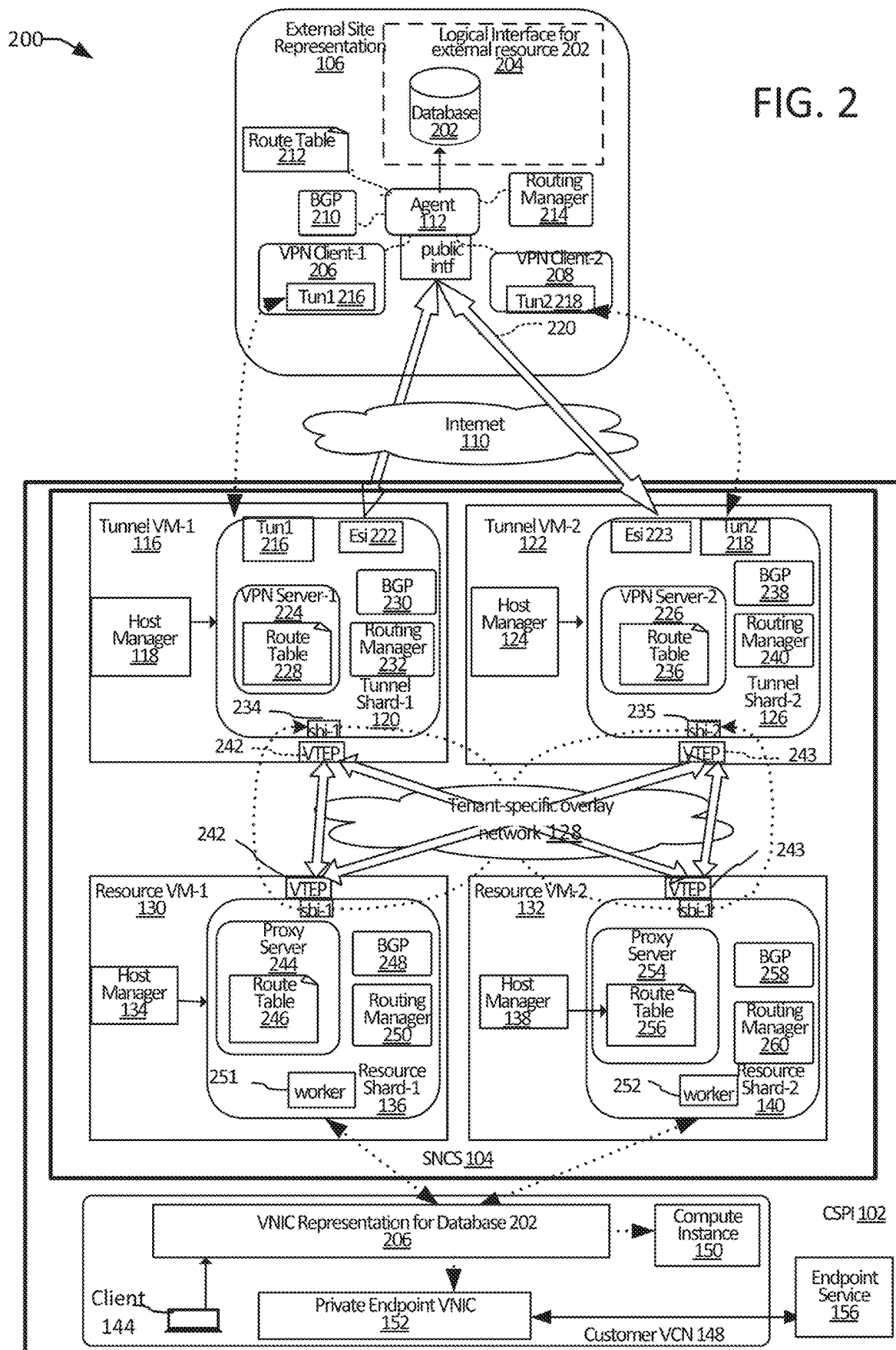


FIG. 1

FIG. 2



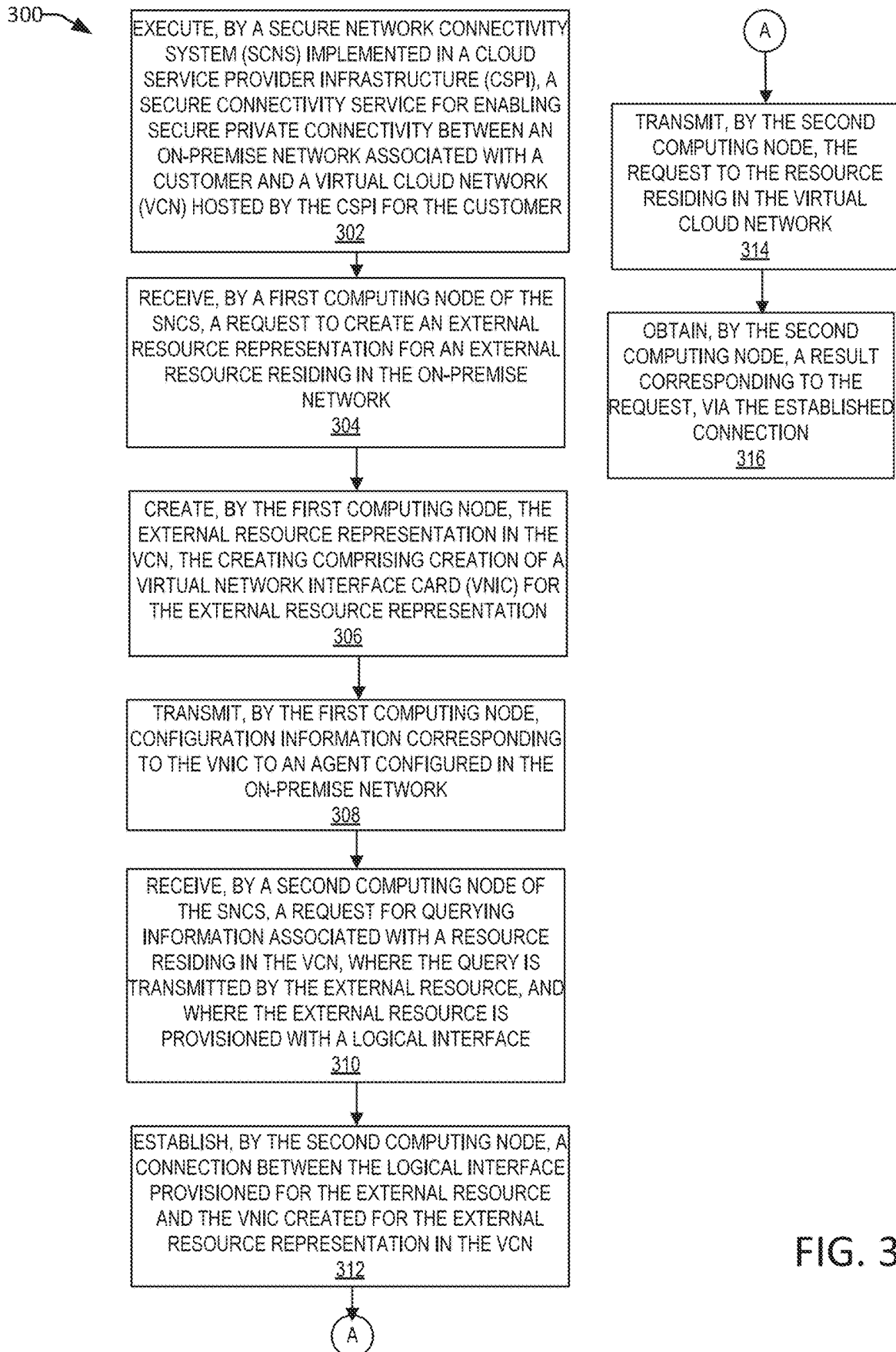


FIG. 3

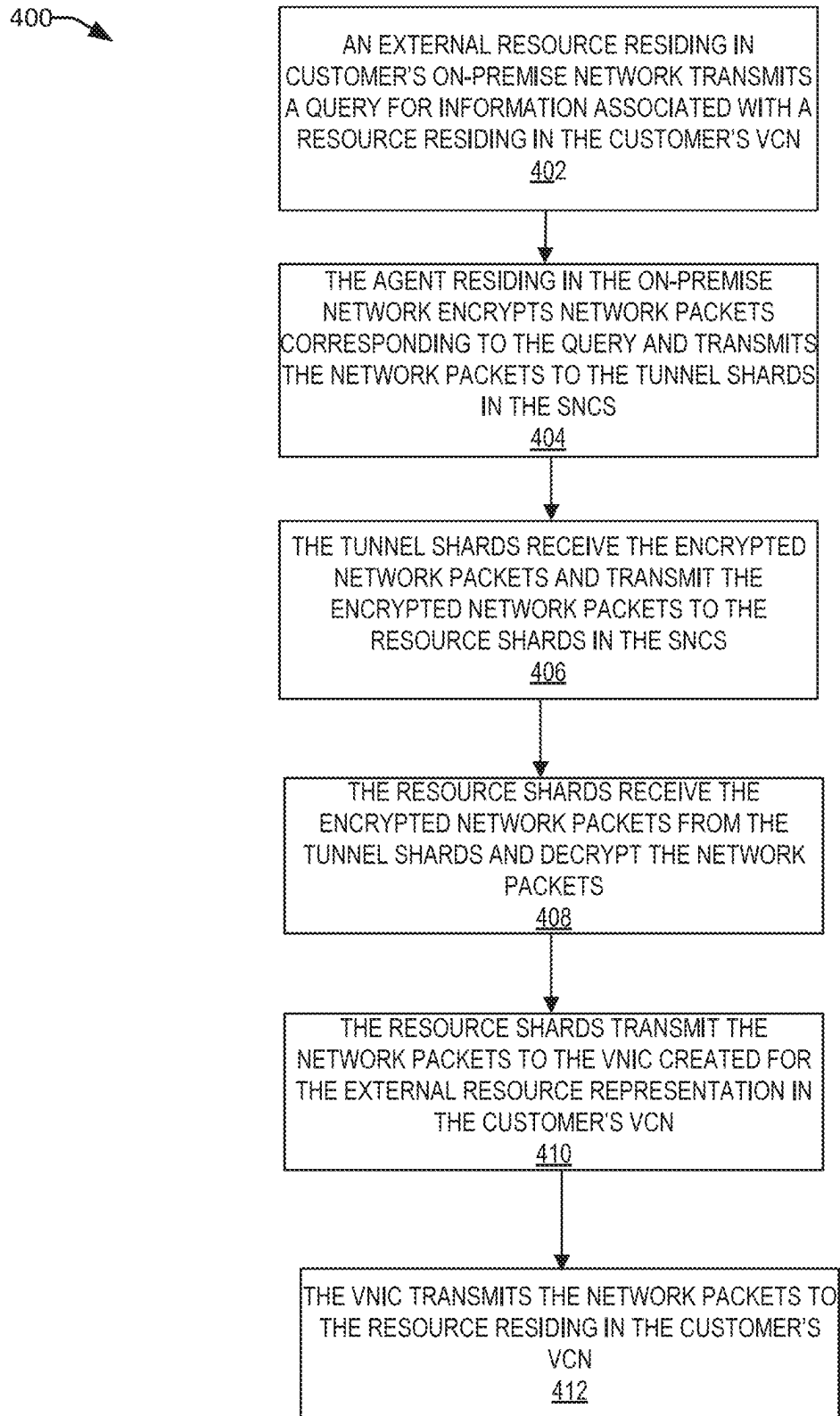


FIG. 4

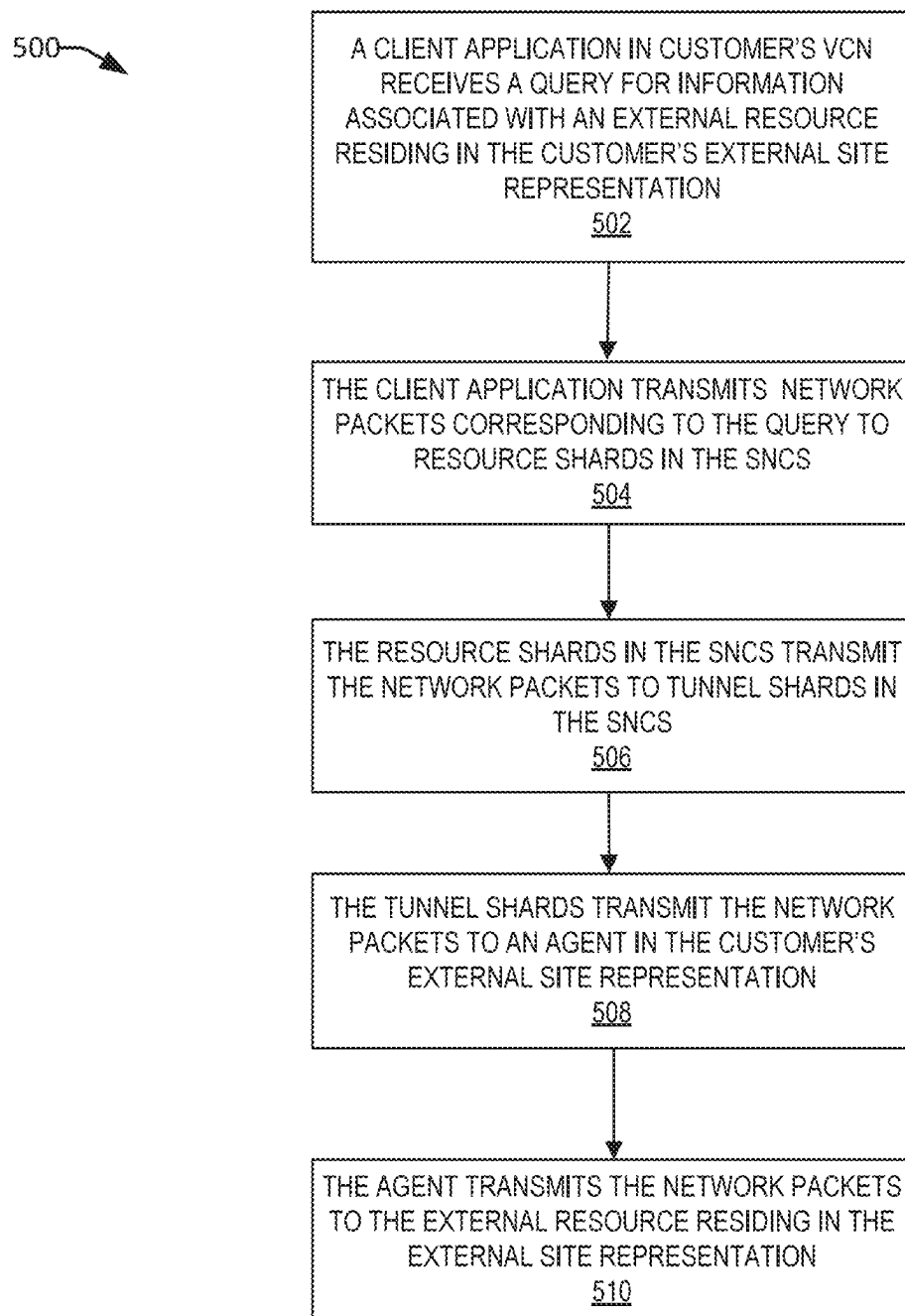


FIG. 5

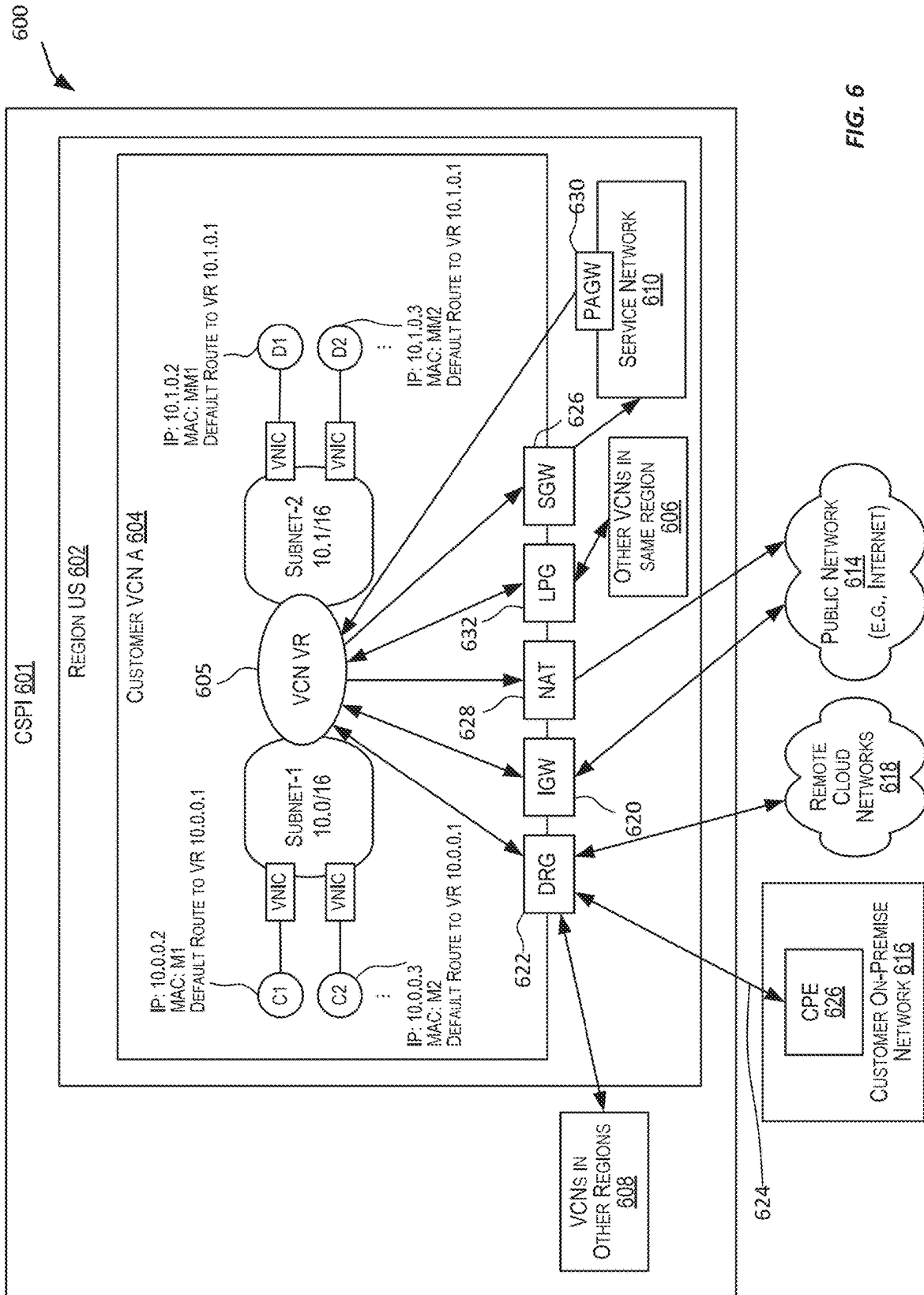
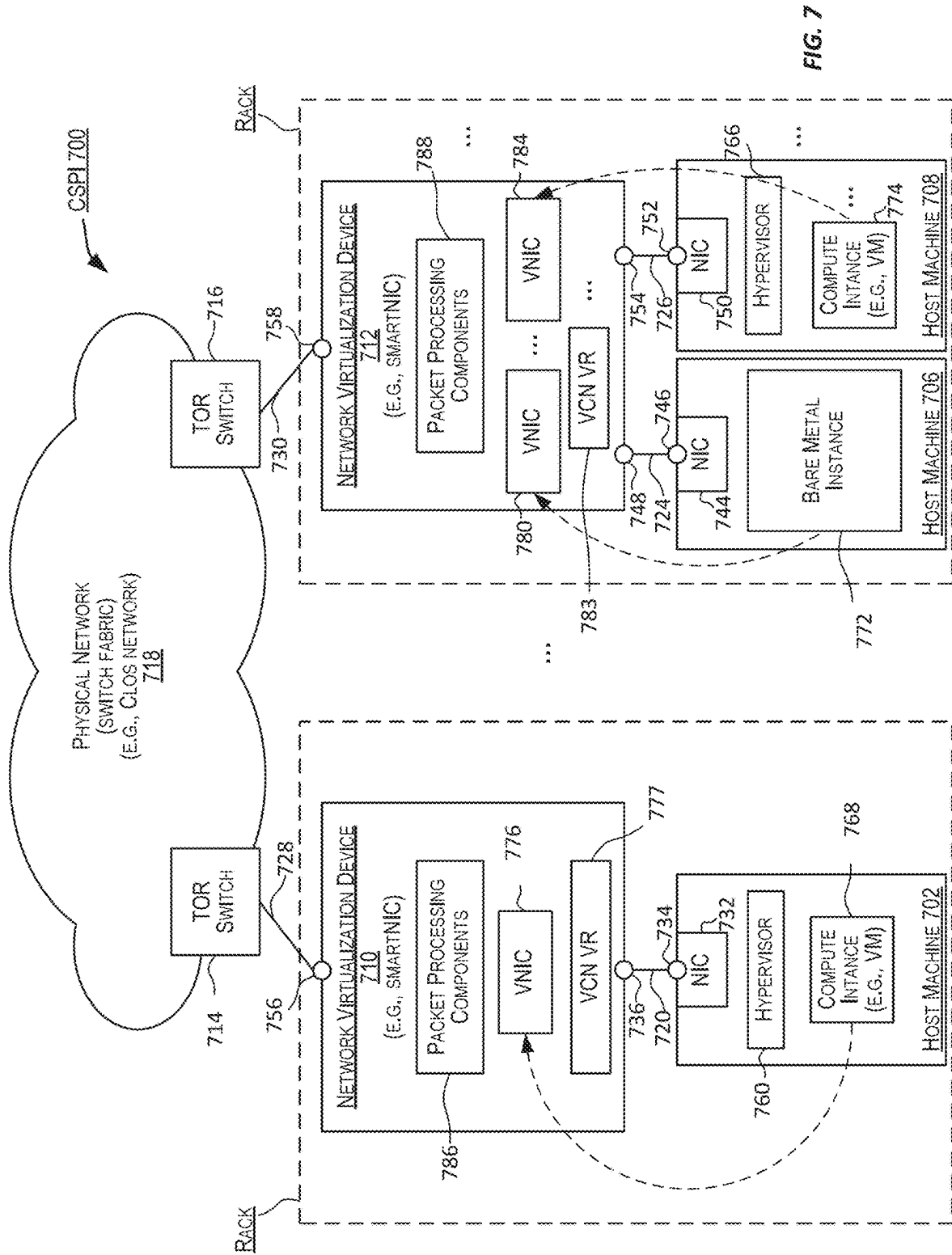
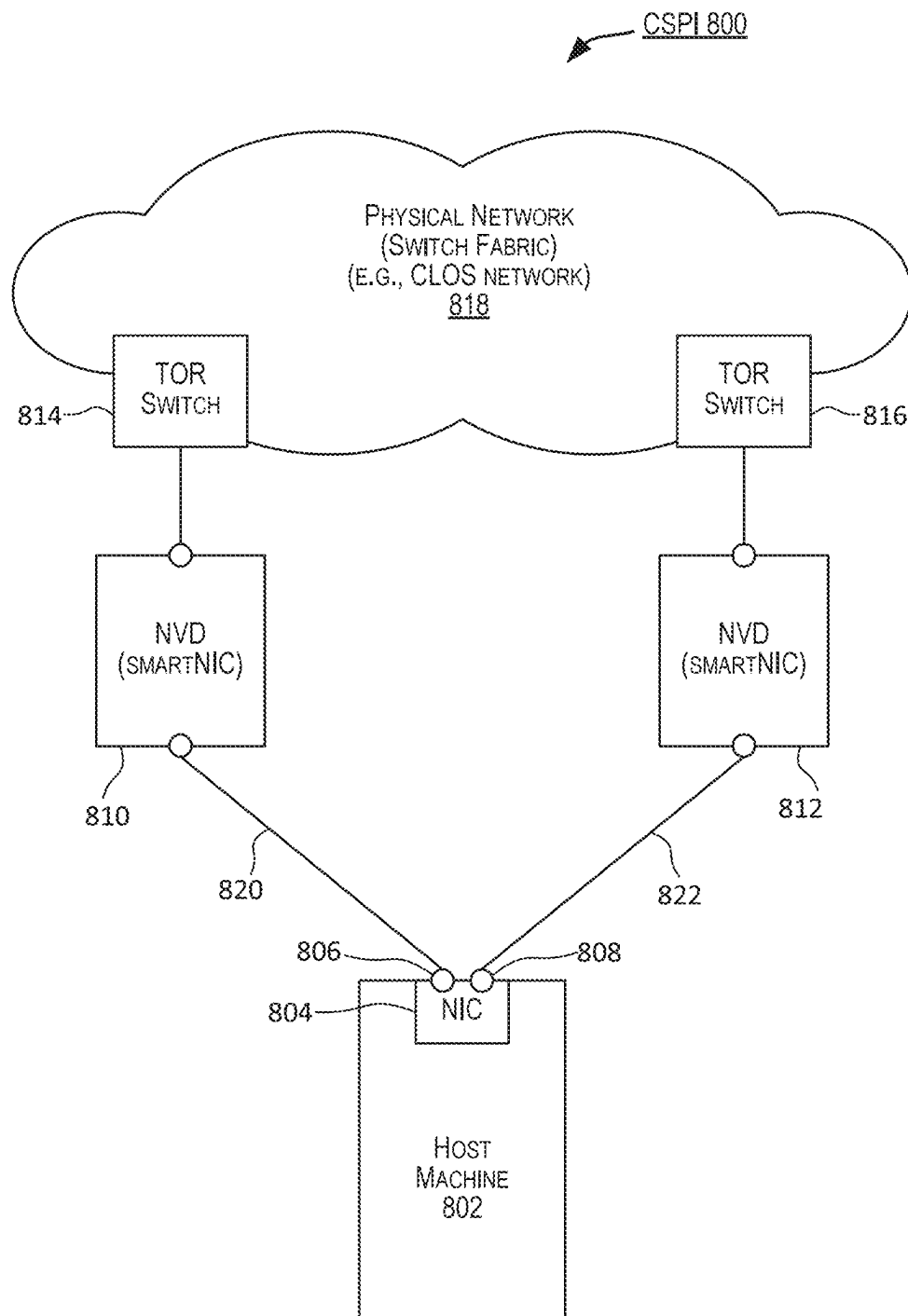
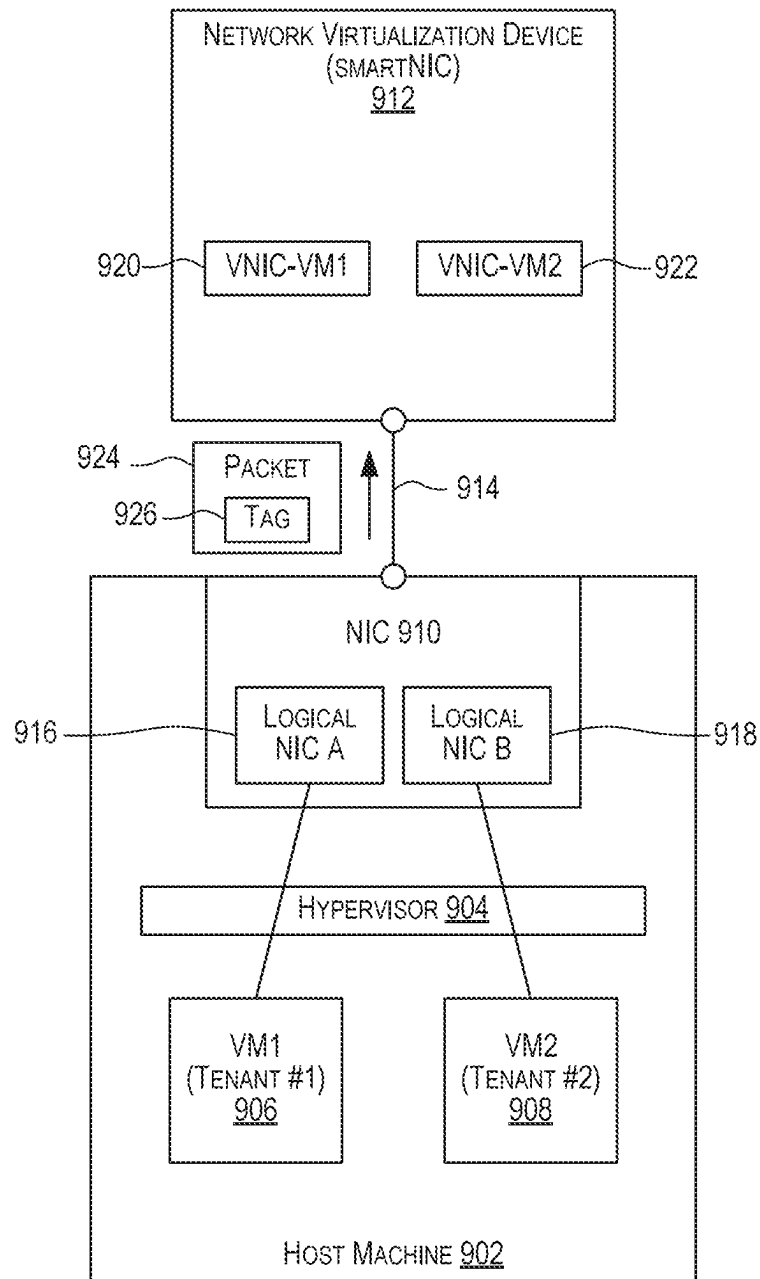


FIG. 6



**FIG. 8**

**FIG. 9**

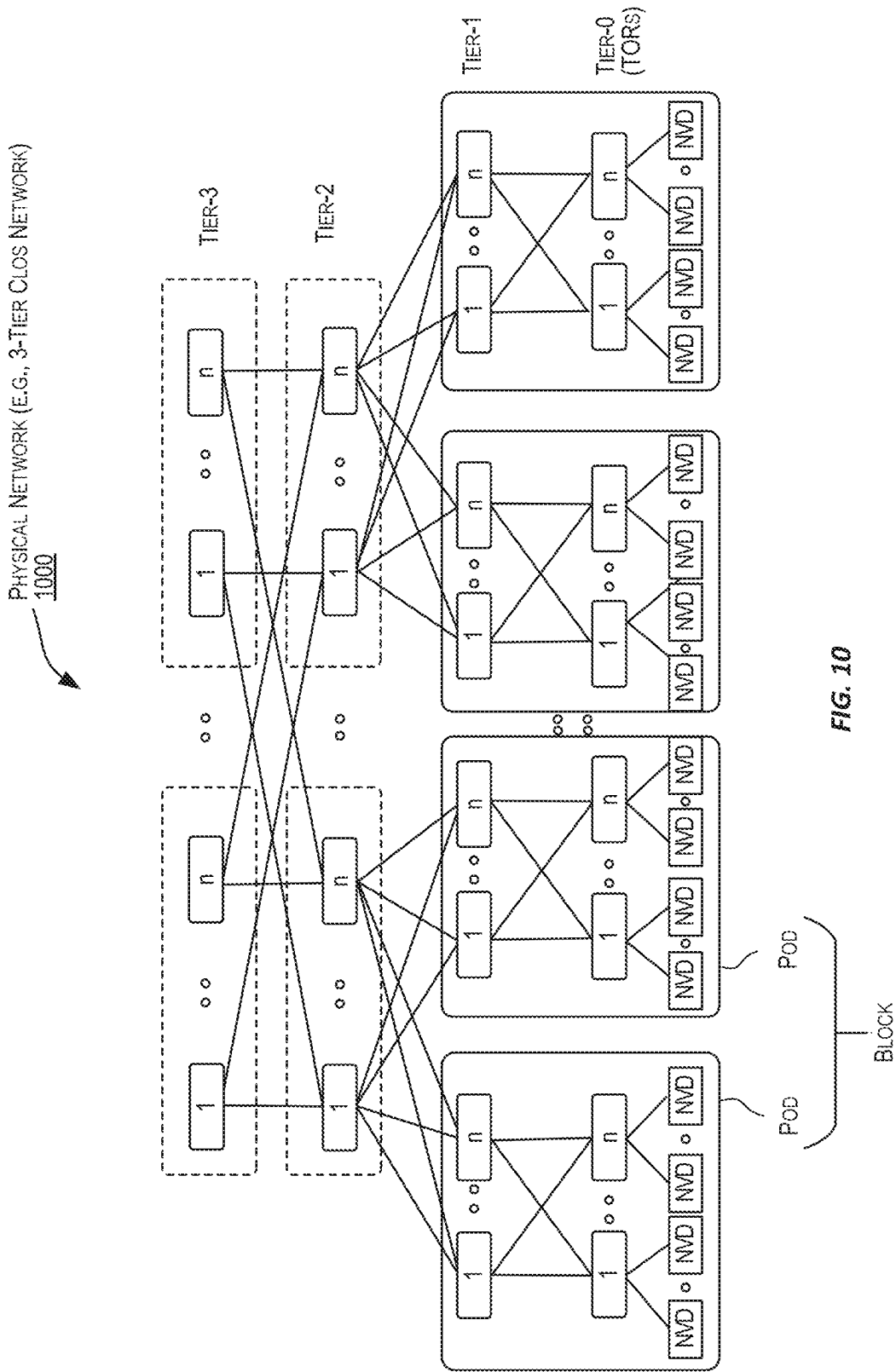
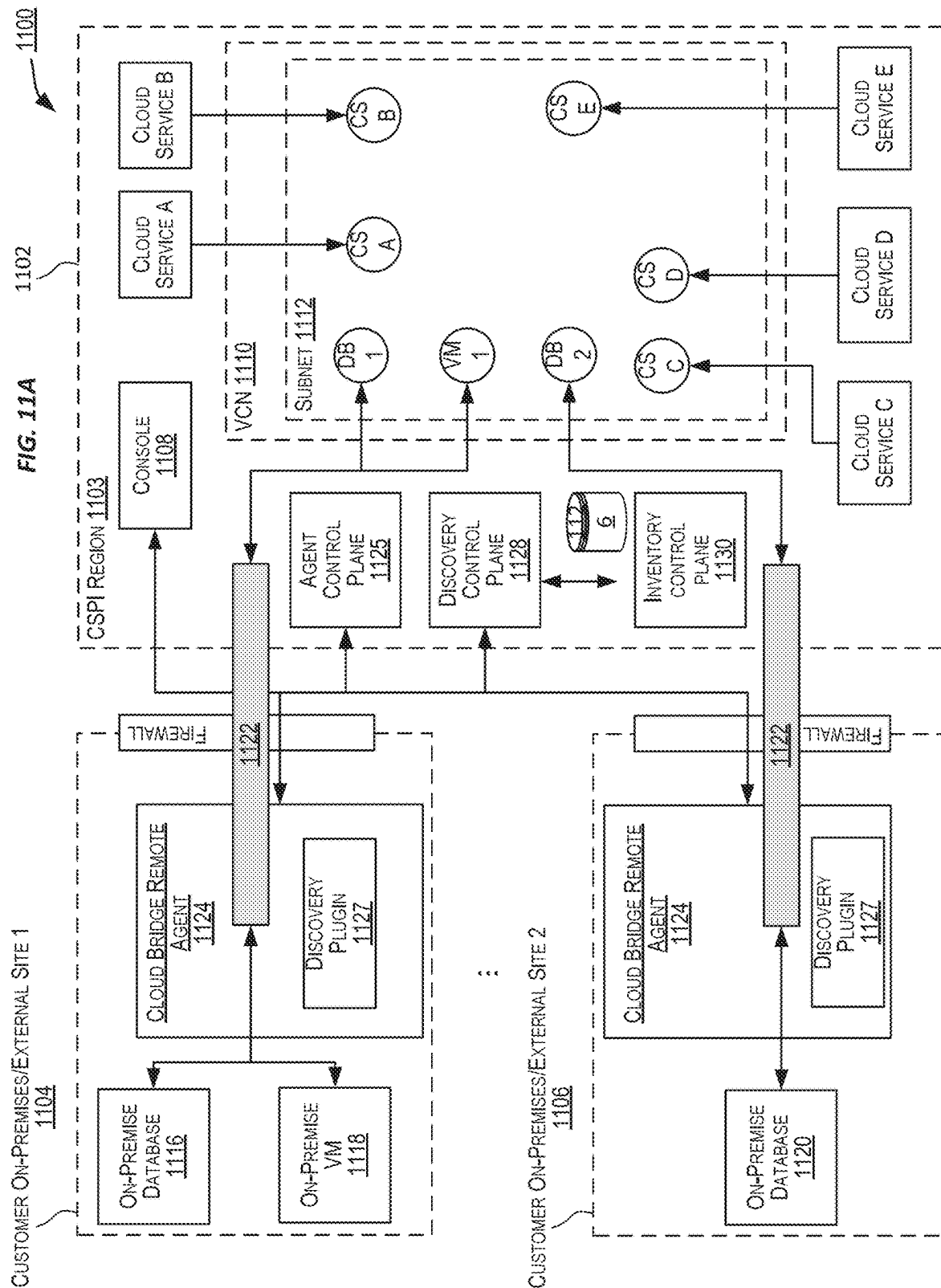


FIG. 11A



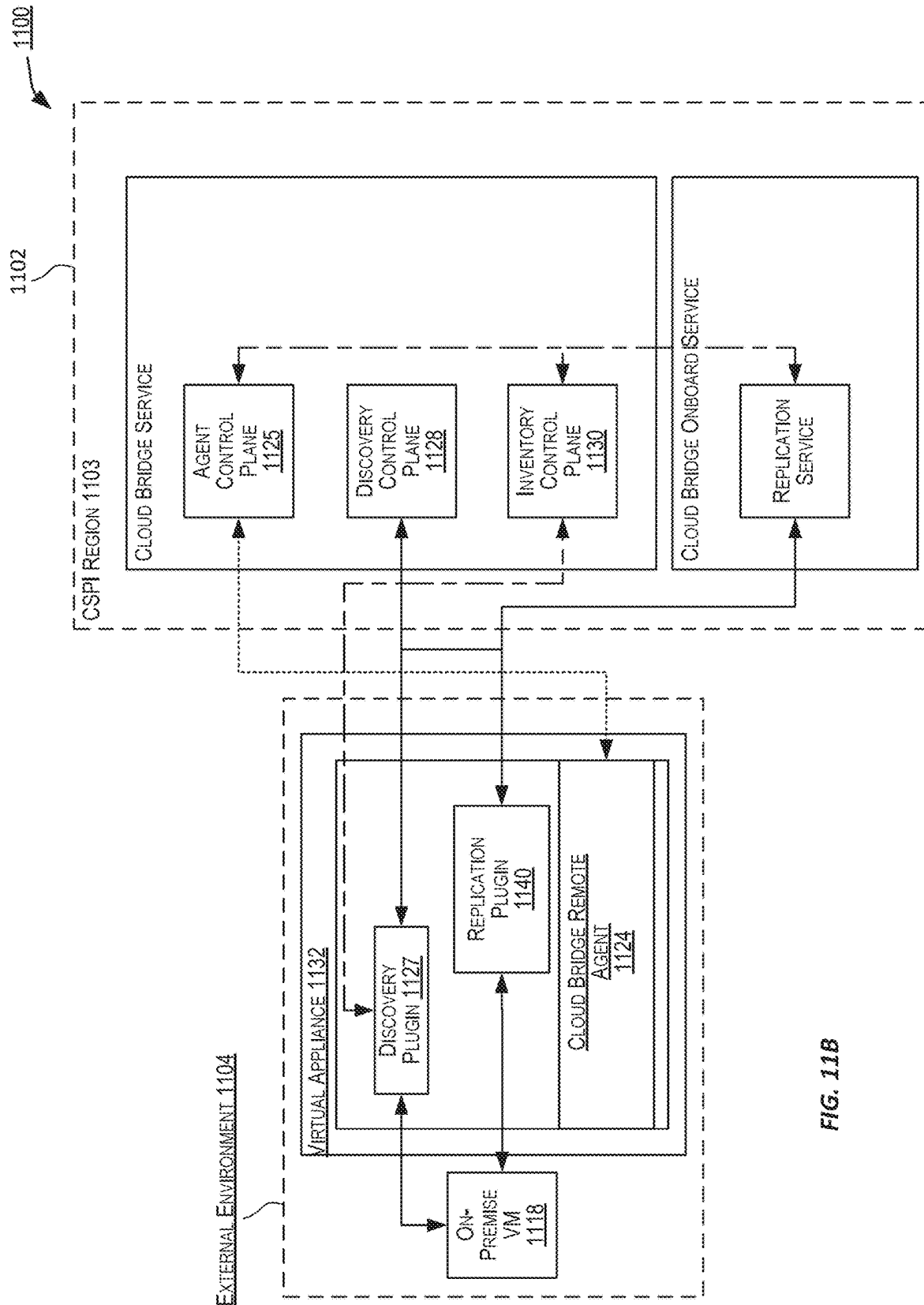


FIG. 11B

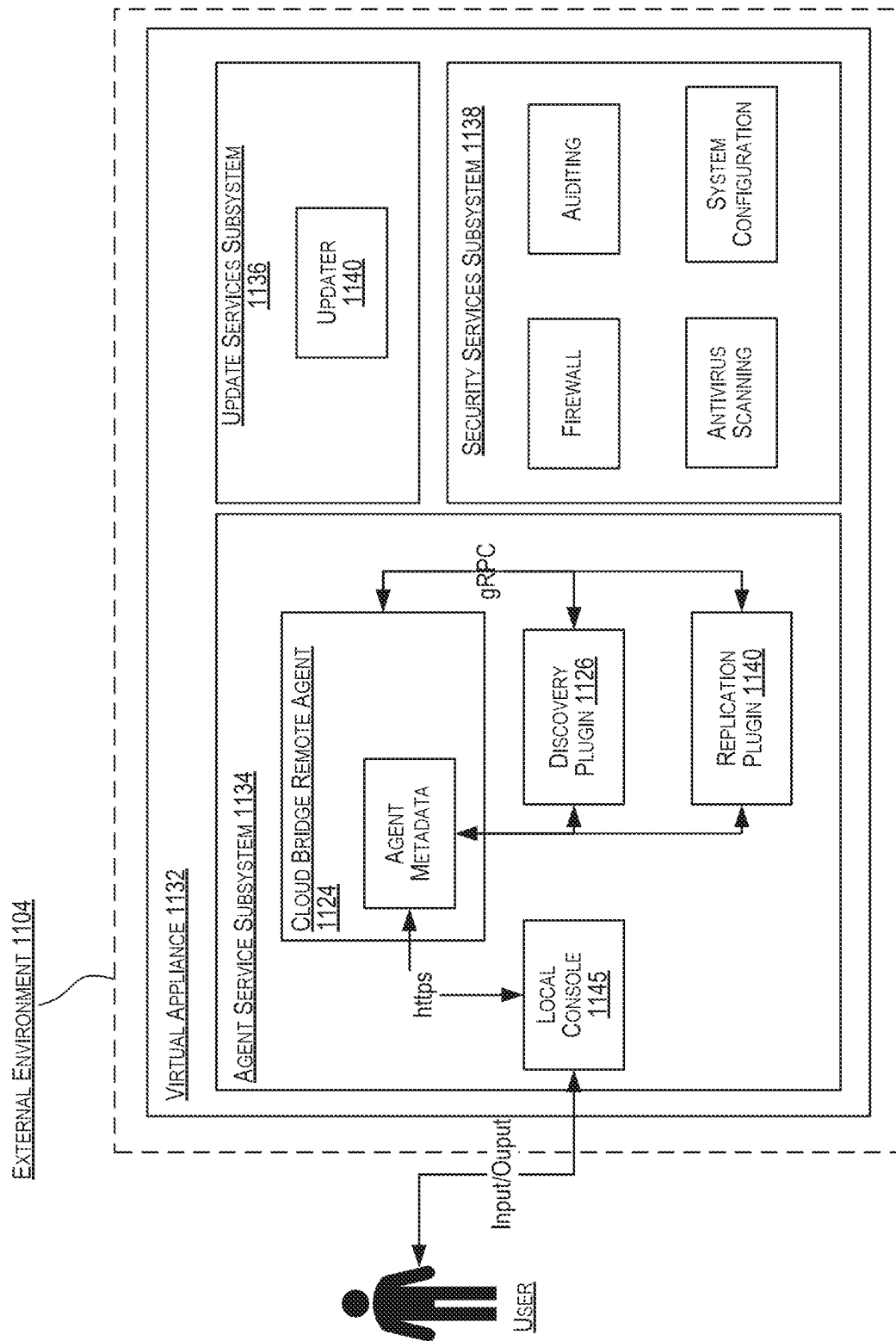


FIG. 11C

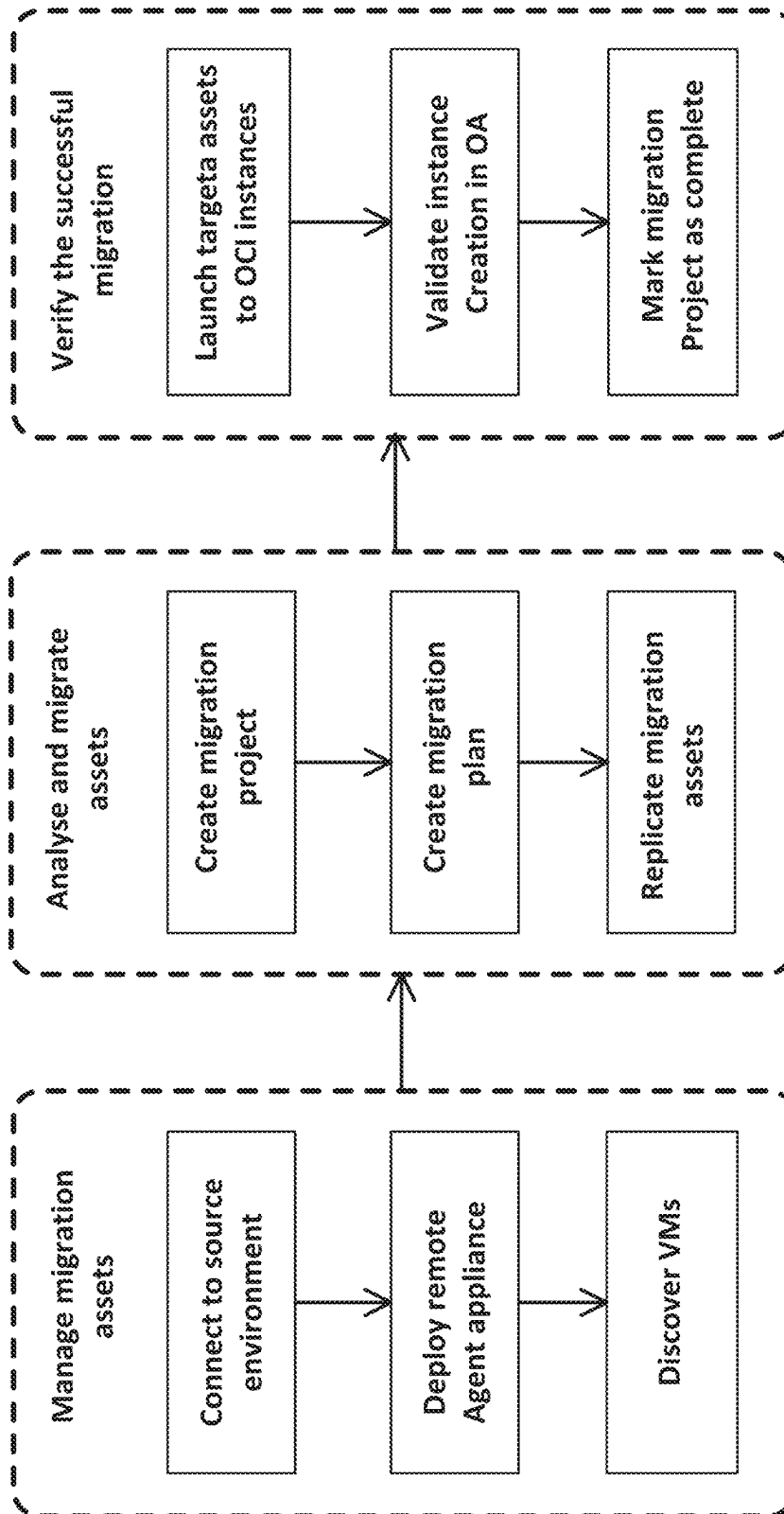


FIG. 12

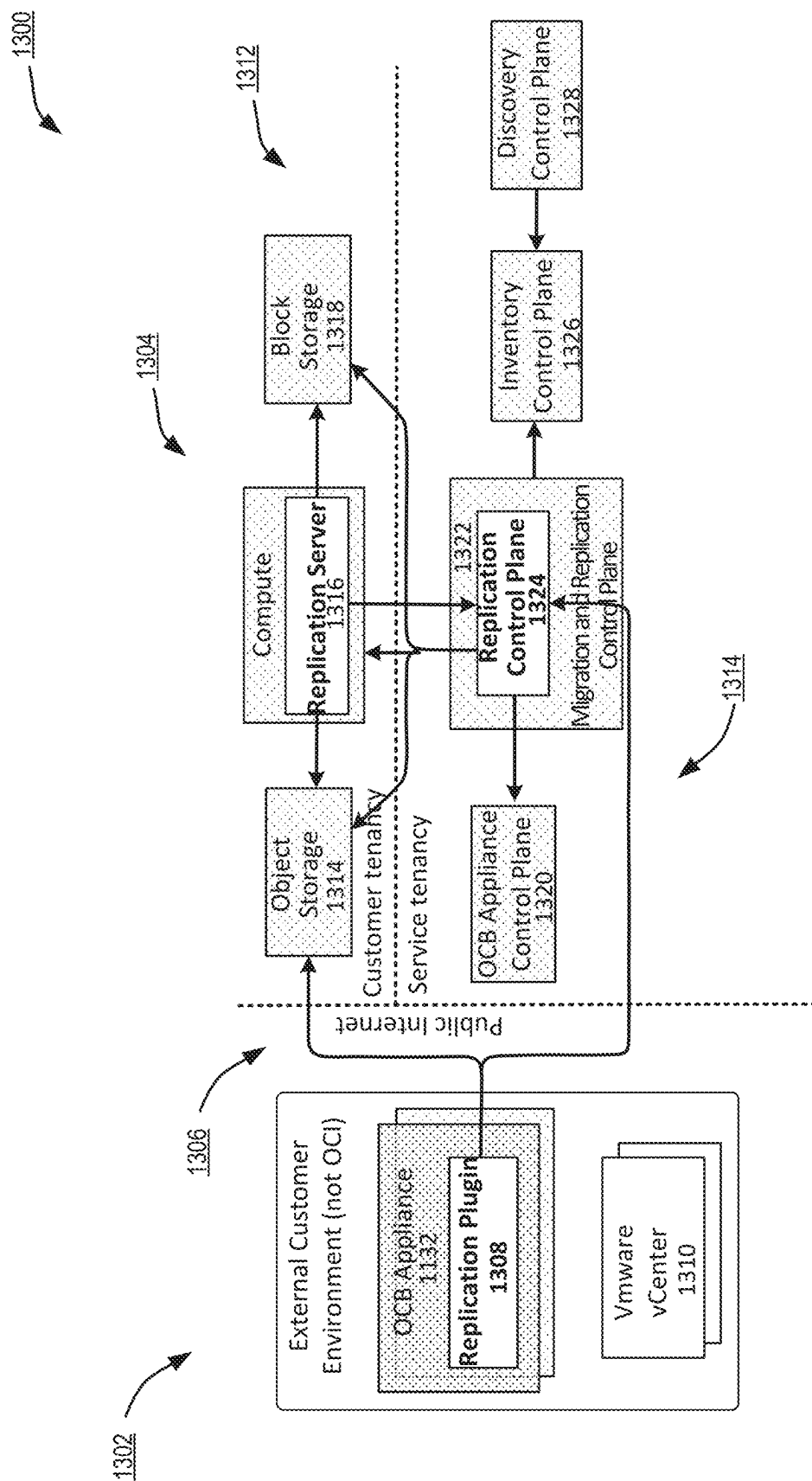


FIG. 13

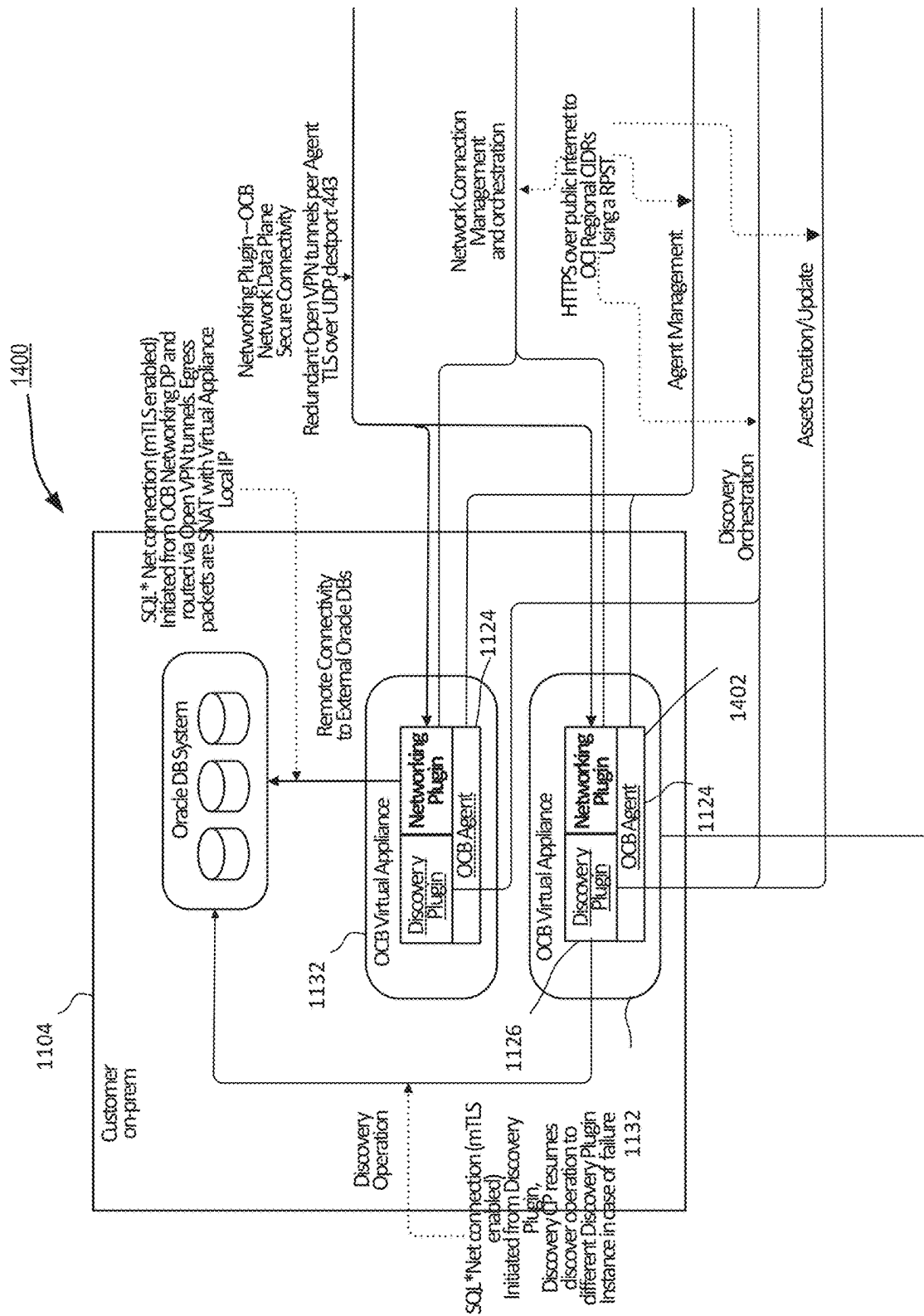


FIG. 14

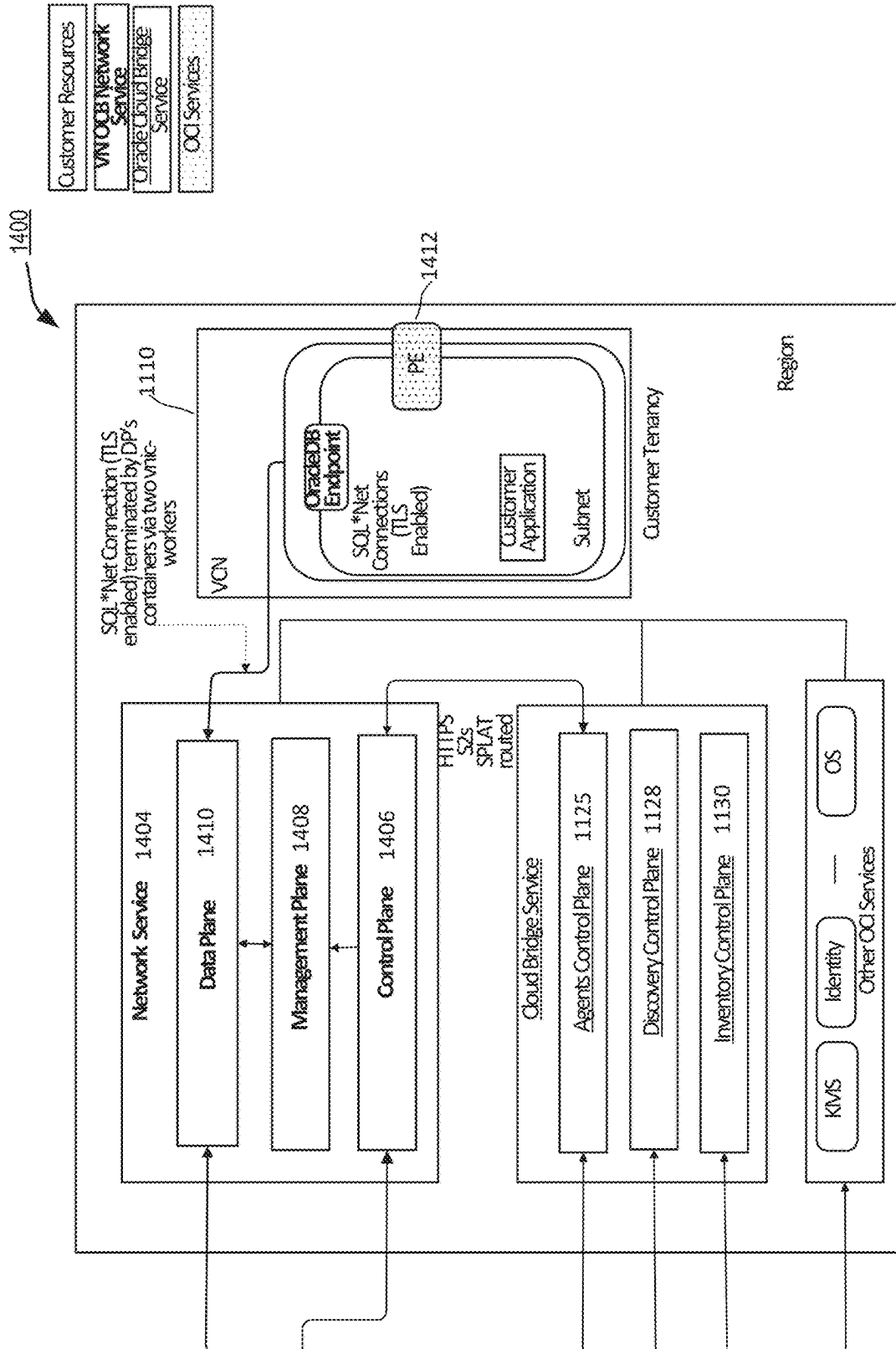


FIG. 14 CONT

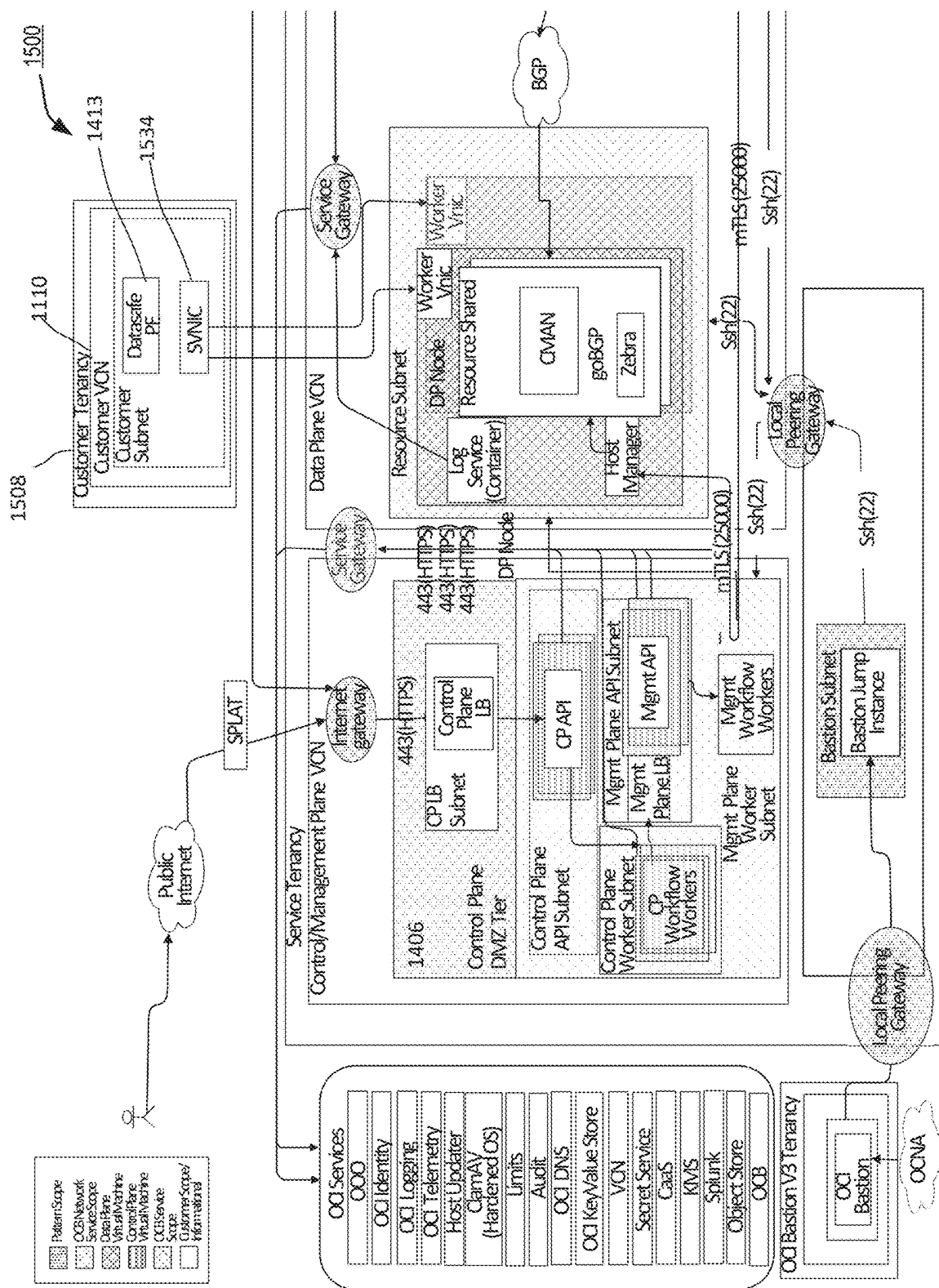


FIG. 15

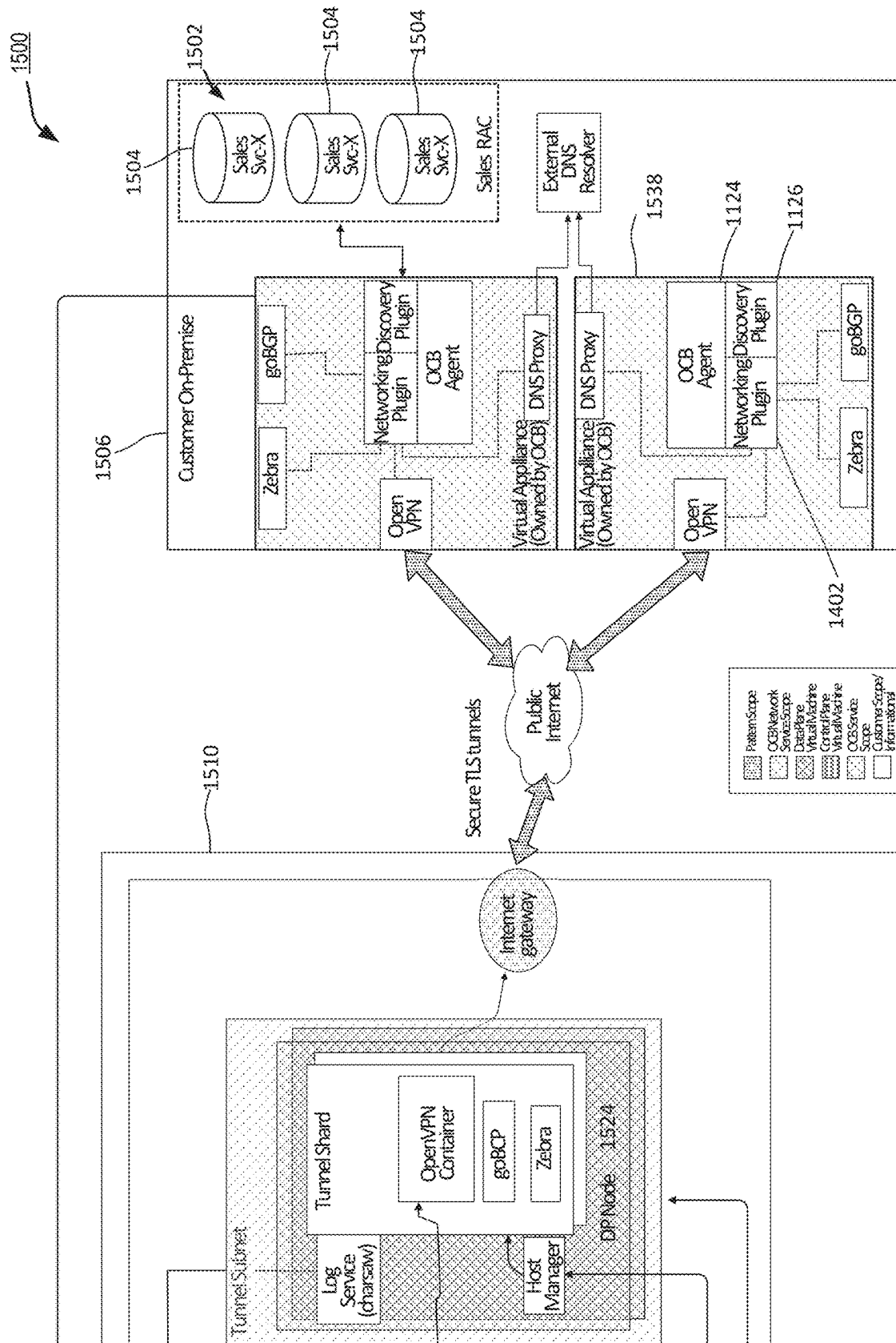


FIG. 15 CONT

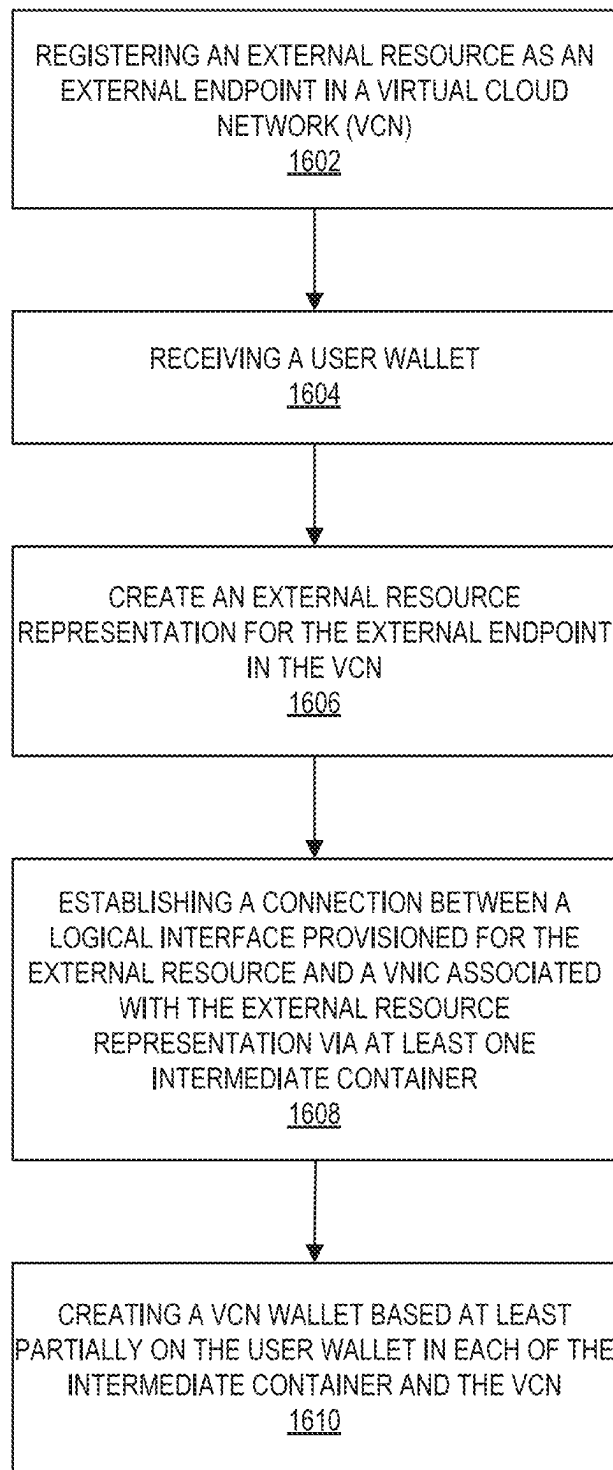
1600 

FIG. 16

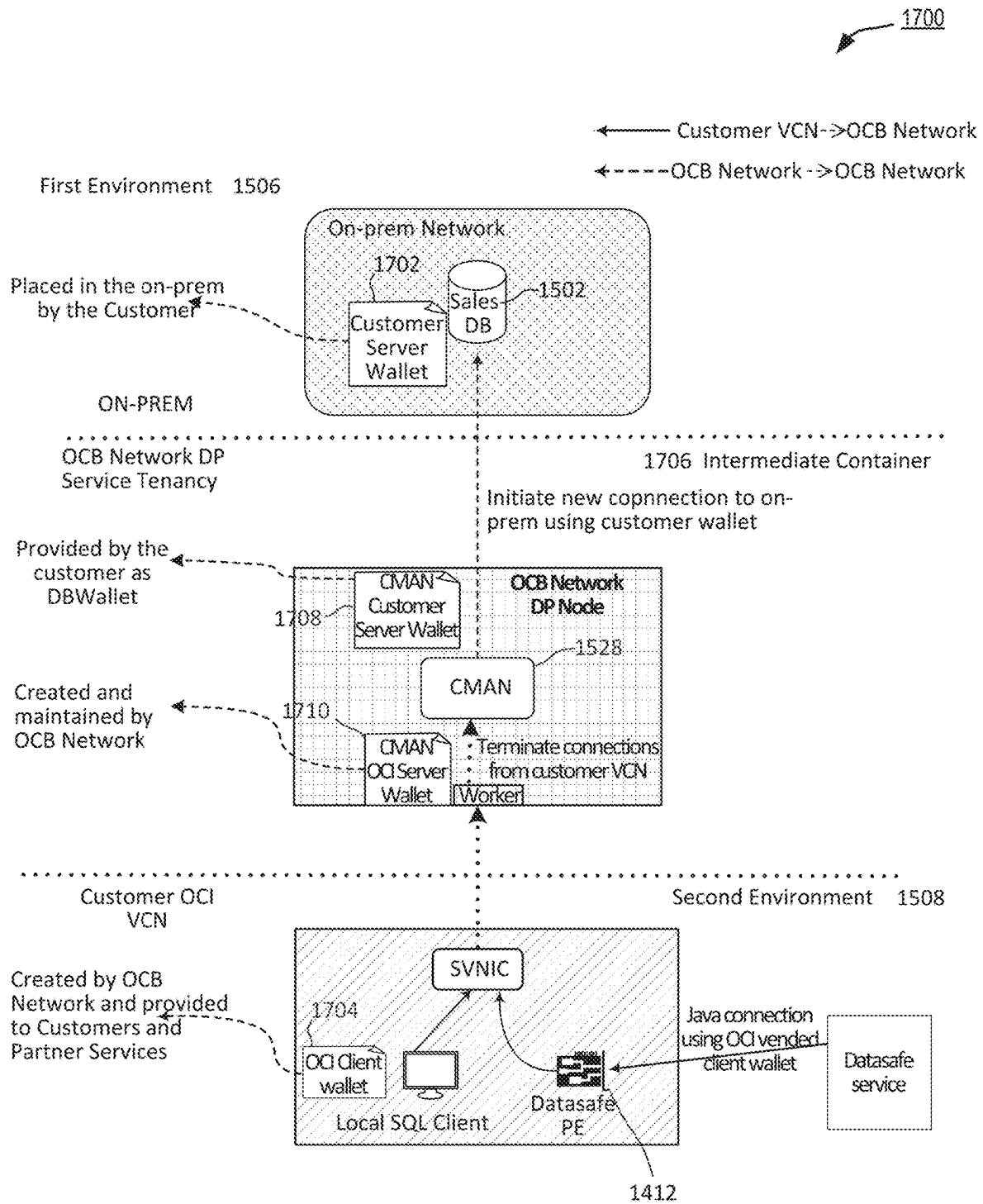


FIG. 17

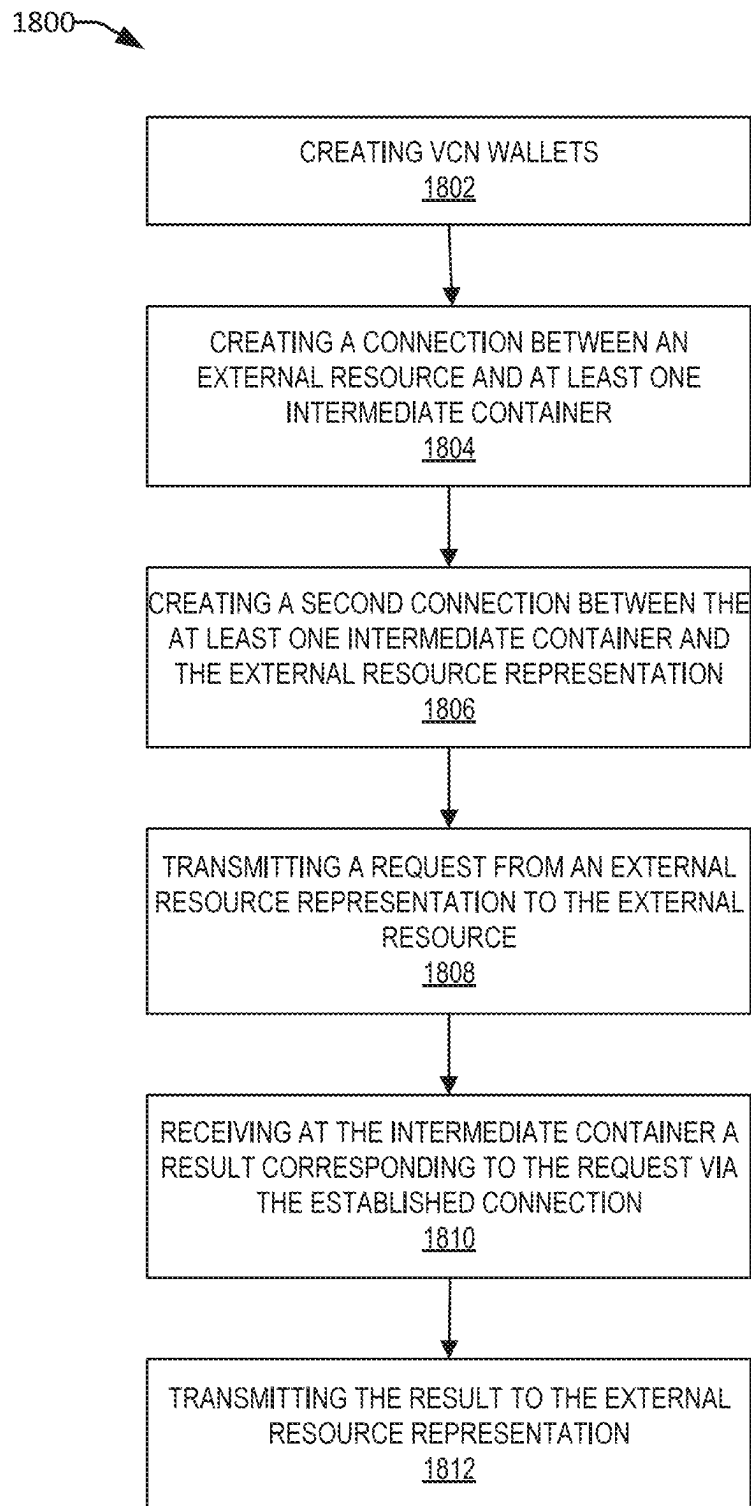
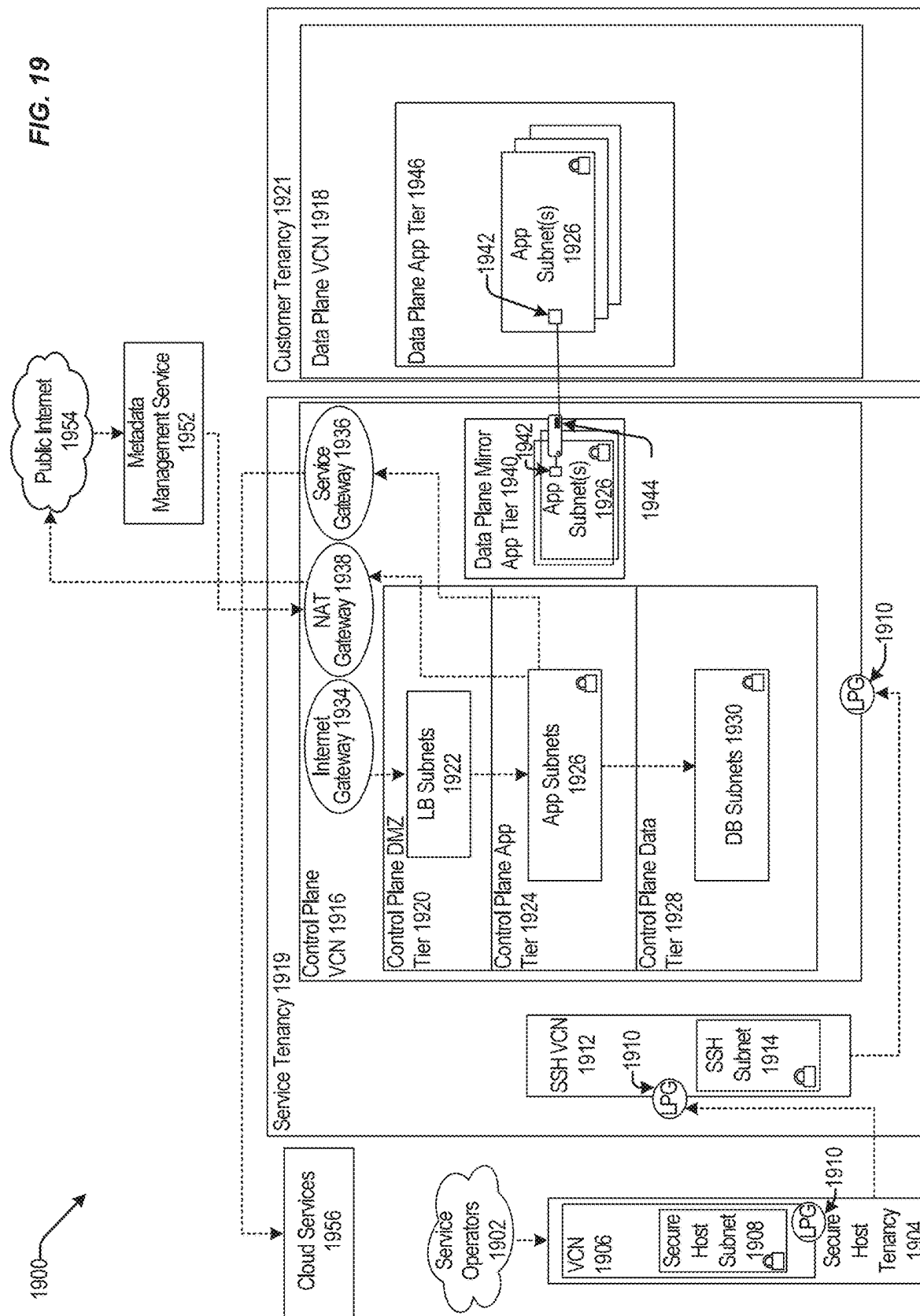
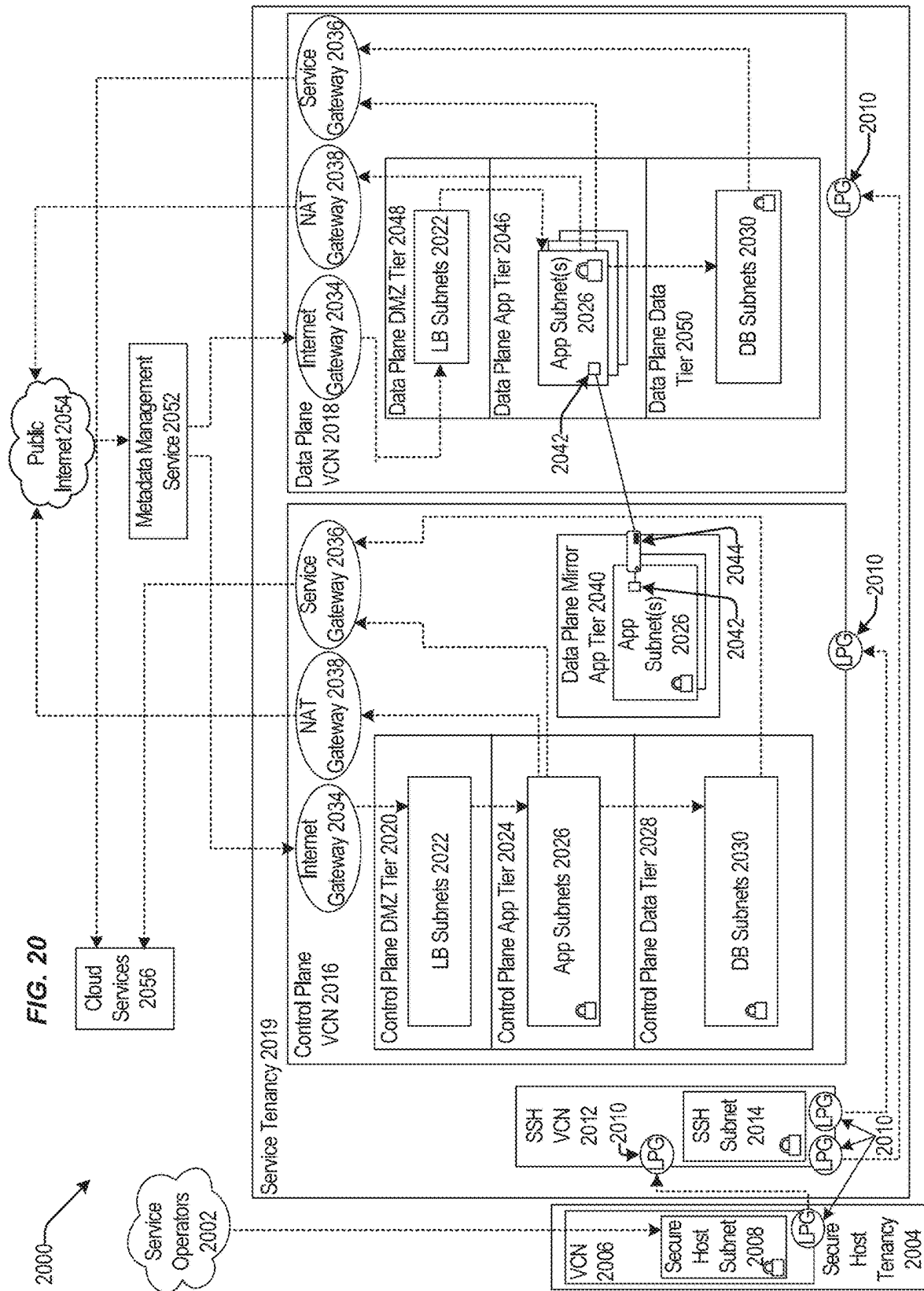
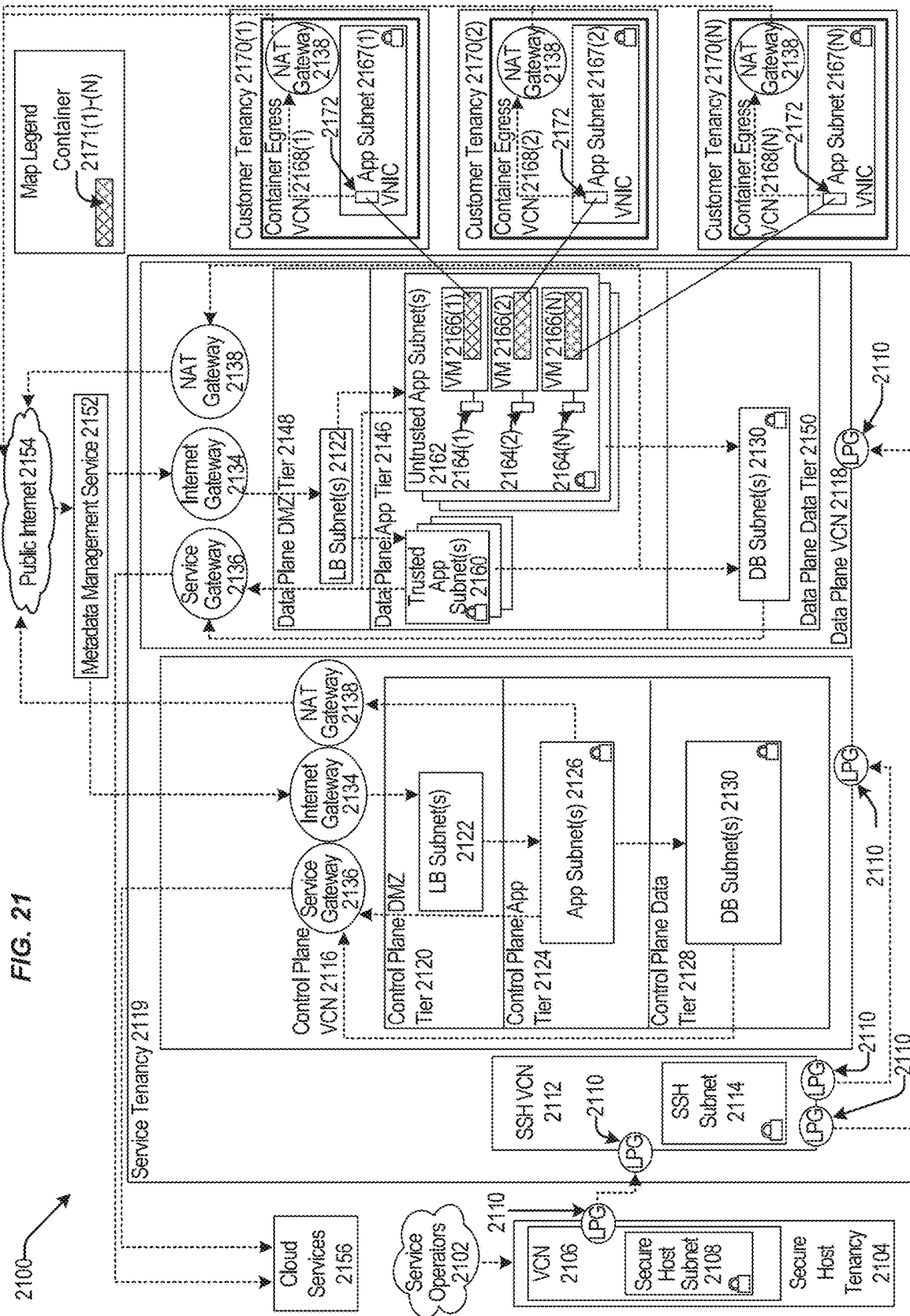


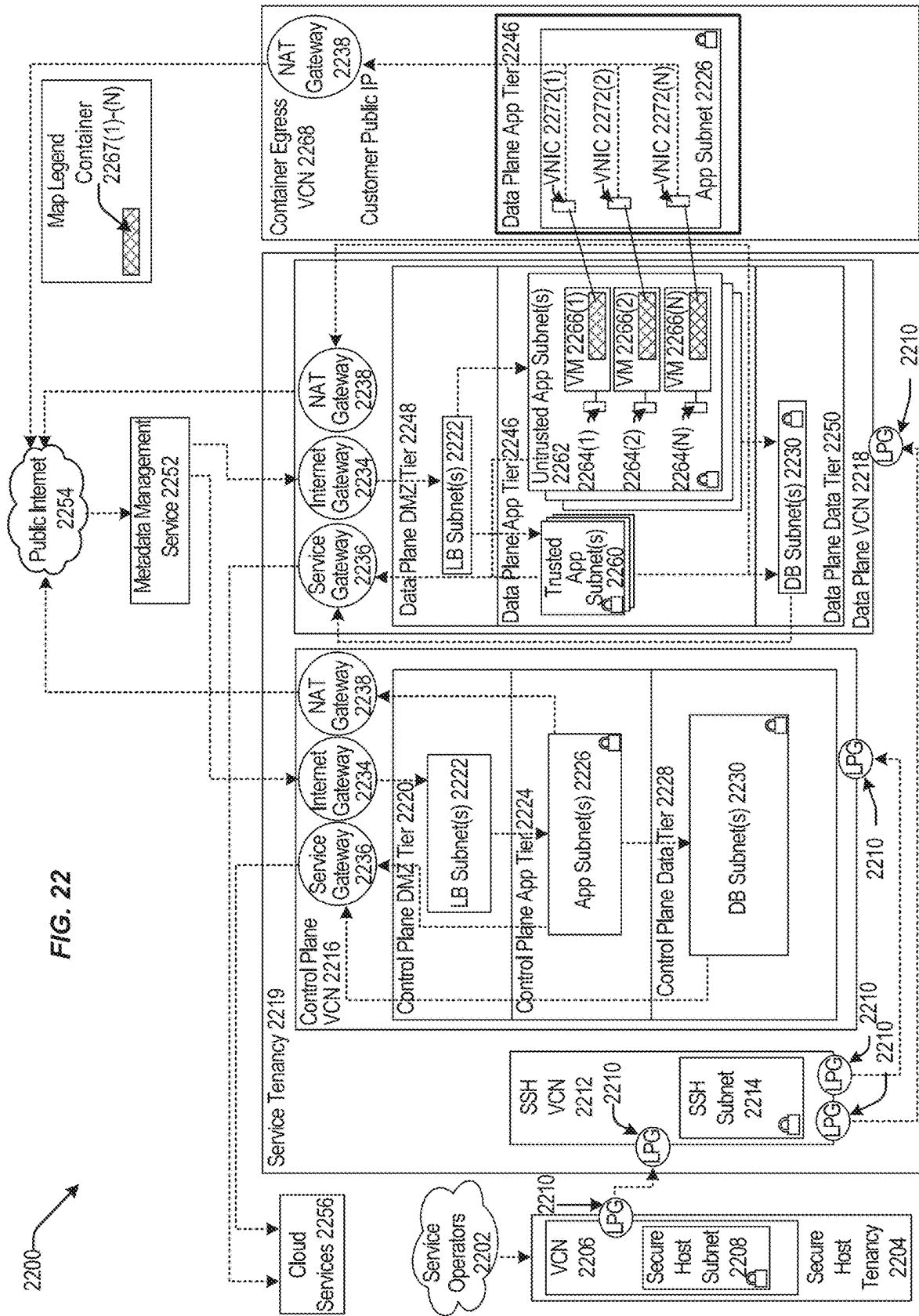
FIG. 18

Fig. 19









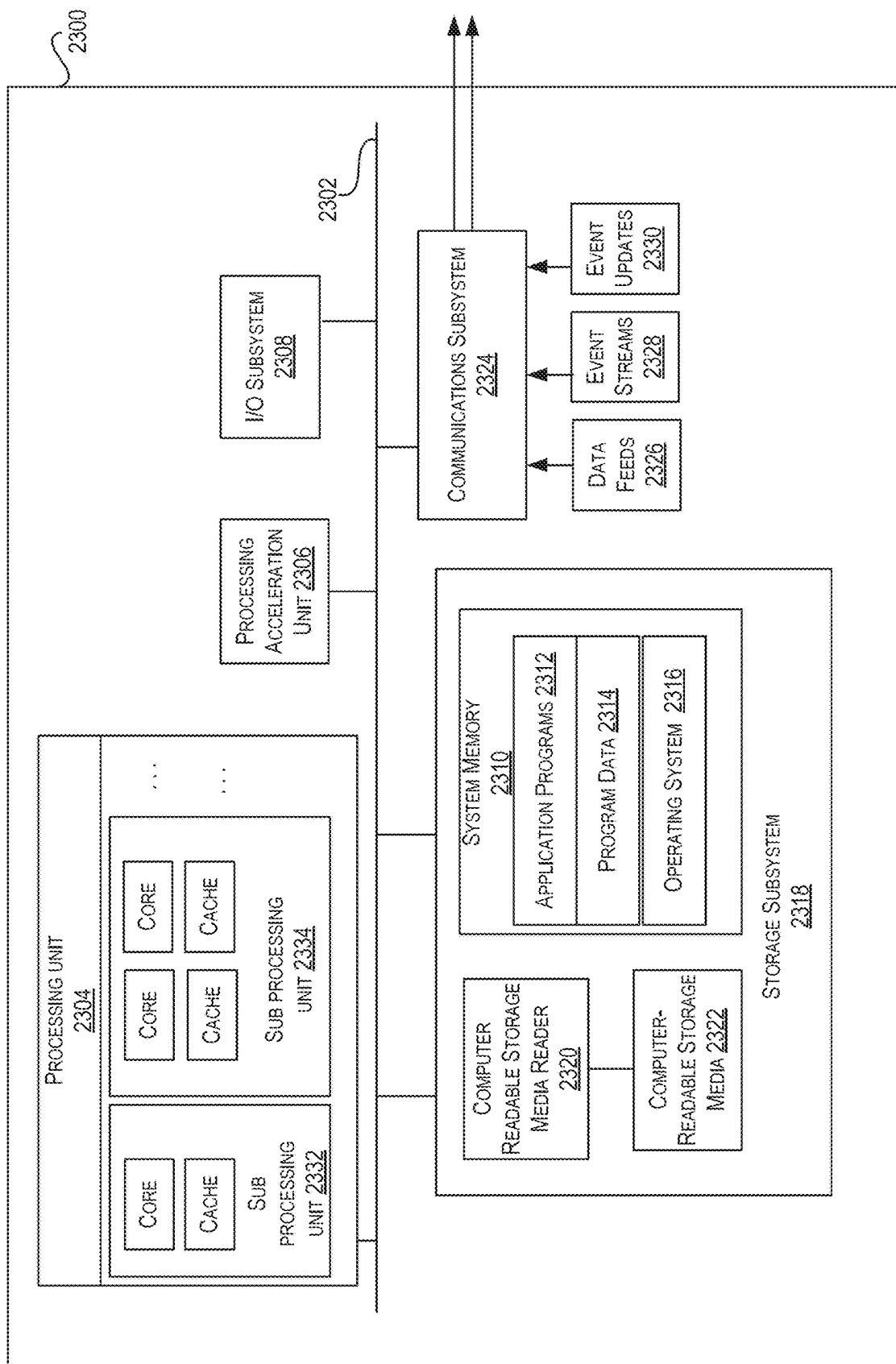


FIG. 23

SECURE BI-DIRECTIONAL NETWORK CONNECTIVITY SYSTEM BETWEEN PRIVATE NETWORKS

CROSS-REFERENCES TO RELATED APPLICATIONS

The present application is a continuation-in-part, and claims the benefit of and priority to, U.S. patent application Ser. No. 18/373,698, filed on Sep. 27, 2023, which is a continuation of, and claims the benefit of and priority to, U.S. patent application Ser. No. 18/078,897, filed on Dec. 9, 2022, which is a continuation of, and claims the benefit of priority to, U.S. patent application Ser. No. 17/515,093, filed Oct. 29, 2021, now U.S. Pat. No. 11,558,245, the entire contents of each of which are incorporated herein by reference for all purposes. The present application claims the benefit of the following applications:

- (1) U.S. Provisional Application No. 63/457,695, filed on Apr. 6, 2023, and entitled "SECURE NETWORK CONNECTIVITY BETWEEN SECURE NETWORKS", and
- (2) U.S. Provisional Application No. 63/457,700, filed on Apr. 6, 2023, and entitled "OCB NETWORK SERVICE ECAR-DATAPLANE", and
- (3) U.S. Provisional Application No. 63/434,879, filed on Dec. 22, 2022, and entitled "CLOUD BRIDGE TECHNIQUES", and
- (4) U.S. Provisional Application No. 63/434,846, filed on Dec. 22, 2022, and entitled "CLOUD MIGRATION TECHNIQUES".

The entire contents of the above-referenced provisional applications are hereby incorporated by reference herein for all purposes.

FIELD

The present disclosure relates generally to resource integration with cloud services, and more particularly, to techniques for extending the cloud service's reach into on- or off-premises environments and other cloud platforms to enable migration and multi-cloud use cases.

BACKGROUND

Cloud computing has been a principal of many organizations for years, offering an array of online services such as collaboration, communication, data storage and backup, user relationship management tools, and more; basically, every technological element needed to run a business. There are presently four main types of cloud computing: private cloud, public cloud, hybrid cloud, and multcloud. A private cloud architecture is a cloud environment dedicated to a single user, group or organization which can be on- or off-premises. A public cloud architecture is a cloud environment dedicated to multiple users, group or organizations which can be on- or off-premises. The hybrid cloud architecture uses both a public cloud architecture and a private cloud architecture or on-premises infrastructure. The multcloud architecture is a combination of clouds (private and/or public) spanning multiple vendors. Each of these types of clouds can be configured to provide various cloud computing services including: Infrastructure as a service (IaaS), Platforms as a service (PaaS), Software as a service (SaaS), and Integration platform as a service (iPaaS). IaaS offers standard compute, storage and network resources on a demand-based scale. PaaS offers software development and deployment

resources. SaaS offers software distribution to end users. iPaaS is a suite of cloud services enabling users to develop, execute and govern integration flows between disparate applications.

The growth of cloud computing has enabled organizations to use a wide assortment of highly scalable resources and services on demand rather than constructing and maintaining them on- or off-premises. However, in some organizations, the arrival of these diverse resources and services has created information technology (IT) silos as administrators strive to manage and maintain each different cloud resource and/or on- or off-premises resource (non-cloud resources). For example, IT silos can occur when only one group of people (e.g., one department within an organization) can access certain resources. Cloud-based integration (also known as cloud integration) unifies all the different cloud resources and/or on- or off-premises resources to avoid such IT silo instances. Cloud integration is a form of systems integration business delivered as a cloud computing service that addresses data, process, service-oriented architecture (SOA) and application integration. At its most basic level, cloud integration means connecting a wide range of processes, applications, systems, data repositories, and other IT environments (e.g., public clouds, private clouds, on-premises infrastructure, etc.)—either in a hybrid or multcloud deployment—so that they can operate as a single, cohesive IT infrastructure for an organization. Without a cloud integration solution, administrators need to perform each integration task separately and manually—a process that is time-consuming and increases the opportunity for error.

To take advantage of the numerous benefits provided by cloud services, an enterprise is often required to migrate on-premise applications and data from their local data center to the public cloud infrastructure. This process typically requires the enterprise to set up a site-to-site network connection to establish secure connectivity between their on-premise data center and the cloud infrastructure. Configuring a high performant, scalable, and highly available site-to-site network connection to process network traffic between different networks can be a complex and time-consuming task for an enterprise especially when the enterprise's on-premises applications and data scale across multiple different networks.

BRIEF SUMMARY

Techniques are provided (e.g., a method, a system, non-transitory computer-readable medium storing code or instructions executable by one or more processors) herein for secure bi-directional network connectivity between private networks. One aspect relates to a method. The method includes creating a connection between an external resource residing in an on-premise network and at least one intermediate container, transmitting a request from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource, receiving at the at least one intermediate container a result corresponding to the request via the created connection, and transmitting, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection. In some embodiments, the on-premise network includes a user wallet and the intermediate container includes a VCN wallet. In some embodiments, creating the connection includes authentication with the user wallet and the VCN wallet. In some embodiments, the external resource is registered as an external endpoint in a virtual cloud network (VCN).

3

In some embodiments, the method includes creating a second connection between the at least one intermediate container and the external resource representation. In some embodiments, the VCN wallet includes a VCN client wallet. In some embodiments, the at least one intermediate container includes a VCN server wallet. In some embodiments, creating the second connection includes authentication with the VCN client wallet and the VCN server wallet. In some embodiments, the method includes creating each of the VCN client wallet and the VCN server wallet based on the user wallet.

In some embodiments, the request is transmitted from the external resource to the external resource representation via the intermediate container. In some embodiments, the intermediate container resides on a data plane. In some embodiments, the intermediate container can include a proxy server. In some embodiments, the intermediate container can include a first container and a second container. In some embodiments, the first container can communicatively couple with the on-premise network via a first tunnel shard. In some embodiments, the first container can communicatively couple with the on-premise network via the first tunnel shard and a public communication network.

In some embodiments, the at least one intermediate container includes a second container that can communicatively couple with the VCN. In some embodiments, the second container includes a connection manager ("CMAN") container that can serve as a proxy server to redirect packets received by the CMAN container from the VCN to the on-premise network. In some embodiments, the first container is communicatively coupled with the second container. In some embodiments, the method includes registering the external resource residing as the external endpoint in the VCN. In some embodiments, the method includes creating the external resource representation for the external endpoint in the VCN, the creating of the external resource representation including creating the VNIC. In some embodiments, the method further includes creating a private endpoint in the VCN. In some embodiments, the private endpoint is communicatively coupled with the external resource via the external resource representation.

One aspect relates to a system. The system includes memory containing processor-executable stored instructions, and a processor that can execute the stored instructions to create a connection between an external resource residing in an on-premise network and at least one intermediate container, transmit a request from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource, receive at the at least one intermediate container a result corresponding to the request via the created connection, and transmit, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection. In some embodiments, the on-premise network includes a user wallet and the intermediate container includes a VCN wallet. In some embodiments, creating the connection includes authentication with the user wallet and the VCN wallet. In some embodiments, the external resource is registered as an external endpoint in a virtual cloud network (VCN).

One aspect relates to a non-transitory computer-readable storage medium storing a plurality of instructions executable by one or more processors, the plurality of instructions when executed by the one or more processors cause the one or more processors to create a connection between an external resource residing in an on-premise network and at least one intermediate container, transmit a request from an external

4

resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource, receive at the at least one intermediate container a result corresponding to the request via the created connection, and transmit, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection. In some embodiments, the on-premise network includes a user wallet and the intermediate container includes a VCN wallet. In some embodiments, creating the connection includes authentication with the user wallet and the VCN wallet. In some embodiments, the external resource is registered as an external endpoint in a virtual cloud network (VCN).

The techniques described above and below may be implemented in a number of ways and in a number of contexts. Several example implementations and contexts are provided with reference to the following figures, as described below in more detail. However, the following implementations and contexts are but a few of many.

BRIEF DESCRIPTION

FIG. 1 depicts a distributed environment **100** that includes a secure private network connectivity service within a cloud service provider infrastructure (CSPI), according to certain embodiments.

FIG. 2 depicts additional details of the operations performed by the systems and subsystems shown in FIG. 1 for providing secure private network connectivity between a customer's external resource residing in a customer's on-premise network and resources and services residing in the customer's VCN, according to certain embodiments.

FIG. 3 depicts an example of a process performed by the systems and subsystems shown in FIG. 1 for providing secure private network connectivity, according to certain embodiments.

FIG. 4 is a flowchart depicting a flow of network packets between an external resource residing in a customer's on-premise network and a representation of the external resource in the customer's virtual cloud network, according to certain embodiments.

FIG. 5 is a flowchart depicting the flow of network packets between an external resource representation in the customer's virtual cloud network and an external resource residing in a customer's on-premise network, according to certain embodiments.

FIG. 6 is a high-level diagram of a distributed environment showing a virtual or overlay cloud network hosted by a cloud service provider infrastructure according to certain embodiments.

FIG. 7 depicts a simplified architectural diagram of the physical components in the physical network within CSPI according to certain embodiments.

FIG. 8 shows an example arrangement within CSPI where a host machine is connected to multiple network virtualization devices (NVDs) according to certain embodiments.

FIG. 9 depicts connectivity between a host machine and an NVD for providing I/O virtualization for supporting multitenancy according to certain embodiments.

FIG. 10 depicts a simplified block diagram of a physical network provided by a CSPI according to certain embodiments.

FIGS. 11A, 11B, 11C are a high-level diagram of a distributed environment showing a cloud bridge architecture for the management and configuration of remote resources that interact with cloud services according to certain embodiments.

5

FIG. 12 is a schematic depiction of one embodiment of a migration.

FIG. 13 is an exemplary embodiment of an architecture that can be involved in a migration.

FIG. 14 is a schematic depiction of one embodiment of a system for secure network connectivity between secure networks.

FIG. 15 is a schematic depiction of another embodiment of a system for secure network connectivity between secure networks.

FIG. 16 is a flowchart illustrating one embodiment of a process for connecting the first and second environment.

FIG. 17 is a schematic depiction of one embodiment of the system for secure network communication between secure networks.

FIG. 18 is a flowchart illustrating one embodiment of a process for accessing an asset in the first environment via the second environment.

FIG. 19 is a block diagram illustrating one pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

FIG. 20 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

FIG. 21 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

FIG. 22 is a block diagram illustrating another pattern for implementing a cloud infrastructure as a service system, according to at least one embodiment.

FIG. 23 is a block diagram illustrating an example computer system, according to at least one embodiment.

DETAILED DESCRIPTION

In the following description, for the purposes of explanation, specific details are set forth in order to provide a thorough understanding of certain embodiments. However, it will be apparent that various embodiments may be practiced without these specific details. The figures and description are not intended to be restrictive. The word “exemplary” is used herein to mean “serving as an example, instance, or illustration.” Any embodiment or design described herein as “exemplary” is not necessarily to be construed as preferred or advantageous over other embodiments or designs.

The present disclosure relates generally to cloud-based services. More specifically, but not by way of limitation, the present disclosure describes a secure private network connectivity service within a cloud infrastructure that includes improved capabilities to establish secure private bi-directional network connectivity between external resources residing in a customer’s on-premise environment and the customer’s resources residing in the cloud.

A cloud infrastructure can offer high-performance compute, storage, and network capabilities in a flexible overlay virtual network that runs on top of the physical underlay network and that is securely accessible from an enterprise’s on-premises network. The cloud infrastructure allows enterprises to manage their cloud-based workloads in the same way they manage their on-premises workloads. Thus, enterprises can get all the benefits of the cloud with the same control, isolation, security, and predictable performance as their on-premises network. An enterprise can build their own networks using compute, memory, and networking resources provided by the cloud. For example, a customer can use resources provided by the cloud to build one or multiple customizable and private network(s) referred to as

6

virtual cloud networks (VCNs). A customer can deploy one or more customer resources, such as compute instances, on these customer VCNs. Compute instances can take the form of virtual machines, bare metal instances, and the like. The cloud thus provides infrastructure and a set of complementary cloud services that enable enterprises (customers) to build and run a wide range of applications and services in a highly available hosted environment.

To take advantage of the numerous benefits provided by the cloud infrastructure, many enterprises perform the migration of on-premise applications and data from their local data center to the public cloud infrastructure. Migrating workloads from a local (i.e., on-premise) data center of an enterprise to the cloud can be a complex and challenging process. Common challenges encountered during cloud migration involve identifying, by the enterprise, the type of migration to perform, the type of resources that need to be moved, and data dependencies between the resources. While migrating workloads, some resources (e.g., databases, applications and the like) may need to reside at the on-premise data center for a certain period of time until they are able to be successfully migrated into the cloud. Certain resources (e.g., business-critical resources, resources that have data portability restrictions or resources that have strict geographic requirements) may be identified as resources that are not migrated to the cloud due to security reasons and remain at the on-premise data center. For an enterprise to be able to securely access on-premise resources from their VCN (in the cloud) and to enable resources residing in their on-premise data center to be able to securely access resources residing in the cloud, secure private network connectivity needs to be established between the customer’s (enterprise’s) on-premise data center and the customer’s VCN. Setting up a secure site-to-site network connection can be a complex and time-consuming task for an enterprise. This typically requires network policy level management, by a user (e.g., an administrator) of the enterprise to set up the site-to-site network (e.g., VPN) connection, setting up multiple configuration parameters, setting up VPN components (e.g., a customer gateway device, a target gateway device) for the site-to-site network connectivity and so on.

Additionally, in order to access a remote asset (e.g., an on-premise resource such as a database or application) residing in their on-premise data center from their VCN using the site-to-site network connection or for remote assets residing in their on-premise data center to be able to securely access resources and services in the customer’s VCN, the user of an enterprise has to perform additional tasks like manually configure the gateway devices to perform route advertisements and network address translations so that secure connectivity between remote assets in the customer’s external environment and resources in the customer’s VCN can be achieved. The user also has to manually configure the remote asset in order for traffic (e.g., network packets) to reach the remote asset from the customer’s VCN and vice versa, configure route tables to include the routes used by the site-to-site VPN connection, enable route propagation for the route table to automatically propagate site-to-site VPN routes, update security rules and so on.

In certain embodiments, a secure private network connectivity service within a cloud service provider infrastructure (CSPI) is described that includes improved capabilities to establish secure private bi-directional network connectivity between external resources residing in a customer’s on-premise environment and the customer’s resources residing in the cloud. The secure private network connectivity service is implemented using a secure network connectivity

system (SNCS) within a cloud service provider infrastructure (CSPI). The SNCS described in the present disclosure provides several technical advancements and/or improvements over conventional cloud-based network connectivity services. Using the disclosed new and improved architecture implemented by the SNCS, secure private bi-directional network connectivity between external resources residing in a customer's external site representation and resources and services residing in the customer's VCN in the cloud can be achieved without a user (e.g., an administrator) of the enterprise having to explicitly configure the external resources, advertise routes or set up site-to-site network connectivity. The SNCS provides a high performant, scalable, and highly available site-to-site network connection for processing network traffic between a customer's on-premise environment and the CSPI by implementing a robust infrastructure of network elements and computing nodes that are used to provide the secure site to site network connectivity. By using the robust infrastructure of network elements and computing nodes implemented by the SNCS, a user of an enterprise can securely access its external resources from the cloud as if they were connecting to any other native resource within their VCN. Additionally, the SNCS enables one or more external resources residing in a customer's on-premise network to be able to securely reach out to and access resources and services residing in the customer's VCN (i.e., cloud). The secure private bi-directional network connectivity between external resources in a customer's external site representation and resources and services residing in the customer's VCN in the cloud can be achieved without a user (e.g., an administrator) of the enterprise having to explicitly configure the external resources, advertise routes or set up site-to-site network connectivity. Systems and methods relevant to the present are disclosed in U.S. application Ser. No. 17/515,087, filed on Oct. 29, 2021, and entitled "Transparent Mounting Of External Endpoints Between Private Networks"; and U.S. application Ser. No. 17/515,093, filed on Oct. 29, 2021, and entitled "Secure Bi-Directional Network Connectivity System Between Private Networks", the entirety of each of which is incorporated by reference herein.

Referring now to the drawings, FIG. 1 depicts a distributed environment 100 that includes a secure private network connectivity service within a cloud service provider infrastructure (CSPI), according to certain embodiments. The distributed environment 100 includes multiple systems communicatively coupled to each other via one or more communication networks. These communication networks may include public and private networks. The distributed environment 100 depicted in FIG. 1 is merely an example and is not intended to unduly limit the scope of claimed embodiments. Many variations, alternatives, and modifications are possible. For example, in some implementations, the distributed environment depicted in FIG. 1 may have more or fewer systems or components than those shown in FIG. 1, may combine two or more systems, or may have a different configuration or arrangement of systems.

As shown in the example depicted in FIG. 1, the distributed environment 100 comprises a CSPI 102 that provides services and resources that customers can subscribe to. In certain embodiments, the CSPI 102 provides a secure private network connectivity service that includes capabilities to provide secure bi-directional private network connectivity between a customer's on-premise network and the customer's VCN hosted by the CSPI 102. In the example shown in FIG. 1, the secure private network connectivity service may be implemented by a secure network connectivity system (SNCS) 104 within the CSPI 102. The secure private net-

work connectivity service provided by the SNCS 104 enables one or more resources (also referred to herein as external resources or remote assets) residing in a customer's on-premise network to be able to securely access resources and services residing in the customer's VCN. The secure private network connectivity service provided by the SNCS 104 additionally enables resources and services in the customer's VCN to securely access external resources residing in the customer's on-premise network. A customer can access the external resources from within their VCN as if they were connecting to any other native resource residing in the customer's VCN. The secure access between the on-premise external resources and the resources residing in the customer's VCN (i.e., cloud) can be enabled without requiring the customer (e.g., a user of the enterprise) to set an elaborate site-to-site network connection between their on-premise network and the cloud, without the customer having to make any changes to their external resources or without having the customer to configure routes to be used by the site-to-site connection. Additional details of the processing performed by the SNCS 104 to enable secure bi-directional connectivity between external resources residing in the customer's on-premise network and the customer's resources and services in the cloud are described in detail below.

In certain approaches, secure bi-directional connectivity between an external resource residing in a customer's on-premise network and the customer's resources and services in the cloud (e.g., the customer's VCN) is achieved by the SNCS 104 using a multi-stage process. In a first stage, a user (e.g., an administrator) associated with the customer can create an "external site representation" 106 of the customer's on-premise network. For instance, the user may interact with the SNCS 104 via a console user interface (UI) 108 of an application executed by a user device, via APIs or via a command line interface (CLI) executed by the user's device to create the external site representation 106. An external site representation (e.g., 106) may represent a logical or virtual representation of the customer's external site (e.g., on-premise network/on-premise data center) and may be identified by an external site identifier and a tenant (customer) identifier. By way of example, an external site representation 106 may represent a high level container resource that logically represents a portion of the customer's on-premise network and a subset of one or more external resources residing in the on-premise network.

In a second stage, the user downloads an agent 112 and installs/configures the agent 112 in the external site representation 106. In certain embodiments, the agent 112 may be a software application that is downloaded by the user as part of a download package provided by the SNCS 104 when the user subscribes to the secure private connectivity services provided by the SNCS 104. The agent 112 may be installed and configured by the user in the external site representation 106 via the console UI 108.

In a third stage, upon successful installation of the agent 112 in the external site representation 106, the SNCS 104 authenticates the agent 112 and orchestrates the setting up of a tenant-specific overlay network 128 for the customer. The tenant-specific overlay network 128 may represent a virtual overlay network that is built on top of a physical network by the SNCS 104 for each tenant (customer) who subscribes to the services provided by the SNCS. The tenant-specific overlay network 128 is used to establish secure private network connectivity between the customer's external site representation 106 and the customer's VCN 148 in the CSPI. As shown in the embodiment depicted in FIG. 1, the

tenant-specific overlay network **128** may comprise a distributed and horizontally scalable fleet of computing nodes that include a set of one or more tunnel hosts (also referred to herein as tunnel virtual machines), tunnel VM-1 **116** and tunnel VM-2 **122** and a set of one or more resource hosts (also referred to herein as resource virtual machines), resource VM-1 **130** and resource VM-2 **132**. A host (e.g., a tunnel host or a resource host) may be composed of a set of containers (also referred to herein as shards) that are interconnected with each other in the tenant-specific overlay network **128**. The tunnel VMs **116** and **122** are used for running per-tenant tunnel shards. For instance, as shown in FIG. 1, the tunnel VM-1 **116** is used to run a tunnel shard **120** and the tunnel VM-2 **122** is used to run a tunnel shard **126** for a specific tenant/customer of the CSPI. Each tunnel shard **120** or **126** is responsible for providing secure connectivity to the customer's external site representation **106**. The set of resource VMs **130** and **132** may be used for running per tenant resource shards. For instance, as shown in FIG. 1, the resource VM-1 **130** is used to run a resource shard **136** and the resource VM-2 **132** is used to run a resource shard **140** for the tenant/customer. A resource shard may be used to receive traffic from the customer's VCN and forward it to the customer's external site representation. Additional details of the operations performed by the tunnel shards and the resource shards shown in FIG. 1 for providing secure connectivity between the customer's external site representation **106** and the customer's VCN **148** is described in detail in FIG. 1.

Each tunnel VM (**116, 122**) and resource VM (**130, 132**) is additionally configured with a host manager (**118, 134**) respectively. The host managers (**118, 134**) represent processes executing on the tunnel VMs and the resource VMs. The host managers (**118** or **134**) may implement API's that are used to create the tunnel and resource shards. In certain implementations, the host managers (**118, 134**) may be stateless and operate in an imperative mode (i.e., as a sequence of commands for the host manager to perform) by receiving instructions from a user (via APIs **108**) regarding the type of shard (tunnel or resource shard) to be created and the specific configuration of the shard. The host managers may additionally be responsible for collecting and monitoring the status of the tunnel shards and the resource shards.

After the SNCS **104** sets up the tenant-specific overlay network **128** for the customer as described above, in a fourth stage, the user registers an on-premise asset (e.g., an external resource **114A**) in the external site representation **106** as an external endpoint in their VCN. The external resource **114A** may represent an on-premise resource or asset such as a database, a computing instance, an application or the like residing in the customer's on-premise network that the customer intends to establish secure bi-directional connectivity to, with resources and services residing in the customer's VCN. To register the external resource **114A**, the user (via the console UI **108**) identifies the external resource (e.g., **114A**) that is to be enabled secure private network connectivity from their VCN **148** and using the console UI **108** (or via APIs), registers the external resource as an external endpoint in their VCN. As part of registering the external resource, the user provides configuration information related to the external resource such as the on-premise physical IP address associated with the external resource, the port number that the external resource is accessible at, and a hostname (or a fully qualified domain name (FQDN)) of the external resource via the console UI or APIs provided by the SNCS. The user also selects a subnet in the customer's VCN where the external endpoint for the external resource

is to be created. The SNCS receives the configuration information and creates an external endpoint for the external resource in the customer's VCN. The external endpoint is identified by an IP address, a port number and a FQDN (hostname) in the customer's VCN.

In a fifth stage, the SNCS **104** (via control plane APIs) then creates an external resource representation for the external endpoint in the customer's VCN. In a certain implementation, the creation of the external resource representation comprises creating a remote mount point VNIC and assigning the IP address associated with the external endpoint to the VNIC. The SNCS (via control plane APIs) then creates resource shards on the resource VMs which are capable of logically attaching the VNIC (via a worker interface) to the resource shards.

In a sixth stage, the agent **112** downloads and reads the configuration information **156** associated with the VNIC **142** created for the external resource representation **114A** in the customer's VCN **148** and copies the configuration information **156** onto the registered external resource **114A**. The configuration information may include, for instance, the virtual IP address of the VNIC, a fully qualified domain name associated with a computing instance associated with the VNIC and a cloud identifier of the virtual cloud network associated with the customer. Using the configuration information **156**, the agent **112** creates/provisions a logical interface **158** for the external resource **114A**. The logical interface **158** may represent a software entity consisting of an IP address. In a certain implementation, the creation or provisioning of the logical interface **158** by the agent **112** comprises assigning, by the agent, the virtual IP address assigned to the VNIC **142** created for the external resource representation to the logical interface **158**.

Since the external resource **114A** is provisioned with a logical interface **158** that is assigned with the virtual IP address corresponding to the VNIC **142** for the external resource representation residing in the customer's VCN, the external resource now becomes part of the customer's VCN and is able to securely access resources and services that are in the cloud (e.g., in the customer's VCN **148**) by using the secure private network connectivity services provided by the SNCS **104**. For instance, the external resource **114A**, via its logical interface **158**, can securely reach/access a compute instance **150** in the customer's VCN **148** or a service **152** (e.g., a streaming service or an object storage service) in the customer's VCN by establishing, via the agent **112**, a connection to the tunnel hosts in the SNCS **104**. Similarly, a client application in the customer's VCN **148** can use the services provided by the SNCS **104** to securely access the external resource **114A** in the customer's on-premise network using the VNIC representation **142** of the external resource as if they were connecting to any other native resource in the customer's VCN **148**. In this manner, secure bi-directional connectivity between an external resource residing in a customer's on-premise network and the customer's resources and services in the cloud (e.g., the customer's VCN) is achieved by the SNCS **104**.

FIG. 2 depicts additional details of the operations performed by the systems and subsystems shown in FIG. 1 for providing secure private network connectivity between a customer's external resource residing in a customer's on-premise network and resources and services residing in the customer's VCN, according to certain embodiments. The systems and subsystems depicted in FIG. 2 may be implemented using software (e.g., code, instructions, program) executed by one or more processing units (e.g., processors, cores) of a computing system, hardware, or combinations

thereof. The software may be stored on a non-transitory storage medium (e.g., on a memory device). The distributed environment **200** depicted in FIG. 2 is merely an example and is not intended to unduly limit the scope of claimed embodiments. Many variations, alternatives, and modifications are possible. For example, in some implementations, the distributed environment depicted in FIG. 2 may have more or fewer systems or components than those shown in FIG. 2, may combine two or more systems, or may have a different configuration or arrangement of systems. Inventors, we have described the system in FIG. 2 largely based on the system architecture (SNCS) described in invention 1 (IaaS 272.1). Please review the description on FIG. 2 below and add/edit it if required for more clarity.

As previously described in FIG. 1, as part of the connection process implemented by the SNCS **104** to provide secure private network connectivity between a customer's on-premise network and the customer's VCN hosted by the CSPI **102**, a user (e.g., an administrator) associated with the customer creates an "external site representation" (e.g., **106**) of the customer's on-premise network, configures an agent (e.g., **112**) in the external site representation, registers an on-premise asset in the external site representation and creates a remote mount point (also referred to herein as an endpoint) in the customer's VCN for the registered external resource. Upon successful installation of the agent (e.g., **112**) in the external site representation, the agent **112** establishes a Virtual Private Network (VPN) connection (also referred to herein as a VPN tunnel) to the SNCS **104** via a public network (e.g., Internet **110**). The VPN connection is an encrypted connection between the customer's external site representation **106** and the customer's VCN **148**. In a certain implementation, the VPN connection utilizes a secure tunneling protocol (e.g., the Layer 2 Tunneling Protocol (L2TP) protocol) to establish secure private network connectivity to the SNCS **104** via the Internet **110**. When the agent **112** is installed in the external site representation **106**, it starts a bootstrap process to activate itself with the SNCS **104** by passing information such as a compartment identifier associated with the agent **112** and an external site representation identifier in a configuration file to the SNCS **104**. When the agent **112** bootstraps itself, it communicates with the control plane of the SNCS which starts the registration and activation process. During the activation process, the control plane sends back all the required information (e.g., certificates, Public IP and the like) that is needed by the gateway appliance to establish a secure tunnel connection to the VPN server. The agent **112** then establishes secure VPN connectivity to the SNCS **104** by executing a VPN client program which opens a secure VPN tunnel connection to a VPN server installed at the SNCS. The secure tunnel terminates on a tunnel shard (e.g., **120**) that is placed on a tunnel VM (e.g., **116**) in the SNCS **104**.

In the specific implementation depicted in FIG. 1, the agent **112** is configured with two VPN clients, VPN client-1 **206** and VPN client-2 **208** that are each configured to establish tunnels that terminate on two different VPN servers, VPN server-1 **224** and VPN server-2 **226** executing in tunnel shards, tunnel shard-1 **120** and tunnel shard-2 **126** respectively. In the implementation depicted in FIG. 2, every physical installation of an agent **112** results in the establishment of two tunnels and hence if one tunnel goes down, traffic (i.e., network packets) can automatically be routed across the second tunnel. While the specific implementation shown in FIG. 2 depicts two tunnels, in other implementations, the SNCS **104** may be configured to implement more

than two redundant tunnels for every physical installation of the agent **112** or may just implement a single tunnel upon installation of the agent **112**.

In certain embodiments, the agent **112** uses a standard exterior gateway protocol such as the Border Gateway Protocol (BGP) to establish BGP peering sessions with the tunnel shards **120** and **126**. Using BGP, the agent **112** exchanges routing and reachability information with the tunnel shards via a public interface **220** implemented within the agent **112**. As part of the configuration information required for the BGP peering sessions, the agent **112** injects its on-premise IP address into its local route table **212** (also referred to herein as a routing information base (RIB)) which is received by the tunnel shards. The tunnel shards import the routing information into their local route tables (**228** and **236**) after applying appropriate route filtering policies. The route filtering is needed to ensure that a compromised agent **112** does not inject arbitrary routes to the tunnel shards. In certain examples, the agent **112** additionally comprises a routing manager **214**. The routing manager **214** may be implemented using an open source routing manager (e.g., Zebra) that is part of a routing suite (e.g., Quagga). When the BGP peering sessions learn routes and import them into its route table **212**, the BGP peering sessions perform the best path calculation and use the routing manager **214** to add the best routes to the local kernel.

A tunnel shard (e.g., **120**, **126**) may be composed of a set of one or more containers. In certain implementations, a tunnel shard (**120** or **126**) may comprise a shell container that may be used to set up various network interfaces that enable the tunnel shard to communicate with both the agent **112** and other shards (e.g., resource shards **136** and **140**) that are part of the tenant-specific overlay network **128**. In the embodiment depicted in FIG. 2, the network interfaces implemented within a shell container in a tunnel shard may include an external site interface (esi), a tunnel interface and a shard backend interface. For instance, the network interfaces implemented in tunnel shard-1 **120** include a tunnel interface **216**, an external site interface **222** and a shard backend interface **234**. Similarly, the network interfaces implemented in tunnel shard-2 **126** include a tunnel interface **218**, an external site interface **223** and a shard backend interface **235**. A tunnel shard (e.g., **120** or **126**) is additionally composed of a VPN server (**224**, **226**) that is used to establish a VPN tunnel to a VPN client (**206**, **208**) executing in the external gateway appliance **112**. An external site interface (**222**, **223**) may be identified by a public IP that is known to the agent **112** on which the VPN client runs and is used to establish a tunnel to the tunnel shard. When the VPN client (e.g., **206**, **208**) connects to the VPN server (**224**, **226**), a tunnel interface (**216**, **218**) is created and placed in a pre-configured VPN subnet of the tunnel shard.

In certain implementations, each tunnel shard (e.g., **120**, **126**) may utilize the Border Gateway Protocol (BGP) to establish BGP peering sessions between the external gateway appliance and the tunnel shards and BGP peering sessions between the tunnel shards and the resource shards. The BGP peering sessions are used to exchange routing and reachability information with the resource shards via the shard backend interfaces (**234**, **235**) and to exchange routing and reachability information with the external site representation **106** via the external site interfaces (**222**, **223**). As part of the configuration information required for the BGP peering sessions, an IP address that identifies the tenant-specific overlay network **128** is added to the route tables **228**, **236** (i.e., routing information bases (RIBs)) implemented in the

tunnel shards **120, 126** respectively. In a certain implementation, the Classless Inter-Domain Routing (CIDR) technique may be used for allocating the IP address to the tenant-specific overlay network. The route tables (**228, 236**) list the routes to particular network destinations such as to the resource shards (**136, 140**) and to the external site representation **106**. In some cases, the route tables (**228, 236**) also list metrics (distances) associated with those routes. When the BGP peering sessions with the resource shards are established, the route in the route tables is propagated to the resource shards. As shown in FIG. 2, each tunnel shard (**120, 126**) additionally comprises a routing manager (**228, 240**). The routing manager (**228 or 240**) may be implemented in a similar manner to the routing manager (**214**) implemented in the external gateway appliance **112**. When BGP learns routes and imports them into the route tables in the tunnel shards, it performs the best path calculation and uses the routing manager to push the best routes to the external site representation **106** and to the resource shards **136, 140**.

A resource shard (**136, 140**) may be composed of a set of containers. In certain implementations, a resource shard (**136, 140**) may comprise a shell container that may be used to set up a virtual tunnel endpoint VTEP (**242, 243**) with the tunnel shards (**116, 126**). Each resource shard (**136, 140**) may use BGP to establish peering sessions with the tunnel shards and to exchange routing and reachability information with the tunnel shards via the virtual tunnel endpoints (**242, 243**). A resource shard (**136, 140**) additionally comprises a routing manager (**250, 260**) that is configured to perform the same functionality as the routing manager (**232 or 240**) implemented in the tunnel shards. Each resource shard (**136, 140**) additionally includes a proxy server (**244, 254**). The proxy servers (**244 or 254**) may be configured to accept connections from a client application **144** in the customer's VCN **148** and initiate new connections to the external resource in the external site representation.

In certain implementations, for every registered external endpoint (i.e., corresponding to an external resource in the external site representation), a unique proxy server container is launched and attached to a resource shard. When registering the external resource and creating a remote mount point VNIC for the registered external endpoint in the customer's VCN, a user may provide configuration information such as the IP address of the external resource, the port number of the external resource, and the name of the external resource to the SNCS **104**. This configuration information is provisioned in the proxy servers and used by a client application in the customer's VCN to connect to the external resource. When the external resource is successfully registered as an external endpoint in the customer's VCN by the SNCS, the SNCS **104** (via control plane APIs) creates a VNIC and assigns the IP address associated with the external endpoint to the VNIC.

In the example depicted in FIG. 2, the registered external endpoint (i.e., external resource in the external site representation) represents a database **202** residing in the external site representation **106**. The proxy servers (**244, 254**) listen on the virtual IP address assigned to the VNIC **206** that is created for the registered external endpoint **202**. This ensures that the same VNIC IP cannot be used to access other registered external resources in the external site representation. When a client application **144** running in the customer's VCN receives a query to obtain information stored in the external database **202**, it transmits network packets to the VNIC IP in the customer's VCN. The network packets are received by worker VNICs (**251, 252**) that are

attached to the resource shards (**136, 140**). The worker VNICs are configured with the virtual IP address of the VNIC and in turn, initiate a connection to the registered external resource in the external site representation via the proxy servers (**244, 254**). The proxy servers (**244, 254**) perform a network address translation (NAT) to translate the virtual IP address assigned to the VNIC **206** to the real/physical IP address of the external resource **114A** in the external site representation **106**. The resource shards (**136, 140**) using the proxy servers (**244, 254**) initiate a connection to the agent **112** via the tunnel shards (**120, 126**). The agent **112** receives the network packets from the tunnel shards (**120, 126**) and, in turn, routes the packets to the registered external resource (i.e., database **202**) in the external site representation **106**. The proxy servers (**244, 254**) additionally include capabilities to load-balance the network traffic to the external resource across the tunnel shards. In certain embodiments, the proxy servers (**244, 254**) may be configured to multiplex connections from multiple client applications onto a single connection towards the external site representation.

In a similar manner, a registered external resource (i.e., database **202**) residing in the external site representation **106** can reach out to and securely access a resource (e.g., compute instance **150**) or a service (e.g., private endpoint VNIC **152**) residing in the customer's VCN. As previously described in FIG. 1, the agent **112** creates/provisions a logical interface **204** for the external resource **202**. By creating a logical interface **158** for the external resource **114A** and assigning the virtual IP address of the VNIC representation **206** of the database **202** to the logical interface, the database **202** (which resides in the customer's on-premise network) is able to securely access a compute instance **150** in the customer's VCN **148** or a service **156** (e.g., a streaming service or an object storage service in the CSPI) via a private endpoint VNIC **142** in the customer's VCN **148** using the secure private network connectivity services provided by the SNCS **104**. For instance, in the embodiment depicted in FIG. 2, the agent **112** can receive a query from the database **202** requesting information associated with a resource (e.g., **150**) in the customer's VCN. The agent **112** then transmits network packets corresponding to the query to the tunnel shards (**120, 126**). The tunnel shards, in turn, initiate a connection to the resource shards (**136, 140**). The proxy servers (**244, 254**) in the resource shards (**136, 140**) perform a network address translation (NAT) to translate the real IP address of the external resource in the external site representation **106** to the virtual IP address of the VNIC representation **206** of the external resource (e.g., database **202**) in the customer's VCN. The worker VNICs (**251, 252**) that are attached to the resource shards (**136, 140**) in turn, initiate a connection to the VNIC representation **206** of the database **202** in the customer's VCN which transmits the network packets corresponding to the query to the requesting resource (e.g., **150**) in the customer's VCN. In this manner, the SNCS **104** ensures that secure private bi-directional network connectivity is established between an external resource (e.g., database **202**) residing in a customer's on-premise network and resources (e.g., **150**) and services (e.g., **152**) in the customer's VCN (i.e., cloud).

In certain implementations, the registered external endpoint that requires private connectivity from the customer's VCN **148** may be implemented as a Service VNIC, also referred to herein as an "SVNIC" in the customer's VCN **148**. The SVNIC may be implemented by the CSPI **102** using a VNIC as a service (VNICaaS) system (not shown in

15

FIG. 2) which may represent a horizontally scalable service implemented by the CSPI 102 that is capable of hosting multiple VNICs (e.g., service VNIC 254) to process and transmit traffic between virtual cloud networks. Specifically, the VNICaaS is a virtual networking feature that enables a VNIC to be represented or used as a service (i.e., SVNIC). The VNICaaS provides functionality of a VNIC without requiring a specific SmartNIC or host of a compute instance within a virtual network to host the VNIC. Techniques used to represent registered endpoints as SVNICs have been described in detail U.S. patent application Ser. No. 17/175,573, entitled “Techniques for high performant virtual routing capabilities.” The techniques described in U.S. patent application Ser. No. 17/175,573 are merely meant as examples and are not intended to be limiting. Various other techniques may also be used to represent registered endpoints and to process and transmit traffic between virtual cloud networks in alternative embodiments. Since an SVNIC is created for every registered external endpoint (e.g., database 202), a flexible number of workers can be configured per SVNIC. In a specific implementation, every SVNIC may be associated with two worker VNICs (e.g., 251, 252) that are attached to the different resource shards (e.g., 136, 140). The worker VNICs are passed into the resource shards and configured with the SVNIC IP.

In the specific implementation depicted in FIG. 2, a single agent 112 is installed in the external site representation 106 and is configured to establish two tunnels resulting in a total of two tunnel shards that need to be placed on the tunnel fleet implemented by the SNCS 104. For a fixed number of tunnel shards as described in this implementation, every resource shard may be pre-configured with a fixed set of BGP peers along with static Address Resolution Protocol (ARP) entries for those peers. In certain approaches, the BGP container on the tunnel shard may be configured with a “Dynamic BGP peer” in which a CIDR block of IP addresses can be specified from where BGP can accept incoming connections. This CIDR block may be set to the CIDR of the tenant-specific overlay network and hence as the resource shards grow/shrink the BGP peers on the tunnel shard also change appropriately. From the perspective of the resource shards, every resource shard now has two BGP peering connections and since each of the BGP peers propagate the same CIDR route to the on-premise network that it learns from the external site representation, every resource shard has two equal-cost paths towards the on-premise network. Similarly, from the external gateway’s perspective, the agent 112 establishes two tunnels towards the SNCS 104 and the tunnel shards advertise the tenant-specific overlay network routes to the agent 112 which then installs the appropriate route to the tenant-specific overlay network.

Using the disclosed new and improved architecture implemented by the SNCS 104, secure private bi-directional network connectivity between external resources in a customer’s external site representation and resources and services residing in the customer’s VCN in the cloud can be achieved without a user (e.g., an administrator) of the enterprise having to explicitly configure the external resource, advertise routes or set up site-to-site network connectivity. The SNCS 104 provides a high performant, scalable, and highly available site-to-site network connection for processing network traffic between a customer’s on-premise environment and the CSPI by implementing a robust infrastructure of network elements and computing nodes (i.e., a tenant-specific overlay network 128) for each tenant/customer that uses the services provided by the SNCS 104. The tunnel hosts in the tenant-specific overlay network

16

128 are used to provide secure connectivity to the customer’s external site representation 106 and the resource hosts are used to receive traffic from the customer’s VCN and forward it to the customer’s external site representation. The resource hosts are additionally capable of receiving traffic from an external resource in the external site representation and forward it to resources or services residing in the customer’s VCN. By using the robust infrastructure of network elements and computing nodes implemented by the tenant-specific overlay network, an enterprise can securely access its external resources from the cloud as if they were connecting to any other native resource within their VCN. A user of the enterprise can access its external resources without setting an elaborate site-to-site network between their on-premise network and the cloud, without making any changes to their external resources and without configuring routes to be used by the site-to-site connection. Similarly, by creating a logical interface for the external resource and associating the logical interface with the virtual IP address of the VNIC representation of the external resource, the external resource (which resides in the customer’s on-premise network) is able to securely access resources and services that are in the cloud (e.g., in the customer’s VCN 148) using the secure private network connectivity services provided by the SNCS 104.

FIG. 3 depicts an example of a process performed by the systems and subsystems shown in FIG. 1 for providing secure private network connectivity, according to certain embodiments. The processing depicted in FIG. 3 may be implemented in software (e.g., code, instructions, program) executed by one or more processing units (e.g., processors, cores) of the respective systems, hardware, or combinations thereof. The software may be stored on a non-transitory storage medium (e.g., on a memory device). The process 300 presented in FIG. 3 and described below is intended to be illustrative and non-limiting. Although FIG. 3 depicts the various processing steps occurring in a particular sequence or order, this is not intended to be limiting. In certain alternative embodiments, the steps may be performed in some different order or some steps may also be performed in parallel. In certain embodiments, such as in the embodiment depicted in FIG. 1, the processing depicted in FIG. 3 may be performed by the computing nodes (e.g., 116, 122, 130 and 132) comprising the tenant-specific overlay network 128.

The processing depicted in FIG. 3 assumes that a user (e.g., an administrator) associated with the customer has created an external site representation (e.g., 106) of the customer’s on-premise network and has configured an agent (e.g., 112) in the external site representation 106. The processing depicted in FIG. 4 further assumes that the SNCS 104 has authenticated the agent 112 and has configured/established a tenant-specific overlay network (e.g., 128) comprising a distributed and horizontally scalable fleet of computing nodes for the customer. As described in FIG. 1, the set of one or more computing nodes include resource hosts (i.e., resource virtual machines, 130, 132) and tunnel hosts (i.e., tunnel virtual machines, 116, 122).

The processing depicted in FIG. 3 may be initiated at block 302 when the SNCS 104 executes the tenant-specific overlay network (e.g., 128) comprising a distributed and horizontally scalable fleet of computing nodes. The tenant-specific overlay network is used to establish secure private network connectivity between the customer’s external site representation and the customer’s VCN (e.g., 148) in the CSPI.

At block 304, the SNCS (via control plane APIs) registers an external resource residing in the customer’s on-premise

17

network as an external endpoint in the customer's VCN. The external endpoint is identified by an IP address in the customer's VCN. As previously described, the external resource may represent a database, a computing instance, or an application in the external site representation **106** that the customer intends to enable secure bi-directional private network connectivity from within their VCN. As part of registering the external resource, a user of the SNCS provides configuration information related to the external resource such as the on-premise physical IP address associated with the external resource, the port number that the external resource is accessible at, and a hostname (or a fully qualified domain name (FQDN)) of the external resource via the console UI or APIs. The user also selects a subnet in the customer's VCN where the external endpoint for the external resource is to be created. Based on the configuration information, the SNCS creates an external endpoint for the external resource in the customer's VCN. The external endpoint is identified by an IP address, a port number and a FQDN (hostname) in the customer's VCN.

At block **306**, a first computing node (e.g., a resource VM **130** or resource VM **132**) in the SNCS creates an external resource representation for the external endpoint in the customer's VCN. In a certain implementation, the creation of an external resource representation involves creating, by the first computing node, a VNIC and assigning the IP address of the external endpoint to the VNIC.

At block **308**, the first computing node transmits configuration information corresponding to the VNIC created for the external resource representation in the customer's VCN to the agent **112** configured in the on-premise network. As part of the processing performed in block **308**, the agent **112** downloads and copies the configuration information onto the registered external resource residing in the on-premise network. As previously described, the configuration information may include, for instance, the virtual IP address of the VNIC, a fully qualified domain name associated with a computing instance associated with the VNIC and a cloud identifier of the virtual cloud network associated with the customer. Using the configuration information, the agent **112** creates/provisions a logical interface (e.g., **158**) for the external resource (e.g., **114A**). In a certain implementation, the creation or provisioning of the logical interface by the agent **112** comprises assigning, by the agent, the virtual IP address assigned to the VNIC **142** created for the external resource representation to the logical interface **158** provisioned for the external resource **114A**.

At block **310**, a second computing node (e.g., a tunnel VM **116** or tunnel VM **122**) receives a request for querying information associated with a resource (e.g., compute instance **150**) residing in the customer's VCN. In certain examples, the request may be transmitted by the external resource (e.g., **114A**) residing in the on-premise network, via its logical interface **158**. For instance, a user associated with the customer may transmit (e.g., via the UI **108**), a request for querying information associated with the compute instance **150** residing in the customer's VCN to the external resource residing in the customer's on-premise network. The external resource may, in turn, may transmit the query to the agent **112** residing in the on-premise network. The agent **112** receives the query request and transmits network packets corresponding to the query request to the tunnel shards (**120** or **126**) running in the tunnel VMs.

At block **312**, the second computing node (e.g., a tunnel VM **116** or tunnel VM **122**) establishes a connection between the logical interface provisioned for the external resource and the VNIC created for the external resource

18

representation in the customer's VCN via a resource shard (e.g., **136** or **140**). In a specific implementation, the tunnel shards (e.g., **120** or **126**) may include capabilities to translate the real/physical IP address assigned to the external resource to the virtual IP address of the VNIC **142** and initiate a connection to the VNIC **142** via the resource shards (**136** or **140**). The resource shards, in turn, initiate a connection to the resource (e.g., **150**) in the customer's VCN.

At block **314**, the second computing node (e.g., a tunnel VM **116** or tunnel VM **122**) transmits the request to the resource residing in the customer's VCN via the connection established in block **410**. At block **412**, the second computing node obtains a result corresponding to the request via the established connection. The result is then transmitted to the external resource residing in the external site representation via the agent **112**. For example, if the resource is a database executing in the customer's VCN, the result may include information stored in one or more tables in the database.

FIG. **4** is a flowchart depicting a flow of network packets between an external resource residing in a customer's on-premise network and a representation of the external resource in the customer's virtual cloud network, according to certain embodiments. The processing depicted in FIG. **4** may be implemented in software (e.g., code, instructions, program) executed by one or more processing units (e.g., processors, cores) of the respective systems, hardware, or combinations thereof. The software may be stored on a non-transitory storage medium (e.g., on a memory device). The process **400** presented in FIG. **4** and described below is intended to be illustrative and non-limiting. Although FIG. **4** depicts the various processing steps occurring in a particular sequence or order, this is not intended to be limiting. In certain alternative embodiments, the steps may be performed in some different order or some steps may also be performed in parallel. In certain embodiments, such as in the embodiment depicted in FIG. **2**, the processing depicted in FIG. **4** may be performed by the computing nodes (e.g., **116**, **122**, **130** and **132**) comprising the tenant-specific overlay network **128**.

The processing depicted in FIG. **4** may be initiated at block **402** when an external resource (e.g., a database **202**) residing in the customer's on-premise network (i.e., the customer's external site representation **106**) receives a query for information associated with a resource (e.g., **150**) residing in the customer's VCN (e.g., **148**). As previously described, a user associated with the customer may transmit (e.g., via the UI **108**), a request for querying information associated with the compute instance **150** residing in the customer's VCN to the external resource residing in the on-premise network. The external resource may, in turn, may transmit the query to the agent **112** residing in the on-premise network.

At block **404**, the agent **112** receives the query and transmits network packets corresponding to the query to the tunnel shards (**120** or **126**) running in the tunnel VMs. In certain examples, as part of the processing performed in block **404**, the agent **112** may encrypt the network packets prior to transmitting the network packets to the tunnel shards.

At block **406**, the tunnel shards receive the encrypted network packets and transmit the encrypted network packets to the resource shards (**136**, **140**) running in the resource VMs. Specifically, and as shown in FIG. **2**, the encrypted network packets may be received by worker VNICs (**251**, **252**) that are attached to the resource shards (**136**, **140**).

At block **408**, the resource shards (**136**, **140**) decrypt the network packets and perform a network address translation

(NAT) to translate the physical IP address assigned to the external resource to the virtual IP address of the VNIC (e.g., 206) that is associated with the external resource representation in the customer's VCN.

At block 410, the resource shards (136, 140) transmit the network packets to the VNIC created for the external resource representation in the customer's VCN.

At block 412, the VNIC transmits the network packets corresponding to the query to the resource (e.g., 150) residing in the customer's VCN.

FIG. 5 is a flowchart depicting the flow of network packets between an external resource representation in the customer's virtual cloud network and an external resource residing in a customer's on-premise network, according to certain embodiments. The processing depicted in FIG. 5 may be implemented in software (e.g., code, instructions, program) executed by one or more processing units (e.g., processors, cores) of the respective systems, hardware, or combinations thereof. The software may be stored on a non-transitory storage medium (e.g., on a memory device). The process 500 presented in FIG. 5 and described below is intended to be illustrative and non-limiting. Although FIG. 5 depicts the various processing steps occurring in a particular sequence or order, this is not intended to be limiting. In certain alternative embodiments, the steps may be performed in some different order or some steps may also be performed in parallel. In certain embodiments, such as in the embodiment depicted in FIG. 2, the processing depicted in FIG. 5 may be performed by the computing nodes (e.g., 116, 122, 130 and 132) comprising the tenant-specific overlay network 128.

The processing depicted in FIG. 5 may be initiated at block 502 when a client application (e.g., 144) in the customer's VCN receives a query for information associated with an external resource (e.g., 202) residing in the customer's on-premise network (i.e., the customer's external site representation 106). For instance, the query may be received from a user associated with the customer via the client application. As part of the processing performed in block 502, the client application 144 initiates a connection to the VNIC (e.g., 206) associated with the external resource representation by transmitting network packets corresponding to the query to the virtual IP address assigned to the VNIC in the customer's VCN.

At block 504, the client application transmits the network packets corresponding to the query to the resource shards (136 or 140) running in the resource VMs in the SNCS. Specifically, and as shown in FIG. 2, the network packets may be received by worker VNICs (251, 252) that are attached to the resource shards (136, 140). The proxy servers (244, 254) in the resource shards (136, 140) perform a network address translation (NAT) to translate the virtual IP address assigned to the VNIC to the physical IP address of the external resource in the external site representation 106 and initiate a connection to the agent 112 via the tunnel shards (120, 126).

At block 506, the resource shards transmit the network packets to the tunnel shards (120, 126). As part of the processing performed in block 506, the tunnel shards may encrypt the network packets received from the resource shards prior to transmitting the packets to the agent 112 in the external site representation 106.

At block 508, the tunnel shards transmit the network packets to the agent 112 residing in the customer's external site representation. As part of the processing performed in block 508, the agent 112 decrypts the network packets prior

to transmitting the network packets to the external resource (e.g., 202) in the external site representation (e.g., 106).

At block 510, the agent 112 transmits the network packets to the external resource (e.g., 202) residing in the customer's external site representation. The external resource receives the network packets corresponding to the query and generates response network packets corresponding to the query. The response network packets may then be transmitted by the external resource back to the client application. For example, as part of a response network packet flow, network packets corresponding to a response to the query are transmitted from the external resource to the agent. The agent encrypts the network packets and transmits the encrypted network packets to the tunnel shards. The tunnel shards receive the encrypted packets and transmit the packets to the resource shards. The resource shards decrypt the network packets and perform a reverse network address translation (NAT) to translate the physical/real IP address assigned to the external resource to the virtual IP address of the VNIC that is associated with the external resource representation in the customer's VCN. The resource shards then transmit the response network packets to the VNIC which in turn transmits the response packets to the requesting client application 144 in the customer's VCN 148.

Example Virtual Networking Architecture

The term cloud service is generally used to refer to a service that is made available by a cloud services provider (CSP) to users or customers on demand (e.g., via a subscription model) using systems and infrastructure (cloud infrastructure) provided by the CSP. Typically, the servers and systems that make up the CSP's infrastructure are separate from the customer's own on-premise servers and systems. Customers can thus avail themselves of cloud services provided by the CSP without having to purchase separate hardware and software resources for the services. Cloud services are designed to provide a subscribing customer easy, scalable access to applications and computing resources without the customer having to invest in procuring the infrastructure that is used for providing the services.

There are several cloud service providers that offer various types of cloud services. There are various different types or models of cloud services including Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS), Infrastructure-as-a-Service (IaaS), and others.

A customer can subscribe to one or more cloud services provided by a CSP. The customer can be any entity such as an individual, an organization, an enterprise, and the like. When a customer subscribes to or registers for a service provided by a CSP, a tenancy or an account is created for that customer. The customer can then, via this account, access the subscribed-to one or more cloud resources associated with the account.

As noted above, infrastructure as a service (IaaS) is one particular type of cloud computing service. In an IaaS model, the CSP provides infrastructure (referred to as cloud services provider infrastructure or CSPI) that can be used by customers to build their own customizable networks and deploy customer resources. The customer's resources and networks are thus hosted in a distributed environment by infrastructure provided by a CSP. This is different from traditional computing, where the customer's resources and networks are hosted by infrastructure provided by the customer.

The CSPI may comprise interconnected high-performance compute resources including various host machines, memory resources, and network resources that form a physical network, which is also referred to as a substrate network

or an underlay network. The resources in CSPI may be spread across one or more data centers that may be geographically spread across one or more geographical regions. Virtualization software may be executed by these physical resources to provide a virtualized distributed environment. The virtualization creates an overlay network (also known as a software-based network, a software-defined network, or a virtual network) over the physical network. The CSPI physical network provides the underlying basis for creating one or more overlay or virtual networks on top of the physical network. The physical network (or substrate network or underlay network) comprises physical network devices such as physical switches, routers, computers and host machines, and the like. An overlay network is a logical (or virtual) network that runs on top of a physical substrate network. A given physical network can support one or multiple overlay networks. Overlay networks typically use encapsulation techniques to differentiate between traffic belonging to different overlay networks. A virtual or overlay network is also referred to as a virtual cloud network (VCN). The virtual networks are implemented using software virtualization technologies (e.g., hypervisors, virtualization functions implemented by network virtualization devices (NVDs) (e.g., smartNICs), top-of-rack (TOR) switches, smart TORs that implement one or more functions performed by an NVD, and other mechanisms) to create layers of network abstraction that can be run on top of the physical network. Virtual networks can take on many forms, including peer-to-peer networks, IP networks, and others. Virtual networks are typically either Layer-3 IP networks or Layer-2 VLANs. This method of virtual or overlay networking is often referred to as virtual or overlay Layer-3 networking. Examples of protocols developed for virtual networks include IP-in-IP (or Generic Routing Encapsulation (GRE)), Virtual Extensible LAN (VXLAN—IETF RFC 7348), Virtual Private Networks (VPNs) (e.g., MPLS Layer-3 Virtual Private Networks (RFC 4364)), VMware's NSX, GENEVE (Generic Network Virtualization Encapsulation), and others.

For IaaS, the infrastructure (CSPI) provided by a CSP can be configured to provide virtualized computing resources over a public network (e.g., the Internet). In an IaaS model, a cloud computing services provider can host the infrastructure components (e.g., servers, storage devices, network nodes (e.g., hardware), deployment software, platform virtualization (e.g., a hypervisor layer), or the like). In some cases, an IaaS provider may also supply a variety of services to accompany those infrastructure components (e.g., billing, monitoring, logging, security, load balancing and clustering, etc.). Thus, as these services may be policy-driven, IaaS users may be able to implement policies to drive load balancing to maintain application availability and performance. CSPI provides infrastructure and a set of complementary cloud services that enable customers to build and run a wide range of applications and services in a highly available hosted distributed environment. CSPI offers high-performance compute resources and capabilities and storage capacity in a flexible virtual network that is securely accessible from various networked locations such as from a customer's on-premises network. When a customer subscribes to or registers for an IaaS service provided by a CSP, the tenancy created for that customer is a secure and isolated partition within the CSPI where the customer can create, organize, and administer their cloud resources.

Customers can build their own virtual networks using compute, memory, and networking resources provided by CSPI. One or more customer resources or workloads, such as compute instances, can be deployed on these virtual

networks. For example, a customer can use resources provided by CSPI to build one or multiple customizable and private virtual network(s) referred to as virtual cloud networks (VCNs). A customer can deploy one or more customer resources, such as compute instances, on a customer VCN. Compute instances can take the form of virtual machines, bare metal instances, and the like. The CSPI thus provides infrastructure and a set of complementary cloud services that enable customers to build and run a wide range of applications and services in a highly available virtual hosted environment. The customer does not manage or control the underlying physical resources provided by CSPI but has control over operating systems, storage, and deployed applications; and possibly limited control of select networking components (e.g., firewalls).

The CSP may provide a console that enables customers and network administrators to configure, access, and manage resources deployed in the cloud using CSPI resources. In certain embodiments, the console provides a web-based user interface that can be used to access and manage CSPI. In some implementations, the console is a web-based application provided by the CSP.

CSPI may support single-tenancy or multi-tenancy architectures. In a single tenancy architecture, a software (e.g., an application, a database) or a hardware component (e.g., a host machine or a server) serves a single customer or tenant. In a multi-tenancy architecture, a software or a hardware component serves multiple customers or tenants. Thus, in a multi-tenancy architecture, CSPI resources are shared between multiple customers or tenants. In a multi-tenancy situation, precautions are taken and safeguards put in place within CSPI to ensure that each tenant's data is isolated and remains invisible to other tenants.

In a physical network, a network endpoint ("endpoint") refers to a computing device or system that is connected to a physical network and communicates back and forth with the network to which it is connected. A network endpoint in the physical network may be connected to a Local Area Network (LAN), a Wide Area Network (WAN), or other type of physical network. Examples of traditional endpoints in a physical network include modems, hubs, bridges, switches, routers, and other networking devices, physical computers (or host machines), and the like. Each physical device in the physical network has a fixed network address that can be used to communicate with the device. This fixed network address can be a Layer-2 address (e.g., a MAC address), a fixed Layer-3 address (e.g., an IP address), and the like. In a virtualized environment or in a virtual network, the endpoints can include various virtual endpoints such as virtual machines that are hosted by components of the physical network (e.g., hosted by physical host machines). These endpoints in the virtual network are addressed by overlay addresses such as overlay Layer-2 addresses (e.g., overlay MAC addresses) and overlay Layer-3 addresses (e.g., overlay IP addresses). Network overlays enable flexibility by allowing network managers to move around the overlay addresses associated with network endpoints using software management (e.g., via software implementing a control plane for the virtual network). Accordingly, unlike in a physical network, in a virtual network, an overlay address (e.g., an overlay IP address) can be moved from one endpoint to another using network management software. Since the virtual network is built on top of a physical network, communications between components in the virtual network involves both the virtual network and the underlying physical network. In order to facilitate such communications, the components of CSPI are configured to learn and store

mappings that map overlay addresses in the virtual network to actual physical addresses in the substrate network, and vice versa. These mappings are then used to facilitate the communications. Customer traffic is encapsulated to facilitate routing in the virtual network.

Accordingly, physical addresses (e.g., physical IP addresses) are associated with components in physical networks and overlay addresses (e.g., overlay IP addresses) are associated with entities in virtual or overlay networks. A physical IP address is an IP address associated with a physical device (e.g., a network device) in the substrate or physical network. For example, each NVD has an associated physical IP address. An overlay IP address is an overlay address associated with an entity in an overlay network, such as with a compute instance in a customer's virtual cloud network (VCN). Two different customers or tenants, each with their own private VCNs can potentially use the same overlay IP address in their VCNs without any knowledge of each other. Both the physical IP addresses and overlay IP addresses are types of real IP addresses. These are separate from virtual IP addresses. A virtual IP address is typically a single IP address that represents or maps to multiple real IP addresses. A virtual IP address provides a 1-to-many mapping between the virtual IP address and multiple real IP addresses. For example, a load balancer may use a VIP to map to or represent multiple servers, each server having its own real IP address.

The cloud infrastructure or CSPI is physically hosted in one or more data centers in one or more regions around the world. The CSPI may include components in the physical or substrate network and virtualized components (e.g., virtual networks, compute instances, virtual machines, etc.) that are in a virtual network built on top of the physical network components. In certain embodiments, the CSPI is organized and hosted in realms, regions and availability domains. A region is typically a localized geographic area that contains one or more data centers. Regions are generally independent of each other and can be separated by vast distances, for example, across countries or even continents. For example, a first region may be in Australia, another one in Japan, yet another one in India, and the like. CSPI resources are divided among regions such that each region has its own independent subset of CSPI resources. Each region may provide a set of core infrastructure services and resources, such as, compute resources (e.g., bare metal servers, virtual machine, containers and related infrastructure, etc.); storage resources (e.g., block volume storage, file storage, object storage, archive storage); networking resources (e.g., virtual cloud networks (VCNs), load balancing resources, connections to on-premise networks), database resources; edge networking resources (e.g., DNS); and access management and monitoring resources, and others. Each region generally has multiple paths connecting it to other regions in the realm.

Generally, an application is deployed in a region (i.e., deployed on infrastructure associated with that region) where it is most heavily used, because using nearby resources is faster than using distant resources. Applications can also be deployed in different regions for various reasons, such as redundancy to mitigate the risk of region-wide events such as large weather systems or earthquakes, to meet varying requirements for legal jurisdictions, tax domains, and other business or social criteria, and the like.

The data centers within a region can be further organized and subdivided into availability domains (ADs). An availability domain may correspond to one or more data centers located within a region. A region can be composed of one or

more availability domains. In such a distributed environment, CSPI resources are either region-specific, such as a virtual cloud network (VCN), or availability domain-specific, such as a compute instance.

ADs within a region are isolated from each other, fault tolerant, and are configured such that they are very unlikely to fail simultaneously. This is achieved by the ADs not sharing critical infrastructure resources such as networking, physical cables, cable paths, cable entry points, etc., such that a failure at one AD within a region is unlikely to impact the availability of the other ADs within the same region. The ADs within the same region may be connected to each other by a low latency, high bandwidth network, which makes it possible to provide high-availability connectivity to other networks (e.g., the Internet, customers' on-premise networks, etc.) and to build replicated systems in multiple ADs for both high-availability and disaster recovery. Cloud services use multiple ADs to ensure high availability and to protect against resource failure. As the infrastructure provided by the IaaS provider grows, more regions and ADs may be added with additional capacity. Traffic between availability domains is usually encrypted.

In certain embodiments, regions are grouped into realms. A realm is a logical collection of regions. Realms are isolated from each other and do not share any data. Regions in the same realm may communicate with each other, but regions in different realms cannot. A customer's tenancy or account with the CSP exists in a single realm and can be spread across one or more regions that belong to that realm. Typically, when a customer subscribes to an IaaS service, a tenancy or account is created for that customer in the customer-specified region (referred to as the "home" region) within a realm. A customer can extend the customer's tenancy across one or more other regions within the realm. A customer cannot access regions that are not in the realm where the customer's tenancy exists.

An IaaS provider can provide multiple realms, each realm catered to a particular set of customers or users. For example, a commercial realm may be provided for commercial customers. As another example, a realm may be provided for a specific country for customers within that country. As yet another example, a government realm may be provided for a government, and the like. For example, the government realm may be catered for a specific government and may have a heightened level of security than a commercial realm. For example, Oracle Cloud Infrastructure (OCI) currently offers a realm for commercial regions and two realms (e.g., FedRAMP authorized and IL5 authorized) for government cloud regions.

In certain embodiments, an AD can be subdivided into one or more fault domains. A fault domain is a grouping of infrastructure resources within an AD to provide anti-affinity. Fault domains allow for the distribution of compute instances such that the instances are not on the same physical hardware within a single AD. This is known as anti-affinity. A fault domain refers to a set of hardware components (computers, switches, and more) that share a single point of failure. A compute pool is logically divided up into fault domains. Due to this, a hardware failure or compute hardware maintenance event that affects one fault domain does not affect instances in other fault domains. Depending on the embodiment, the number of fault domains for each AD may vary. For instance, in certain embodiments each AD contains three fault domains. A fault domain acts as a logical data center within an AD.

When a customer subscribes to an IaaS service, resources from CSPI are provisioned for the customer and associated

with the customer's tenancy. The customer can use these provisioned resources to build private networks and deploy resources on these networks. The customer networks that are hosted in the cloud by the CSPI are referred to as virtual cloud networks (VCNs). A customer can set up one or more virtual cloud networks (VCNs) using CSPI resources allocated for the customer. A VCN is a virtual or software defined private network. The customer resources that are deployed in the customer's VCN can include compute instances (e.g., virtual machines, bare-metal instances) and other resources. These compute instances may represent various customer workloads such as applications, load balancers, databases, and the like. A compute instance deployed on a VCN can communicate with public accessible endpoints ("public endpoints") over a public network such as the Internet, with other instances in the same VCN or other VCNs (e.g., the customer's other VCNs, or VCNs not belonging to the customer), with the customer's on-premise data centers or networks, and with service endpoints, and other types of endpoints.

The CSP may provide various services using the CSPI. In some instances, customers of CSPI may themselves act like service providers and provide services using CSPI resources. A service provider may expose a service endpoint, which is characterized by identification information (e.g., an IP Address, a DNS name and port). A customer's resource (e.g., a compute instance) can consume a particular service by accessing a service endpoint exposed by the service for that particular service. These service endpoints are generally endpoints that are publicly accessible by users using public IP addresses associated with the endpoints via a public communication network such as the Internet. Network endpoints that are publicly accessible are also sometimes referred to as public endpoints.

In certain embodiments, a service provider may expose a service via an endpoint (sometimes referred to as a service endpoint) for the service. Customers of the service can then use this service endpoint to access the service. In certain implementations, a service endpoint provided for a service can be accessed by multiple customers that intend to consume that service. In other implementations, a dedicated service endpoint may be provided for a customer such that only that customer can access the service using that dedicated service endpoint.

In certain embodiments, when a VCN is created, it is associated with a private overlay Classless Inter-Domain Routing (CIDR) address space, which is a range of private overlay IP addresses that are assigned to the VCN (e.g., 10.0/16). A VCN includes associated subnets, route tables, and gateways. A VCN resides within a single region but can span one or more or all of the region's availability domains. A gateway is a virtual interface that is configured for a VCN and enables communication of traffic to and from the VCN to one or more endpoints outside the VCN. One or more different types of gateways may be configured for a VCN to enable communication to and from different types of endpoints.

A VCN can be subdivided into one or more sub-networks such as one or more subnets. A subnet is thus a unit of configuration or a subdivision that can be created within a VCN. A VCN can have one or multiple subnets. Each subnet within a VCN is associated with a contiguous range of overlay IP addresses (e.g., 10.0.0.0/24 and 10.0.1.0/24) that do not overlap with other subnets in that VCN and which represent an address space subset within the address space of the VCN.

Each compute instance is associated with a virtual network interface card (VNIC), that enables the compute instance to participate in a subnet of a VCN. A VNIC is a logical representation of physical Network Interface Card (NIC). In general, a VNIC is an interface between an entity (e.g., a compute instance, a service) and a virtual network. A VNIC exists in a subnet, has one or more associated IP addresses, and associated security rules or policies. A VNIC is equivalent to a Layer-2 port on a switch. A VNIC is attached to a compute instance and to a subnet within a VCN. A VNIC associated with a compute instance enables the compute instance to be a part of a subnet of a VCN and enables the compute instance to communicate (e.g., send and receive packets) with endpoints that are on the same subnet as the compute instance, with endpoints in different subnets in the VCN, or with endpoints outside the VCN. The VNIC associated with a compute instance thus determines how the compute instance connects with endpoints inside and outside the VCN. A VNIC for a compute instance is created and associated with that compute instance when the compute instance is created and added to a subnet within a VCN. For a subnet comprising a set of compute instances, the subnet contains the VNICs corresponding to the set of compute instances, each VNIC attached to a compute instance within the set of computer instances.

Each compute instance is assigned a private overlay IP address via the VNIC associated with the compute instance. This private overlay IP address is assigned to the VNIC that is associated with the compute instance when the compute instance is created and used for routing traffic to and from the compute instance. All VNICs in a given subnet use the same route table, security lists, and DHCP options. As described above, each subnet within a VCN is associated with a contiguous range of overlay IP addresses (e.g., 10.0.0.0/24 and 10.0.1.0/24) that do not overlap with other subnets in that VCN and which represent an address space subset within the address space of the VCN. For a VNIC on a particular subnet of a VCN, the private overlay IP address that is assigned to the VNIC is an address from the contiguous range of overlay IP addresses allocated for the subnet.

In certain embodiments, a compute instance may optionally be assigned additional overlay IP addresses in addition to the private overlay IP address, such as, for example, one or more public IP addresses if in a public subnet. These multiple addresses are assigned either on the same VNIC or over multiple VNICs that are associated with the compute instance. Each instance however has a primary VNIC that is created during instance launch and is associated with the overlay private IP address assigned to the instance—this primary VNIC cannot be removed. Additional VNICs, referred to as secondary VNICs, can be added to an existing instance in the same availability domain as the primary VNIC. All the VNICs are in the same availability domain as the instance. A secondary VNIC can be in a subnet in the same VCN as the primary VNIC, or in a different subnet that is either in the same VCN or a different one.

A compute instance may optionally be assigned a public IP address if it is in a public subnet. A subnet can be designated as either a public subnet or a private subnet at the time the subnet is created. A private subnet means that the resources (e.g., compute instances) and associated VNICs in the subnet cannot have public overlay IP addresses. A public subnet means that the resources and associated VNICs in the subnet can have public IP addresses. A customer can designate a subnet to exist either in a single availability domain or across multiple availability domains in a region or realm.

As described above, a VCN may be subdivided into one or more subnets. In certain embodiments, a Virtual Router (VR) configured for the VCN (referred to as the VCN VR or just VR) enables communications between the subnets of the VCN. For a subnet within a VCN, the VR represents a logical gateway for that subnet that enables the subnet (i.e., the compute instances on that subnet) to communicate with endpoints on other subnets within the VCN, and with other endpoints outside the VCN. The VCN VR is a logical entity that is configured to route traffic between VNICs in the VCN and virtual gateways (“gateways”) associated with the VCN. Gateways are further described below with respect to FIG. 6. A VCN VR is a Layer-3/IP Layer concept. In one embodiment, there is one VCN VR for a VCN where the VCN VR has potentially an unlimited number of ports addressed by IP addresses, with one port for each subnet of the VCN. In this manner, the VCN VR has a different IP address for each subnet in the VCN that the VCN VR is attached to. The VR is also connected to the various gateways configured for a VCN. In certain embodiments, a particular overlay IP address from the overlay IP address range for a subnet is reserved for a port of the VCN VR for that subnet. For example, consider a VCN having two subnets with associated address ranges 10.0/16 and 10.1/16, respectively. For the first subnet within the VCN with address range 10.0/16, an address from this range is reserved for a port of the VCN VR for that subnet. In some instances, the first IP address from the range may be reserved for the VCN VR. For example, for the subnet with overlay IP address range 10.0/16, IP address 10.0.0.1 may be reserved for a port of the VCN VR for that subnet. For the second subnet within the same VCN with address range 10.1/16, the VCN VR may have a port for that second subnet with IP address 10.1.0.1. The VCN VR has a different IP address for each of the subnets in the VCN.

In some other embodiments, each subnet within a VCN may have its own associated VR that is addressable by the subnet using a reserved or default IP address associated with the VR. The reserved or default IP address may, for example, be the first IP address from the range of IP addresses associated with that subnet. The VNICs in the subnet can communicate (e.g., send and receive packets) with the VR associated with the subnet using this default or reserved IP address. In such an embodiment, the VR is the ingress/egress point for that subnet. The VR associated with a subnet within the VCN can communicate with other VRs associated with other subnets within the VCN. The VRs can also communicate with gateways associated with the VCN. The VR function for a subnet is running on or executed by one or more NVDs executing VNICs functionality for VNICs in the subnet.

Route tables, security rules, and DHCP options may be configured for a VCN. Route tables are virtual route tables for the VCN and include rules to route traffic from subnets within the VCN to destinations outside the VCN by way of gateways or specially configured instances. A VCN’s route tables can be customized to control how packets are forwarded/routed to and from the VCN. DHCP options refers to configuration information that is automatically provided to the instances when they boot up.

Security rules configured for a VCN represent overlay firewall rules for the VCN. The security rules can include ingress and egress rules, and specify the types of traffic (e.g., based upon protocol and port) that is allowed in and out of the instances within the VCN. The customer can choose whether a given rule is stateful or stateless. For instance, the customer can allow incoming SSH traffic from anywhere to

a set of instances by setting up a stateful ingress rule with source CIDR 0.0.0.0/0, and destination TCP port 22. Security rules can be implemented using network security groups or security lists. A network security group consists of a set of security rules that apply only to the resources in that group. A security list, on the other hand, includes rules that apply to all the resources in any subnet that uses the security list. A VCN may be provided with a default security list with default security rules. DHCP options configured for a VCN provide configuration information that is automatically provided to the instances in the VCN when the instances boot up.

In certain embodiments, the configuration information for a VCN is determined and stored by a VCN Control Plane. The configuration information for a VCN may include, for example, information about: the address range associated with the VCN, subnets within the VCN and associated information, one or more VRs associated with the VCN, compute instances in the VCN and associated VNICs, NVDs executing the various virtualization network functions (e.g., VNICs, VRs, gateways) associated with the VCN, state information for the VCN, and other VCN-related information. In certain embodiments, a VCN Distribution Service publishes the configuration information stored by the VCN Control Plane, or portions thereof, to the NVDs. The distributed information may be used to update information (e.g., forwarding tables, routing tables, etc.) stored and used by the NVDs to forward packets to and from the compute instances in the VCN.

In certain embodiments, the creation of VCNs and subnets are handled by a VCN Control Plane (CP) and the launching of compute instances is handled by a Compute Control Plane. The Compute Control Plane is responsible for allocating the physical resources for the compute instance and then calls the VCN Control Plane to create and attach VNICs to the compute instance. The VCN CP also sends VCN data mappings to the VCN data plane that is configured to perform packet forwarding and routing functions. In certain embodiments, the VCN CP provides a distribution service that is responsible for providing updates to the VCN data plane. Examples of a VCN Control Plane are also depicted in FIGS. 19, 20, 21, and 22 (see references 1916, 2016, 2116, and 2216) and described below.

A customer may create one or more VCNs using resources hosted by CSPI. A compute instance deployed on a customer VCN may communicate with different endpoints. These endpoints can include endpoints that are hosted by CSPI and endpoints outside CSPI.

Various different architectures for implementing cloud-based service using CSPI are depicted in FIGS. 6, 7, 8, 9, 10, 19, 20, 21, and 23, and are described below. FIG. 6 is a high-level diagram of a distributed environment 600 showing an overlay or customer VCN hosted by CSPI according to certain embodiments. The distributed environment depicted in FIG. 6 includes multiple components in the overlay network. Distributed environment 600 depicted in FIG. 6 is merely an example and is not intended to unduly limit the scope of claimed embodiments. Many variations, alternatives, and modifications are possible. For example, in some implementations, the distributed environment depicted in FIG. 6 may have more or fewer systems or components than those shown in FIG. 1, may combine two or more systems, or may have a different configuration or arrangement of systems.

As shown in the example depicted in FIG. 6, distributed environment 600 comprises CSPI 601 that provides services and resources that customers can subscribe to and use to

build their virtual cloud networks (VCNs). In certain embodiments, CSPI 601 offers IaaS services to subscribing customers. The data centers within CSPI 601 may be organized into one or more regions. One example region "Region US" 602 is shown in FIG. 6. A customer has configured a customer VCN 604 for region 602. The customer may deploy various compute instances on VCN 604, where the compute instances may include virtual machines or bare metal instances. Examples of instances include applications, database, load balancers, and the like.

In the embodiment depicted in FIG. 6, customer VCN 604 comprises two subnets, namely, "Subnet-1" and "Subnet-2", each subnet with its own CIDR IP address range. In FIG. 6, the overlay IP address range for Subnet-1 is 10.0/16 and the address range for Subnet-2 is 10.1/16. A VCN Virtual Router 605 represents a logical gateway for the VCN that enables communications between subnets of the VCN 604, and with other endpoints outside the VCN. VCN VR 605 is configured to route traffic between VNICs in VCN 604 and gateways associated with VCN 604. VCN VR 605 provides a port for each subnet of VCN 604. For example, VR 605 may provide a port with IP address 10.0.0.1 for Subnet-1 and a port with IP address 10.1.0.1 for Subnet-2.

Multiple compute instances may be deployed on each subnet, where the compute instances can be virtual machine instances, and/or bare metal instances. The compute instances in a subnet may be hosted by one or more host machines within CSPI 601. A compute instance participates in a subnet via a VNIC associated with the compute instance. For example, as shown in FIG. 6, a compute instance C1 is part of Subnet-1 via a VNIC associated with the compute instance. Likewise, compute instance C2 is part of Subnet-1 via a VNIC associated with C2. In a similar manner, multiple compute instances, which may be virtual machine instances or bare metal instances, may be part of Subnet-1. Via its associated VNIC, each compute instance is assigned a private overlay IP address and a MAC address. For example, in FIG. 6, compute instance C1 has an overlay IP address of 10.0.0.2 and a MAC address of M1, while compute instance C2 has a private overlay IP address of 10.0.0.3 and a MAC address of M2. Each compute instance in Subnet-1, including compute instances C1 and C2, has a default route to VCN VR 605 using IP address 10.0.0.1, which is the IP address for a port of VCN VR 605 for Subnet-1.

Subnet-2 can have multiple compute instances deployed on it, including virtual machine instances and/or bare metal instances. For example, as shown in FIG. 6, compute instances D1 and D2 are part of Subnet-2 via VNICs associated with the respective compute instances. In the embodiment depicted in FIG. 6, compute instance D1 has an overlay IP address of 10.1.0.2 and a MAC address of MM1, while compute instance D2 has a private overlay IP address of 10.1.0.3 and a MAC address of MM2. Each compute instance in Subnet-2, including compute instances D1 and D2, has a default route to VCN VR 605 using IP address 10.1.0.1, which is the IP address for a port of VCN VR 605 for Subnet-2.

VCN A 604 may also include one or more load balancers. For example, a load balancer may be provided for a subnet and may be configured to load balance traffic across multiple compute instances on the subnet. A load balancer may also be provided to load balance traffic across subnets in the VCN.

A particular compute instance deployed on VCN 604 can communicate with various different endpoints. These endpoints may include endpoints that are hosted by CSPI 700 and endpoints outside CSPI 700. Endpoints that are hosted

by CSPI 601 may include: an endpoint on the same subnet as the particular compute instance (e.g., communications between two compute instances in Subnet-1); an endpoint on a different subnet but within the same VCN (e.g., communication between a compute instance in Subnet-1 and a compute instance in Subnet-2); an endpoint in a different VCN in the same region (e.g., communications between a compute instance in Subnet-1 and an endpoint in a VCN in the same region 606 or 610, communications between a compute instance in Subnet-1 and an endpoint in service network 610 in the same region); or an endpoint in a VCN in a different region (e.g., communications between a compute instance in Subnet-1 and an endpoint in a VCN in a different region 608). A compute instance in a subnet hosted by CSPI 601 may also communicate with endpoints that are not hosted by CSPI 601 (i.e., are outside CSPI 601). These outside endpoints include endpoints in the customer's on-premise network 616, endpoints within other remote cloud hosted networks 618, public endpoints 614 accessible via a public network such as the Internet, and other endpoints.

Communications between compute instances on the same subnet are facilitated using VNICs associated with the source compute instance and the destination compute instance. For example, compute instance C1 in Subnet-1 may want to send packets to compute instance C2 in Subnet-1. For a packet originating at a source compute instance and whose destination is another compute instance in the same subnet, the packet is first processed by the VNIC associated with the source compute instance. Processing performed by the VNIC associated with the source compute instance can include determining destination information for the packet from the packet headers, identifying any policies (e.g., security lists) configured for the VNIC associated with the source compute instance, determining a next hop for the packet, performing any packet encapsulation/decapsulation functions as needed, and then forwarding/routing the packet to the next hop with the goal of facilitating communication of the packet to its intended destination. When the destination compute instance is in the same subnet as the source compute instance, the VNIC associated with the source compute instance is configured to identify the VNIC associated with the destination compute instance and forward the packet to that VNIC for processing. The VNIC associated with the destination compute instance is then executed and forwards the packet to the destination compute instance.

For a packet to be communicated from a compute instance in a subnet to an endpoint in a different subnet in the same VCN, the communication is facilitated by the VNICs associated with the source and destination compute instances and the VCN VR. For example, if compute instance C1 in Subnet-1 in FIG. 6 wants to send a packet to compute instance D1 in Subnet-2, the packet is first processed by the VNIC associated with compute instance C1. The VNIC associated with compute instance C1 is configured to route the packet to the VCN VR 605 using default route or port 10.0.0.1 of the VCN VR. VCN VR 605 is configured to route the packet to Subnet-2 using port 10.1.0.1. The packet is then received and processed by the VNIC associated with D1 and the VNIC forwards the packet to compute instance D1.

For a packet to be communicated from a compute instance in VCN 604 to an endpoint that is outside VCN 604, the communication is facilitated by the VNIC associated with the source compute instance, VCN VR 605, and gateways associated with VCN 604. One or more types of gateways may be associated with VCN 604. A gateway is an interface between a VCN and another endpoint, where the another

31

endpoint is outside the VCN. A gateway is a Layer-3/IP layer concept and enables a VCN to communicate with endpoints outside the VCN. A gateway thus facilitates traffic flow between a VCN and other VCNs or networks. Various different types of gateways may be configured for a VCN to facilitate different types of communications with different types of endpoints. Depending upon the gateway, the communications may be over public networks (e.g., the Internet) or over private networks. Various communication protocols may be used for these communications.

For example, compute instance C1 may want to communicate with an endpoint outside VCN 604. The packet may be first processed by the VNIC associated with source compute instance C1. The VNIC processing determines that the destination for the packet is outside the Subnet-1 of C1. The VNIC associated with C1 may forward the packet to VCN VR 605 for VCN 604. VCN VR 605 then processes the packet and as part of the processing, based upon the destination for the packet, determines a particular gateway associated with VCN 604 as the next hop for the packet. VCN VR 605 may then forward the packet to the particular identified gateway. For example, if the destination is an endpoint within the customer's on-premise network, then the packet may be forwarded by VCN VR 605 to Dynamic Routing Gateway (DRG) gateway 622 configured for VCN 604. The packet may then be forwarded from the gateway to a next hop to facilitate communication of the packet to its final intended destination.

Various different types of gateways may be configured for a VCN. Examples of gateways that may be configured for a VCN are depicted in FIG. 6 and described below. Examples of gateways associated with a VCN are also depicted in FIGS. 19, 20, 21, and 22 (for example, gateways referenced by reference numbers 1934, 1936, 1938, 2034, 2036, 2038, 2134, 2136, 2138, 2234, 2236, and 2238) and described below. As shown in the embodiment depicted in FIG. 6, a Dynamic Routing Gateway (DRG) 622 may be added to or be associated with customer VCN 604 and provides a path for private network traffic communication between customer VCN 604 and another endpoint, where the another endpoint can be the customer's on-premise network 616, a VCN 608 in a different region of CSPI 601, or other remote cloud networks 618 not hosted by CSPI 601. Customer on-premise network 616 may be a customer network or a customer data center built using the customer's resources. Access to customer on-premise network 616 is generally very restricted. For a customer that has both a customer on-premise network 616 and one or more VCNs 604 deployed or hosted in the cloud by CSPI 601, the customer may want their on-premise network 616 and their cloud-based VCN 604 to be able to communicate with each other. This enables a customer to build an extended hybrid environment encompassing the customer's VCN 604 hosted by CSPI 601 and their on-premises network 616. DRG 622 enables this communication. To enable such communications, a communication channel 624 is set up where one endpoint of the channel is in customer on-premise network 616 and the other endpoint is in CSPI 601 and connected to customer VCN 604. Communication channel 624 can be over public communication networks such as the Internet or private communication networks. Various different communication protocols may be used such as IPsec VPN technology over a public communication network such as the Internet, Oracle's FastConnect technology that uses a private network instead of a public network, and others. The device or equipment in customer on-premise network 616 that forms one end point for communication channel 624 is referred to as the cus-

32

tomers premise equipment (CPE), such as CPE 626 depicted in FIG. 6. On the CSPI 601 side, the endpoint may be a host machine executing DRG 622.

In certain embodiments, a Remote Peering Connection (RPC) can be added to a DRG, which allows a customer to peer one VCN with another VCN in a different region. Using such an RPC, customer VCN 604 can use DRG 622 to connect with a VCN 608 in another region. DRG 622 may also be used to communicate with other remote cloud networks 618, not hosted by CSPI 601 such as a Microsoft Azure cloud, Amazon AWS cloud, and others.

As shown in FIG. 6, an Internet Gateway (IGW) 620 may be configured for customer VCN 604 that enables a compute instance on VCN 604 to communicate with public endpoints 614 accessible over a public network such as the Internet. IGW 620 is a gateway that connects a VCN to a public network such as the Internet. IGW 620 enables a public subnet (where the resources in the public subnet have public overlay IP addresses) within a VCN, such as VCN 604, direct access to public endpoints 612 on a public network 614 such as the Internet. Using IGW 620, connections can be initiated from a subnet within VCN 604 or from the Internet.

A Network Address Translation (NAT) gateway 628 can be configured for customer's VCN 604 and enables cloud resources in the customer's VCN, which do not have dedicated public overlay IP addresses, access to the Internet and it does so without exposing those resources to direct incoming Internet connections (e.g., L4-L7 connections). This enables a private subnet within a VCN, such as private Subnet-1 in VCN 604, with private access to public endpoints on the Internet. In NAT gateways, connections can be initiated only from the private subnet to the public Internet and not from the Internet to the private subnet.

In certain embodiments, a Service Gateway (SGW) 626 can be configured for customer VCN 604 and provides a path for private network traffic between VCN 604 and supported services endpoints in a service network 610. In certain embodiments, service network 610 may be provided by the CSP and may provide various services. An example of such a service network is Oracle's Services Network, which provides various services that can be used by customers. For example, a compute instance (e.g., a database system) in a private subnet of customer VCN 604 can back up data to a service endpoint (e.g., Object Storage) without needing public IP addresses or access to the Internet. In certain embodiments, a VCN can have only one SGW, and connections can only be initiated from a subnet within the VCN and not from service network 610. If a VCN is peered with another, resources in the other VCN typically cannot access the SGW. Resources in on-premises networks that are connected to a VCN with FastConnect or VPN Connect can also use the service gateway configured for that VCN.

In certain implementations, SGW 626 uses the concept of a service Classless Inter-Domain Routing (CIDR) label, which is a string that represents all the regional public IP address ranges for the service or group of services of interest. The customer uses the service CIDR label when they configure the SGW and related route rules to control traffic to the service. The customer can optionally utilize it when configuring security rules without needing to adjust them if the service's public IP addresses change in the future.

A Local Peering Gateway (LPG) 632 is a gateway that can be added to customer VCN 604 and enables VCN 604 to peer with another VCN in the same region. Peering means that the VCNs communicate using private IP addresses,

without the traffic traversing a public network such as the Internet or without routing the traffic through the customer's on-premises network **616**. In preferred embodiments, a VCN has a separate LPG for each peering it establishes. Local Peering or VCN Peering is a common practice used to establish network connectivity between different applications or infrastructure management functions.

Service providers, such as providers of services in service network **610**, may provide access to services using different access models. According to a public access model, services may be exposed as public endpoints that are publicly accessible by compute instance in a customer VCN via a public network such as the Internet and or may be privately accessible via SGW **626**. According to a specific private access model, services are made accessible as private IP endpoints in a private subnet in the customer's VCN. This is referred to as a Private Endpoint (PE) access and enables a service provider to expose their service as an instance in the customer's private network. A Private Endpoint resource represents a service within the customer's VCN. Each PE manifests as a VNIC (referred to as a PE-VNIC, with one or more private IPs) in a subnet chosen by the customer in the customer's VCN. A PE thus provides a way to present a service within a private customer VCN subnet using a VNIC. Since the endpoint is exposed as a VNIC, all the features associates with a VNIC such as routing rules, security lists, etc., are now available for the PE VNIC.

A service provider can register their service to enable access through a PE. The provider can associate policies with the service that restricts the service's visibility to the customer tenancies. A provider can register multiple services under a single virtual IP address (VIP), especially for multi-tenant services. There may be multiple such private endpoints (in multiple VCNs) that represent the same service.

Compute instances in the private subnet can then use the PE VNIC's private IP address or the service DNS name to access the service. Compute instances in the customer VCN can access the service by sending traffic to the private IP address of the PE in the customer VCN. A Private Access Gateway (PAGW) **630** is a gateway resource that can be attached to a service provider VCN (e.g., a VCN in service network **610**) that acts as an ingress/egress point for all traffic from/to customer subnet private endpoints. PAGW **630** enables a provider to scale the number of PE connections without utilizing its internal IP address resources. A provider needs only configure one PAGW for any number of services registered in a single VCN. Providers can represent a service as a private endpoint in multiple VCNs of one or more customers. From the customer's perspective, the PE VNIC, which, instead of being attached to a customer's instance, appears attached to the service with which the customer wishes to interact. The traffic destined to the private endpoint is routed via PAGW **630** to the service. These are referred to as customer-to-service private connections (C2S connections).

The PE concept can also be used to extend the private access for the service to customer's on-premises networks and data centers, by allowing the traffic to flow through FastConnect/IPsec links and the private endpoint in the customer VCN. Private access for the service can also be extended to the customer's peered VCNs, by allowing the traffic to flow between LPG **632** and the PE in the customer's VCN.

A customer can control routing in a VCN at the subnet level, so the customer can specify which subnets in the customer's VCN, such as VCN **604**, use each gateway. A

VCN's route tables are used to decide if traffic is allowed out of a VCN through a particular gateway. For example, in a particular instance, a route table for a public subnet within customer VCN **604** may send non-local traffic through IGW **620**. The route table for a private subnet within the same customer VCN **604** may send traffic destined for CSP services through SGW **626**. All remaining traffic may be sent via the NAT gateway **628**. Route tables only control traffic going out of a VCN.

Security lists associated with a VCN are used to control traffic that comes into a VCN via a gateway via inbound connections. All resources in a subnet use the same route table and security lists. Security lists may be used to control specific types of traffic allowed in and out of instances in a subnet of a VCN. Security list rules may comprise ingress (inbound) and egress (outbound) rules. For example, an ingress rule may specify an allowed source address range, while an egress rule may specify an allowed destination address range. Security rules may specify a particular protocol (e.g., TCP, ICMP), a particular port (e.g., 22 for SSH, 3389 for Windows RDP), etc. In certain implementations, an instance's operating system may enforce its own firewall rules that are aligned with the security list rules. Rules may be stateful (e.g., a connection is tracked and the response is automatically allowed without an explicit security list rule for the response traffic) or stateless.

Access from a customer VCN (i.e., by a resource or compute instance deployed on VCN **604**) can be categorized as public access, private access, or dedicated access. Public access refers to an access model where a public IP address or a NAT is used to access a public endpoint. Private access enables customer workloads in VCN **604** with private IP addresses (e.g., resources in a private subnet) to access services without traversing a public network such as the Internet. In certain embodiments, CSPI **601** enables customer VCN workloads with private IP addresses to access the (public service endpoints of) services using a service gateway. A service gateway thus offers a private access model by establishing a virtual link between the customer's VCN and the service's public endpoint residing outside the customer's private network.

Additionally, CSPI may offer dedicated public access using technologies such as FastConnect public peering where customer on-premises instances can access one or more services in a customer VCN using a FastConnect connection and without traversing a public network such as the Internet. CSPI also may also offer dedicated private access using FastConnect private peering where customer on-premises instances with private IP addresses can access the customer's VCN workloads using a FastConnect connection. FastConnect is a network connectivity alternative to using the public Internet to connect a customer's on-premise network to CSPI and its services. FastConnect provides an easy, elastic, and economical way to create a dedicated and private connection with higher bandwidth options and a more reliable and consistent networking experience when compared to Internet-based connections.

FIG. 6 and the accompanying description above describes various virtualized components in an example virtual network. As described above, the virtual network is built on the underlying physical or substrate network. FIG. 7 depicts a simplified architectural diagram of the physical components in the physical network within CSPI **700** that provide the underlay for the virtual network according to certain embodiments. As shown, CSPI **700** provides a distributed environment comprising components and resources (e.g., compute, memory, and networking resources) provided by a

cloud service provider (CSP). These components and resources are used to provide cloud services (e.g., IaaS services) to subscribing customers, i.e., customers that have subscribed to one or more services provided by the CSP. Based upon the services subscribed to by a customer, a subset of resources (e.g., compute, memory, and networking resources) of CSPI 700 are provisioned for the customer. Customers can then build their own cloud-based (i.e., CSPI-hosted) customizable and private virtual networks using physical compute, memory, and networking resources provided by CSPI 700. As previously indicated, these customer networks are referred to as virtual cloud networks (VCNs). A customer can deploy one or more customer resources, such as compute instances, on these customer VCNs. Compute instances can be in the form of virtual machines, bare metal instances, and the like. CSPI 700 provides infrastructure and a set of complementary cloud services that enable customers to build and run a wide range of applications and services in a highly available hosted environment.

In the example embodiment depicted in FIG. 7, the physical components of CSPI 700 include one or more physical host machines or physical servers (e.g., 702, 706, 708), network virtualization devices (NVDs) (e.g., 710, 712), top-of-rack (TOR) switches (e.g., 714, 716), and a physical network (e.g., 718), and switches in physical network 718. The physical host machines or servers may host and execute various compute instances that participate in one or more subnets of a VCN. The compute instances may include virtual machine instances, and bare metal instances. For example, the various compute instances depicted in FIG. 6 may be hosted by the physical host machines depicted in FIG. 7. The virtual machine compute instances in a VCN may be executed by one host machine or by multiple different host machines. The physical host machines may also host virtual host machines, container-based hosts or functions, and the like. The VNICs and VCN VR depicted in FIG. 6 may be executed by the NVDs depicted in FIG. 7. The gateways depicted in FIG. 6 may be executed by the host machines and/or by the NVDs depicted in FIG. 7.

The host machines or servers may execute a hypervisor (also referred to as a virtual machine monitor or VMM) that creates and enables a virtualized environment on the host machines. The virtualization or virtualized environment facilitates cloud-based computing. One or more compute instances may be created, executed, and managed on a host machine by a hypervisor on that host machine. The hypervisor on a host machine enables the physical computing resources of the host machine (e.g., compute, memory, and networking resources) to be shared between the various compute instances executed by the host machine.

For example, as depicted in FIG. 7, host machines 702 and 708 execute hypervisors 760 and 766, respectively. These hypervisors may be implemented using software, firmware, or hardware, or combinations thereof. Typically, a hypervisor is a process or a software layer that sits on top of the host machine's operating system (OS), which in turn executes on the hardware processors of the host machine. The hypervisor provides a virtualized environment by enabling the physical computing resources (e.g., processing resources such as processors/cores, memory resources, networking resources) of the host machine to be shared among the various virtual machine compute instances executed by the host machine. For example, in FIG. 7, hypervisor 760 may sit on top of the OS of host machine 702 and enables the computing resources (e.g., processing, memory, and networking resources) of host machine 702 to be shared between compute instances (e.g., virtual machines) executed

by host machine 702. A virtual machine can have its own operating system (referred to as a guest operating system), which may be the same as or different from the OS of the host machine. The operating system of a virtual machine executed by a host machine may be the same as or different from the operating system of another virtual machine executed by the same host machine. A hypervisor thus enables multiple operating systems to be executed alongside each other while sharing the same computing resources of the host machine. The host machines depicted in FIG. 7 may have the same or different types of hypervisors.

A compute instance can be a virtual machine instance or a bare metal instance. In FIG. 7, compute instances 768 on host machine 702 and 774 on host machine 708 are examples of virtual machine instances. Host machine 706 is an example of a bare metal instance that is provided to a customer.

In certain instances, an entire host machine may be provisioned to a single customer, and all of the one or more compute instances (either virtual machines or bare metal instance) hosted by that host machine belong to that same customer. In other instances, a host machine may be shared between multiple customers (i.e., multiple tenants). In such a multi-tenancy scenario, a host machine may host virtual machine compute instances belonging to different customers. These compute instances may be members of different VCNs of different customers. In certain embodiments, a bare metal compute instance is hosted by a bare metal server without a hypervisor. When a bare metal compute instance is provisioned, a single customer or tenant maintains control of the physical CPU, memory, and network interfaces of the host machine hosting the bare metal instance and the host machine is not shared with other customers or tenants.

As previously described, each compute instance that is part of a VCN is associated with a VNIC that enables the compute instance to become a member of a subnet of the VCN. The VNIC associated with a compute instance facilitates the communication of packets or frames to and from the compute instance. A VNIC is associated with a compute instance when the compute instance is created. In certain embodiments, for a compute instance executed by a host machine, the VNIC associated with that compute instance is executed by an NVD connected to the host machine. For example, in FIG. 7, host machine 702 executes a virtual machine compute instance 768 that is associated with VNIC 776, and VNIC 776 is executed by NVD 710 connected to host machine 702. As another example, bare metal instance 772 hosted by host machine 706 is associated with VNIC 780 that is executed by NVD 712 connected to host machine 706. As yet another example, VNIC 784 is associated with compute instance 774 executed by host machine 708, and VNIC 784 is executed by NVD 712 connected to host machine 708.

For compute instances hosted by a host machine, an NVD connected to that host machine also executes VCN VRs corresponding to VCNs of which the compute instances are members. For example, in the embodiment depicted in FIG. 7, NVD 710 executes VCN VR 777 corresponding to the VCN of which compute instance 768 is a member. NVD 712 may also execute one or more VCN VRs 783 corresponding to VCNs corresponding to the compute instances hosted by host machines 706 and 708.

A host machine may include one or more network interface cards (NIC) that enable the host machine to be connected to other devices. A NIC on a host machine may provide one or more ports (or interfaces) that enable the host machine to be communicatively connected to another

device. For example, a host machine may be connected to an NVD using one or more ports (or interfaces) provided on the host machine and on the NVD. A host machine may also be connected to other devices such as another host machine.

For example, in FIG. 7, host machine 702 is connected to NVD 710 using link 720 that extends between a port 734 provided by a NIC 732 of host machine 702 and between a port 736 of NVD 710. Host machine 706 is connected to NVD 712 using link 724 that extends between a port 746 provided by a NIC of host machine 706 and between a port 748 of NVD 712. Host machine 708 is connected to NVD 712 using link 726 that extends between a port 752 provided by a NIC 750 of host machine 708 and between a port 754 of NVD 712.

The NVDs are in turn connected via communication links to top-of-the-rack (TOR) switches, which are connected to physical network 718 (also referred to as the switch fabric). In certain embodiments, the links between a host machine and an NVD, and between an NVD and a TOR switch are Ethernet links. For example, in FIG. 7, NVDs 710 and 712 are connected to TOR switches 714 and 716, respectively, using links 728 and 730. In certain embodiments, the links 720, 724, 726, 728, and 730 are Ethernet links. The collection of host machines and NVDs that are connected to a TOR is sometimes referred to as a rack.

Physical network 718 provides a communication fabric that enables TOR switches to communicate with each other. Physical network 718 can be a multi-tiered network. In certain implementations, physical network 718 is a multi-tiered Clos network of switches, with TOR switches 714 and 716 representing the leaf level nodes of the multi-tiered and multi-node physical switching network 718. Different Clos network configurations are possible including but not limited to a 2-tier network, a 3-tier network, a 4-tier network, a 5-tier network, and in general a “n”-tiered network. An example of a Clos network is depicted in FIG. 10 and described below.

Various different connection configurations are possible between host machines and NVDs such as one-to-one configuration, many-to-one configuration, one-to-many configuration, and others. In a one-to-one configuration implementation, each host machine is connected to its own separate NVD. For example, in FIG. 7, host machine 702 is connected to NVD 710 via NIC 732 of host machine 702. In a many-to-one configuration, multiple host machines are connected to one NVD. For example, in FIG. 7, host machines 706 and 708 are connected to the same NVD 712 via NICs 744 and 750, respectively.

In a one-to-many configuration, one host machine is connected to multiple NVDs. FIG. 8 shows an example within CSPI 800 where a host machine is connected to multiple NVDs. As shown in FIG. 8, host machine 802 comprises a network interface card (NIC) 804 that includes multiple ports 806 and 808. Host machine 800 is connected to a first NVD 810 via port 806 and link 820, and connected to a second NVD 812 via port 808 and link 822. Ports 806 and 808 may be Ethernet ports and the links 820 and 822 between host machine 802 and NVDs 810 and 812 may be Ethernet links. NVD 810 is in turn connected to a first TOR switch 814 and NVD 812 is connected to a second TOR switch 816. The links between NVDs 810 and 812, and TOR switches 814 and 816 may be Ethernet links. TOR switches 814 and 816 represent the Tier-0 switching devices in multi-tiered physical network 818.

The arrangement depicted in FIG. 8 provides two separate physical network paths to and from physical switch network 818 to host machine 802: a first path traversing TOR switch

814 to NVD 810 to host machine 802, and a second path traversing TOR switch 816 to NVD 812 to host machine 802. The separate paths provide for enhanced availability (referred to as high availability) of host machine 802. If there are problems in one of the paths (e.g., a link in one of the paths goes down) or devices (e.g., a particular NVD is not functioning), then the other path may be used for communications to/from host machine 802.

In the configuration depicted in FIG. 8, the host machine is connected to two different NVDs using two different ports provided by a NIC of the host machine. In other embodiments, a host machine may include multiple NICs that enable connectivity of the host machine to multiple NVDs.

Referring back to FIG. 7, an NVD is a physical device or component that performs one or more network and/or storage virtualization functions. An NVD may be any device with one or more processing units (e.g., CPUs, Network Processing Units (NPU)s, FPGAs, packet processing pipelines, etc.), memory including cache, and ports. The various virtualization functions may be performed by software/firmware executed by the one or more processing units of the NVD.

An NVD may be implemented in various different forms. For example, in certain embodiments, an NVD is implemented as an interface card referred to as a smartNIC or an intelligent NIC with an embedded processor onboard. A smartNIC is a separate device from the NICs on the host machines. In FIG. 7, the NVDs 710 and 712 may be implemented as smartNICs that are connected to host machines 702, and host machines 706 and 708, respectively.

A smartNIC is however just one example of an NVD implementation. Various other implementations are possible. For example, in some other implementations, an NVD or one or more functions performed by the NVD may be incorporated into or performed by one or more host machines, one or more TOR switches, and other components of CSPI 700. For example, an NVD may be embodied in a host machine where the functions performed by an NVD are performed by the host machine. As another example, an NVD may be part of a TOR switch or a TOR switch may be configured to perform functions performed by an NVD that enables the TOR switch to perform various complex packet transformations that are used for a public cloud. A TOR that performs the functions of an NVD is sometimes referred to as a smart TOR. In yet other implementations, where virtual machines (VMs) instances, but not bare metal (BM) instances, are offered to customers, functions performed by an NVD may be implemented inside a hypervisor of the host machine. In some other implementations, some of the functions of the NVD may be offloaded to a centralized service running on a fleet of host machines.

In certain embodiments, such as when implemented as a smartNIC as shown in FIG. 7, an NVD may comprise multiple physical ports that enable it to be connected to one or more host machines and to one or more TOR switches. A port on an NVD can be classified as a host-facing port (also referred to as a “south port”) or a network-facing or TOR-facing port (also referred to as a “north port”). A host-facing port of an NVD is a port that is used to connect the NVD to a host machine. Examples of host-facing ports in FIG. 7 include port 736 on NVD 710, and ports 748 and 754 on NVD 712. A network-facing port of an NVD is a port that is used to connect the NVD to a TOR switch. Examples of network-facing ports in FIG. 7 include port 756 on NVD 710, and port 758 on NVD 712. As shown in FIG. 7, NVD 710 is connected to TOR switch 714 using link 728 that extends from port 756 of NVD 710 to the TOR switch 714.

Likewise, NVD **712** is connected to TOR switch **716** using link **730** that extends from port **758** of NVD **712** to the TOR switch **716**.

An NVD receives packets and frames from a host machine (e.g., packets and frames generated by a compute instance hosted by the host machine) via a host-facing port and, after performing the necessary packet processing, may forward the packets and frames to a TOR switch via a network-facing port of the NVD. An NVD may receive packets and frames from a TOR switch via a network-facing port of the NVD and, after performing the necessary packet processing, may forward the packets and frames to a host machine via a host-facing port of the NVD.

In certain embodiments, there may be multiple ports and associated links between an NVD and a TOR switch. These ports and links may be aggregated to form a link aggregator group of multiple ports or links (referred to as a LAG). Link aggregation allows multiple physical links between two end-points (e.g., between an NVD and a TOR switch) to be treated as a single logical link. All the physical links in a given LAG may operate in full-duplex mode at the same speed. LAGs help increase the bandwidth and reliability of the connection between two endpoints. If one of the physical links in the LAG goes down, traffic is dynamically and transparently reassigned to one of the other physical links in the LAG. The aggregated physical links deliver higher bandwidth than each individual link. The multiple ports associated with a LAG are treated as a single logical port. Traffic can be load-balanced across the multiple physical links of a LAG. One or more LAGs may be configured between two endpoints. The two endpoints may be between an NVD and a TOR switch, between a host machine and an NVD, and the like.

An NVD implements or performs network virtualization functions. These functions are performed by software/firmware executed by the NVD. Examples of network virtualization functions include without limitation: packet encapsulation and de-capsulation functions; functions for creating a VCN network; functions for implementing network policies such as VCN security list (firewall) functionality; functions that facilitate the routing and forwarding of packets to and from compute instances in a VCN; and the like. In certain embodiments, upon receiving a packet, an NVD is configured to execute a packet processing pipeline for processing the packet and determining how the packet is to be forwarded or routed. As part of this packet processing pipeline, the NVD may execute one or more virtual functions associated with the overlay network such as executing VNICs associated with compute instances in the VCN, executing a Virtual Router (VR) associated with the VCN, the encapsulation and decapsulation of packets to facilitate forwarding or routing in the virtual network, execution of certain gateways (e.g., the Local Peering Gateway), the implementation of Security Lists, Network Security Groups, network address translation (NAT) functionality (e.g., the translation of Public IP to Private IP on a host by host basis), throttling functions, and other functions.

In certain embodiments, the packet processing data path in an NVD may comprise multiple packet pipelines, each composed of a series of packet transformation stages. In certain implementations, upon receiving a packet, the packet is parsed and classified to a single pipeline. The packet is then processed in a linear fashion, one stage after another, until the packet is either dropped or sent out over an interface of the NVD. These stages provide basic functional packet processing building blocks (e.g., validating headers, enforcing throttle, inserting new Layer-2 headers, enforcing

L4 firewall, VCN encapsulation/decapsulation, etc.) so that new pipelines can be constructed by composing existing stages, and new functionality can be added by creating new stages and inserting them into existing pipelines.

An NVD may perform both control plane and data plane functions corresponding to a control plane and a data plane of a VCN. Examples of a VCN Control Plane are also depicted in FIGS. **19**, **20**, **21**, and **22** (see references **1916**, **2016**, **2116**, and **2216**) and described below. Examples of a VCN Data Plane are depicted in FIGS. **19**, **20**, **21**, and **22** (see references **1918**, **2018**, **2118**, and **2218**) and described below. The control plane functions include functions used for configuring a network (e.g., setting up routes and route tables, configuring VNICs, etc.) that controls how data is to be forwarded. In certain embodiments, a VCN Control Plane is provided that computes all the overlay-to-substrate mappings centrally and publishes them to the NVDs and to the virtual network edge devices such as various gateways such as the DRG, the SGW, the IGW, etc. Firewall rules may also be published using the same mechanism. In certain embodiments, an NVD only gets the mappings that are relevant for that NVD. The data plane functions include functions for the actual routing/forwarding of a packet based upon configuration set up using control plane. A VCN data plane is implemented by encapsulating the customer's network packets before they traverse the substrate network. The encapsulation/decapsulation functionality is implemented on the NVDs. In certain embodiments, an NVD is configured to intercept all network packets in and out of host machines and perform network virtualization functions.

As indicated above, an NVD executes various virtualization functions including VNICs and VCN VRs. An NVD may execute VNICs associated with the compute instances hosted by one or more host machines connected to the VNIC. For example, as depicted in FIG. **7**, NVD **710** executes the functionality for VNIC **776** that is associated with compute instance **768** hosted by host machine **702** connected to NVD **710**. As another example, NVD **712** executes VNIC **780** that is associated with bare metal compute instance **772** hosted by host machine **706**, and executes VNIC **784** that is associated with compute instance **774** hosted by host machine **708**. A host machine may host compute instances belonging to different VCNs, which belong to different customers, and the NVD connected to the host machine may execute the VNICs (i.e., execute VNICs-related functionality) corresponding to the compute instances.

An NVD also executes VCN Virtual Routers corresponding to the VCNs of the compute instances. For example, in the embodiment depicted in FIG. **7**, NVD **710** executes VCN VR **777** corresponding to the VCN to which compute instance **768** belongs. NVD **712** executes one or more VCN VRs **783** corresponding to one or more VCNs to which compute instances hosted by host machines **706** and **708** belong. In certain embodiments, the VCN VR corresponding to that VCN is executed by all the NVDs connected to host machines that host at least one compute instance belonging to that VCN. If a host machine hosts compute instances belonging to different VCNs, an NVD connected to that host machine may execute VCN VRs corresponding to those different VCNs.

In addition to VNICs and VCN VRs, an NVD may execute various software (e.g., daemons) and include one or more hardware components that facilitate the various network virtualization functions performed by the NVD. For purposes of simplicity, these various components are grouped together as "packet processing components" shown in FIG. **7**. For example, NVD **710** comprises packet pro-

cessing components **786** and NVD **712** comprises packet processing components **788**. For example, the packet processor that is configured to interact with the NVD's ports and hardware interfaces to monitor all packets received by and communicated using the NVD and store network information. The network information may, for example, include network flow information identifying different network flows handled by the NVD and per flow information (e.g., per flow statistics). In certain embodiments, network flows information may be stored on a per VNIC basis. The packet processor may perform packet-by-packet manipulations as well as implement stateful NAT and L4 firewall (FW). As another example, the packet processing components may include a replication agent that is configured to replicate information stored by the NVD to one or more different replication target stores. As yet another example, the packet processing components may include a logging agent that is configured to perform logging functions for the NVD. The packet processing components may also include software for monitoring the performance and health of the NVD and, also possibly of monitoring the state and health of other components connected to the NVD.

FIG. 6 shows the components of an example virtual or overlay network including a VCN, subnets within the VCN, compute instances deployed on subnets, VNICs associated with the compute instances, a VR for a VCN, and a set of gateways configured for the VCN. The overlay components depicted in FIG. 6 may be executed or hosted by one or more of the physical components depicted in FIG. 7. For example, the compute instances in a VCN may be executed or hosted by one or more host machines depicted in FIG. 7. For a compute instance hosted by a host machine, the VNIC associated with that compute instance is typically executed by an NVD connected to that host machine (i.e., the VNIC functionality is provided by the NVD connected to that host machine). The VCN VR function for a VCN is executed by all the NVDs that are connected to host machines hosting or executing the compute instances that are part of that VCN. The gateways associated with a VCN may be executed by one or more different types of NVDs. For example, certain gateways may be executed by smartNICs, while others may be executed by one or more host machines or other implementations of NVDs.

As described above, a compute instance in a customer VCN may communicate with various different endpoints, where the endpoints can be within the same subnet as the source compute instance, in a different subnet but within the same VCN as the source compute instance, or with an endpoint that is outside the VCN of the source compute instance. These communications are facilitated using VNICs associated with the compute instances, the VCN VRs, and the gateways associated with the VCNs.

For communications between two compute instances on the same subnet in a VCN, the communication is facilitated using VNICs associated with the source and destination compute instances. The source and destination compute instances may be hosted by the same host machine or by different host machines. A packet originating from a source compute instance may be forwarded from a host machine hosting the source compute instance to an NVD connected to that host machine. On the NVD, the packet is processed using a packet processing pipeline, which can include execution of the VNIC associated with the source compute instance. Since the destination endpoint for the packet is within the same subnet, execution of the VNIC associated with the source compute instance results in the packet being

forwarded to an NVD executing the VNIC associated with the destination compute instance, which then processes and forwards the packet to the destination compute instance. The VNICs associated with the source and destination compute instances may be executed on the same NVD (e.g., when both the source and destination compute instances are hosted by the same host machine) or on different NVDs (e.g., when the source and destination compute instances are hosted by different host machines connected to different NVDs). The VNICs may use routing/forwarding tables stored by the NVD to determine the next hop for the packet.

For a packet to be communicated from a compute instance in a subnet to an endpoint in a different subnet in the same VCN, the packet originating from the source compute instance is communicated from the host machine hosting the source compute instance to the NVD connected to that host machine. On the NVD, the packet is processed using a packet processing pipeline, which can include execution of one or more VNICs, and the VR associated with the VCN. For example, as part of the packet processing pipeline, the NVD executes or invokes functionality corresponding to the VNIC (also referred to as executes the VNIC) associated with source compute instance. The functionality performed by the VNIC may include looking at the VLAN tag on the packet. Since the packet's destination is outside the subnet, the VCN VR functionality is next invoked and executed by the NVD. The VCN VR then routes the packet to the NVD executing the VNIC associated with the destination compute instance. The VNIC associated with the destination compute instance then processes the packet and forwards the packet to the destination compute instance. The VNICs associated with the source and destination compute instances may be executed on the same NVD (e.g., when both the source and destination compute instances are hosted by the same host machine) or on different NVDs (e.g., when the source and destination compute instances are hosted by different host machines connected to different NVDs).

If the destination for the packet is outside the VCN of the source compute instance, then the packet originating from the source compute instance is communicated from the host machine hosting the source compute instance to the NVD connected to that host machine. The NVD executes the VNIC associated with the source compute instance. Since the destination end point of the packet is outside the VCN, the packet is then processed by the VCN VR for that VCN. The NVD invokes the VCN VR functionality, which may result in the packet being forwarded to an NVD executing the appropriate gateway associated with the VCN. For example, if the destination is an endpoint within the customer's on-premise network, then the packet may be forwarded by the VCN VR to the NVD executing the DRG gateway configured for the VCN. The VCN VR may be executed on the same NVD as the NVD executing the VNIC associated with the source compute instance or by a different NVD. The gateway may be executed by an NVD, which may be a smartNIC, a host machine, or other NVD implementation. The packet is then processed by the gateway and forwarded to a next hop that facilitates communication of the packet to its intended destination endpoint. For example, in the embodiment depicted in FIG. 7, a packet originating from compute instance **768** may be communicated from host machine **702** to NVD **710** over link **720** (using NIC **732**). On NVD **710**, VNIC **776** is invoked since it is the VNIC associated with source compute instance **768**. VNIC **776** is configured to examine the encapsulated information in the packet, and determine a next hop for forwarding the packet with the goal of facilitating communication of the packet to

its intended destination endpoint, and then forward the packet to the determined next hop.

A compute instance deployed on a VCN can communicate with various different endpoints. These endpoints may include endpoints that are hosted by CSPI 700 and endpoints outside CSPI 700. Endpoints hosted by CSPI 700 may include instances in the same VCN or other VCNs, which may be the customer's VCNs, or VCNs not belonging to the customer. Communications between endpoints hosted by CSPI 700 may be performed over physical network 718. A compute instance may also communicate with endpoints that are not hosted by CSPI 700, or are outside CSPI 700. Examples of these endpoints include endpoints within a customer's on-premise network or data center, or public endpoints accessible over a public network such as the Internet. Communications with endpoints outside CSPI 700 may be performed over public networks (e.g., the Internet) (not shown in FIG. 7) or private networks (not shown in FIG. 7) using various communication protocols.

The architecture of CSPI 700 depicted in FIG. 7 is merely an example and is not intended to be limiting. Variations, alternatives, and modifications are possible in alternative embodiments. For example, in some implementations, CSPI 700 may have more or fewer systems or components than those shown in FIG. 7, may combine two or more systems, or may have a different configuration or arrangement of systems. The systems, subsystems, and other components depicted in FIG. 7 may be implemented in software (e.g., code, instructions, program) executed by one or more processing units (e.g., processors, cores) of the respective systems, using hardware, or combinations thereof. The software may be stored on a non-transitory storage medium (e.g., on a memory device).

FIG. 9 depicts connectivity between a host machine and an NVD for providing I/O virtualization for supporting multitenancy according to certain embodiments. As depicted in FIG. 9, host machine 902 executes a hypervisor 904 that provides a virtualized environment. Host machine 902 executes two virtual machine instances, VM1 906 belonging to customer/tenant #1 and VM2 908 belonging to customer/tenant #2. Host machine 902 comprises a physical NIC 910 that is connected to an NVD 912 via link 914. Each of the compute instances is attached to a VNIC that is executed by NVD 912. In the embodiment in FIG. 9, VM1 906 is attached to VNIC-VM1 920 and VM2 908 is attached to VNIC-VM2 922.

As shown in FIG. 9, NIC 910 comprises two logical NICs, logical NIC A 916 and logical NIC B 918. Each virtual machine is attached to and configured to work with its own logical NIC. For example, VM1 906 is attached to logical NIC A 916 and VM2 908 is attached to logical NIC B 918. Even though host machine 902 comprises only one physical NIC 910 that is shared by the multiple tenants, due to the logical NICs, each tenant's virtual machine believes they have their own host machine and NIC.

In certain embodiments, each logical NIC is assigned its own VLAN ID. Thus, a specific VLAN ID is assigned to logical NIC A 916 for Tenant #1 and a separate VLAN ID is assigned to logical NIC B 918 for Tenant #2. When a packet is communicated from VM1 906, a tag assigned to Tenant #1 is attached to the packet by the hypervisor and the packet is then communicated from host machine 902 to NVD 912 over link 914. In a similar manner, when a packet is communicated from VM2 908, a tag assigned to Tenant #2 is attached to the packet by the hypervisor and the packet is then communicated from host machine 902 to NVD 912 over link 914. Accordingly, a packet 924 communicated

from host machine 902 to NVD 912 has an associated tag 926 that identifies a specific tenant and associated VM. On the NVD, for a packet 924 received from host machine 902, the tag 926 associated with the packet is used to determine whether the packet is to be processed by VNIC-VM1 920 or by VNIC-VM2 922. The packet is then processed by the corresponding VNIC. The configuration depicted in FIG. 9 enables each tenant's compute instance to believe that they own their own host machine and NIC. The setup depicted in FIG. 9 provides for I/O virtualization for supporting multitenancy.

FIG. 10 depicts a simplified block diagram of a physical network 1000 according to certain embodiments. The embodiment depicted in FIG. 10 is structured as a Clos network. A Clos network is a particular type of network topology designed to provide connection redundancy while maintaining high bisection bandwidth and maximum resource utilization. A Clos network is a type of non-blocking, multistage or multi-tiered switching network, where the number of stages or tiers can be two, three, four, five, etc. The embodiment depicted in FIG. 10 is a 3-tiered network topology comprising tiers 1, 2, and 3. The TOR switches 1004 represent Tier-0 switches in the Clos network. One or more NVDs are connected to the TOR switches. Tier-0 switches are also referred to as edge devices of the physical network. The Tier-0 switches are connected to Tier-1 switches, which are also referred to as leaf switches. In the embodiment depicted in FIG. 10, a set of "n" Tier-0 TOR switches are connected to a set of "n" Tier-1 switches and together form a pod. Each Tier-0 switch in a pod is interconnected to all the Tier-1 switches in the pod, but there is no connectivity of switches between pods. In certain implementations, two pods are referred to as a block. Each block is served by or connected to a set of "n" Tier-2 switches (sometimes referred to as spine switches). There can be several blocks in the physical network topology. The Tier-2 switches are in turn connected to "n" Tier-3 switches (sometimes referred to as super-spine switches). Communication of packets over physical network 1000 is typically performed using one or more Layer-3 communication protocols. Typically, all the layers of the physical network, except for the TORs layer are n-ways redundant thus allowing for high availability. Policies may be specified for pods and blocks to control the visibility of switches to each other in the physical network so as to enable scaling of the physical network.

A feature of a Clos network is that the maximum hop count to reach from one Tier-0 switch to another Tier-0 switch (or from an NVD connected to a Tier-0 switch to another NVD connected to a Tier-0 switch) is fixed. For example, in a 3-Tiered Clos network at most seven hops are needed for a packet to reach from one NVD to another NVD, where the source and target NVDs are connected to the leaf tier of the Clos network. Likewise, in a 4-tiered Clos network, at most nine hops are needed for a packet to reach from one NVD to another NVD, where the source and target NVDs are connected to the leaf tier of the Clos network. Thus, a Clos network architecture maintains consistent latency throughout the network, which is important for communication within and between data centers. A Clos topology scales horizontally and is cost effective. The bandwidth/throughput capacity of the network can be easily increased by adding more switches at the various tiers (e.g., more leaf and spine switches) and by increasing the number of links between the switches at adjacent tiers.

In certain embodiments, each resource within CSPI is assigned a unique identifier called a Cloud Identifier (CID).

This identifier is included as part of the resource's information and can be used to manage the resource, for example, via a Console or through APIs. An example syntax for a CID is:

ocid1.<RESOURCE TYPE>.<REALM>.[REGION]
[.FUTURE USE].<UNIQUE ID>

where,

ocid1: The literal string indicating the version of the CID;
resource type: The type of resource (for example,
instance, volume, VCN, subnet, user, group, and so on);

realm: The realm the resource is in. Example values are
"c1" for the commercial realm, "c2" for the Govern-
ment Cloud realm, or "c3" for the Federal Government
Cloud realm, etc. Each realm may have its own domain
name;

region: The region the resource is in. If the region is not
applicable to the resource, this part might be blank;

future use: Reserved for future use.

unique ID: The unique portion of the ID. The format may
vary depending on the type of resource or service.

Cloud Bridge and Cloud Migration Service Architectures

A CSPI (e.g., CSPI **601** and **700** described with respect to
FIGS. 6 and 7) can be configured to provide a cloud
integration service. At least a portion of the cloud integration
service is provided using a cloud bridge service (e.g., the
Oracle Cloud Bridge (OCB)) for the management and
configuration of remote resources that interact with other
cloud services provided by the CSPI. The cloud bridge
addresses key aspects of this integration including automatic
discovery and identity for remote resources, representation
of remote resources as assets in the user tenancy, secure
network connectivity between remote resources and the
CSPI, lifecycle management of agent technology running in
remote environments, and assisting with migration of exist-
ing user workloads from external environments to the CSPI.
The cloud bridge enables users to give remote resources
(e.g., databases, compute nodes, etc.) a virtual presence in
the CSPI for seamless interaction with other cloud services
provided by the CSPI. In order to enable the virtual pres-
ence, the cloud bridge provides the remote resources with
unique cloud identifiers (CloudIDs, resource principles, etc.)
and remote network connectivity for service interaction.
This allows for the cloud bridge to be a single location for
users to orchestrate and manage CSPI agent functionality on
remote resources. For users, the cloud bridge provides a
unified, cloud-centric experience across both cloud-native
and remote resources with minimal setup in the remote
environment. For CSPI service teams, the cloud bridge
simplifies environment integration by allowing services to
interact with remote resources as they would with cloud
resources and by providing a standardized framework for
deploying and managing remote agent functionality. This
enables CSPI service teams to focus on the core cloud
experience without being concerned about remote connec-
tivity or software lifecycle management.

The cloud bridge is managed and configured via a console
(e.g., the CSPI console described with respect to FIGS. 6 and
7), application programming interfaces (APIs), and a soft-
ware development kit (SDK). The management and con-
figuration of the cloud bridge is focused around three basic
resource types—environments, agents, and assets. Users
create an environment for each location where they desire
access to other cloud services provided by the CSPI. The
environment serves as a container for assets and defines the
scope for managing default policy at that location.

An agent is part of a virtual machine that the user creates
from an image provided by the cloud bridge. Agents serve

as an extension of the cloud bridge services in the remote
environment and are monitored, updated, and operated by
the CSPI based on configuration of the associated environ-
ment. Agents provide the foundational cloud bridge services,
such as remote asset discovery and inventory integration,
and include a framework for service-specific plug-ins to
execute in the environment. For example, Oracle Cloud
Migration service deploys a replication plug-in that creates
virtual machine snapshots and uploads them to OCI for
migration into OCI compute. The cloud bridge allows users
to select from a catalog of agent-based services and auto-
matically deploy the associated functionality into the envi-
ronments. The cloud bridge will monitor the deployed
agents and software versions and provide manually triggered
or automated lifecycle management. Multiple agents may
exist in the same environment for redundancy and perfor-
mance reasons.

An asset is a CSPI resource that represents a remote
resource discovered by an agent, captures its properties in
the remote environment and provides CSPI properties (e.g.,
CloudID, resource principle, etc.) for use across other cloud
services provided by the CSPI. Users can manually add
assets or trigger automatic discovery at the environment
based on integrations with infrastructure/system manage-
ment products such as Oracle Enterprise Manager and
Microsoft Active Directory. The CSPI applies default access
policies based on the asset type. For example, a database
asset may default to allow only SSL-encrypted SQLNet
connections. Environments and discovered assets can be
managed through the asset inventory exposed to the user
through the console. A user may choose to perform any
action on an asset depending on the asset type and service
plugins available for the asset type. For example, virtual
machine asset types may support multiple actions: cloud
migrate, for example with Oracle Cloud Migrations, as
described in detail herein, expose as a VNIC with virtual
networking or ship metrics to CSPI with observability. A
database asset type may similarly support integration with
various database services such as Data Safe and Database
Migration services. The cloud bridge may also be integrated
with various data management services such as Management
Agent Cloud Service (MACS) agents which provide on-
guest agent and agent plugin capabilities that gather addi-
tional metadata regarding the assets. MACs agents may use
the cloud bridge agent framework, so the integration and
capabilities are seamless to the user.

FIG. 11A is a high-level diagram of a distributed envi-
ronment **1100** showing a cloud bridge architecture for the
management and configuration of remote resources that
interact with cloud services according to certain embod-
iments. As shown in the example depicted in FIG. 11A,
distributed environment **1100** comprises a cloud infrastruc-
ture **1102** (CSPI) that is physically hosted in one or more
global regions **1103** (i.e., data centers managed by a cloud
services provider such as Oracle) and provides high-perfor-
mance compute capabilities (as physical or virtual hardware
instances) and storage capacity in a flexible overlay virtual
network that is securely accessible from a user's on-prem-
ises network (e.g., on-premises network **1104** and/or on-
premises network **1106**). The CSPI **1102** comprises a con-
sole **1108** that enables users and network administrators to
configure, access, and manage remote resources and
resources deployed in the cloud using CSPI resources. In
certain embodiments, the console **1108** provides a web-
based user interface that can be used to access and manage
CSPI **1102** and various services functionalities including the

cloud bridge and migration services. In some implementations, the console **1108** is a web-based application provided by the CSP.

A CSP may provide various services using the CSPI **1102**. In some instances, users of CSPI **1102** may themselves act like service providers and provide services using CSPI **1102** resources. The service providers may expose a service endpoint, which is characterized by identification information (e.g., an IP Address, a DNS name and port). A user's resource (e.g., a compute instance, on-premises resource, off-premises resource, etc.) can consume a particular service by accessing a service endpoint exposed by the service for that particular service. These service endpoints are generally endpoints that are publicly accessible by users using public IP addresses associated with the endpoints via a public communication network such as the Internet. In certain embodiments, a service provider may expose a service (e.g., Cloud Service A, Cloud Service B, Cloud Service C, Cloud Service D, Cloud Service E) via an endpoint (sometimes referred to as a service endpoint) (e.g., CSA, CSB, CSC, CSD, CSE) for the service. Users of the service can then use this service endpoint to access the service. In certain implementations, a service endpoint provided for a service can be accessed by multiple users that intend to consume that service. In other implementations, a dedicated service endpoint may be provided for a user such that only that user can access the service using that dedicated service endpoint.

CSPI **1102** provides services and resources that users can subscribe to and use to build their VCNs. In certain embodiments, CSPI **1102** offers IaaS and iPaaS services to subscribing users. In this example, a user has configured a VCN **1110** for the global region **1103**. The user may deploy various compute instances on VCN **1110**, where the compute instances may include virtual machines or bare metal instances. Examples of instances include applications, database, load balancers, and the like. Multiple compute instances may be deployed on each subnet. In the embodiment depicted in FIG. **11A**, the VCN **1110** comprises a single subnet **1112**; however, it should be understood that the VCN **1110** could be configured with any number of subnets. The compute instances in a subnet may be hosted by one or more host machines within CSPI **1102**. A compute instance participates in a subnet via a VNIC associated with the compute instance as described in detail with respect to FIG. **1**.

A compute instance deployed on a subnet such as the subnet **1112** of the VCN **1110** can communicate with various different endpoints. These endpoints may include endpoints that are hosted by CSPI **1102** and endpoints outside CSPI **1102**. Endpoints that are hosted by CSPI **1102** may include: an endpoint on the same subnet as the particular compute instance; an endpoint on a different subnet but within the same VCN; an endpoint in a different VCN in the same region; or an endpoint in a VCN in a different region. A compute instance in a subnet hosted by CSPI **1102** may also communicate with endpoints that are not hosted by CSPI **1102** (i.e., are outside CSPI **1102**). These outside endpoints include endpoints in the user's on-premises network (e.g., on-premises network **1104** and/or on-premises network **1106**), endpoints within other remote cloud hosted networks, public endpoints accessible via a public network such as the Internet, and other endpoints.

User on-premises network **1104** and/or on-premises network **1106** may be a user network or a user data center built using the user's resources. Although the user's external resources are all shown on-premises in FIG. **11A**, it should be understood that the resources could also be off-premises

such as part of a VCN in a CSPI of a different CSP. Access to on-premises network **1104** and/or on-premises network **1106** is generally very restricted. For a user that has both a user on-premises network **1104** and/or on-premises network **1106** and one or more VCNs **1110** deployed or hosted in the cloud by CSPI **1102**, the user may want allow cloud services (e.g., cloud services A-E) to interact with a user's external resources (e.g., on-premises database **1116**, on-premise virtual machine **1118**, on-premises database **1120**, etc.). This enables a user to build an extended hybrid or multi-cloud environment encompassing the user's VCN **104** hosted by CSPI **101** and their external resources. The cloud bridge service enables this interaction. To enable such interaction, a communication channel **1122** is set up where one endpoint of the channel is in on-premises network **1104** and/or on-premises network **1106** and the other endpoint is in CSPI **1102** and connected to user VCN **1110**. Communication channel **1122** can be over public communication networks such as the Internet or private communication networks. Various communication protocols may be used such as IPsec VPN technology over a public communication network such as the Internet, Oracle's FastConnect technology that uses a private network instead of a public network, and others. The software or virtual machine in user on-premises network **1104** and/or on-premises network **1106** that forms one endpoint for communication channel **1122** is referred to as the agent **1124**. On the CSPI **1102** side, the endpoint (e.g., DB1, DB2, VM1) may be a host machine executing a gateway such as DRG.

The cloud bridge service is comprised of the following three major functionalities: agents, assets discovery, and asset inventory.

Agent functionality enables the cloud services provided by CSPI **1102** to interact with a user's external resources (e.g., on-premises database **1116**, on-premise virtual machine **1118**, and on-premises database **1120**). Agent functionality is implemented using agent **1124** and its corresponding cloud bridge control plane **1125** (also known as the agent control plane). The agent **1124** is a specialized software component used as a platform for deploying cloud service functionalities to interact with external resources. Each agent **1124** has a corresponding agent identifier (e.g., a CSPI identifier used to identify the asset as a CSPI **1102** resource) and it is associated with an external site identifier (e.g., externalSiteId) for the external environment (e.g., on-premises network **1104** and/or on-premises network **1106**) in which the agent **1124** is deployed. The external site identifier is used to identify the external environment in which external resources and agents are deployed. The agent control plane **1125** provides management and orchestration across the cloud bridge architecture such as agent registration and agent life cycle management.

Assets discovery functionality enables learning about a user's resources in external environments (e.g., on-premises network **1104** and/or on-premises network **1106**) and creating/updating assets that represent the resources in a cloud bridge inventory **1126**. Assets are a CSPI **1102** resource that represents the metadata of the resources present in the external environments such as vSphere VMs, EC2 instances, databases, etc. Each asset has an asset type defining the different kinds of assets, an asset identifier (e.g., a CSPI identifier used to identify the asset as a CSPI **1102** resource), an associated external site identifier, and a source identifier used to identify a discovery plugin that discovered the resource associate with the asset. Assets discovery functionality is implemented using a discovery plugin **1127** integrated with agent **1125** and its corresponding discovery

control plane **1128**. The discovery plugin **1127** is a software component deployed in an external environment and provides functionalities such as asset-discovery, metadata and metrics collection, and reporting of resources and metadata to the inventory **1126**. The discovery control plane **1128** comprises one or more applications deployed in an overlay, backed by a datastore (e.g., an autonomous transaction processing (ATP) datastore), and is responsible for handling asset-discovery-related requests received from a customer facing API and the communication and coordination of tasks for discovery plugins **1127**.

Asset inventory functionality enables storing the metadata on assets and relationships discovered by assets discovery or imported by a user. Asset inventory also exposes a user's facing API for creating, fetching, searching, and analyzing various assets. Asset inventory is implemented using the inventory **1126** and its corresponding inventory control plane **1130**. The inventory **1126** is a region specific CSPI **1102** resource (e.g., a data table or database) that supports the standard CRUDL operations. The inventory control plane **1130** is applications deployed in an overlay, backed by a datastore (e.g., ATP datastore), and is responsible for handling asset-inventory-related requests received from a customer facing API and the communication and coordination of tasks for inventory **1126** and assets. In certain embodiments, the asset inventory functionality is restricted to a single inventory per region. Advantageously, this helps to de-duplicate the assets in a tenancy. Avoiding duplicate assets helps downstream services such as migration service and replication service, to avoid copying the same VM data multiple times and avoids creating multiple CSPI **1102** resources for the same asset. This will also provide one global view of a user's assets across different external environments (e.g., multi-cloud services).

As shown in FIGS. **11B** and **11C**, the agent **1124** is deployed within an external environment of a user (e.g., on-premises network **1104** and/or on-premises network **1106**) as part of a virtual appliance **1132**. The virtual appliance **1132** is a virtual machine pre-configured with the agent software, plugins, and management components. The main components of the virtual appliance **1132** are an agent service subsystem **1134**, an update service subsystem **1136**, and a security service subsystem **1138**. The agent service subsystem **1134** provides agent core functionality and life cycle management of plugins such as discovery plugin **1127** and replication plugin **1140**. This includes collecting plugin state and reporting back to the agent control plane **1125**, generating and communicating agent's metrics (e.g., CPU, Memory, etc.) to CSPI monitoring, publishing local logs (e.g., appliance, agent and its plugins) to CSPI logging, and executing the local console application functionality (e.g., local console **1145** used for agent registration).

Plugins are self-contained/stand-alone applications that integrate with the agent **1124** as part of the virtual appliance **1132** that a user creates on external environments from an OVA template. The OVA template is a virtual appliance in Open Virtualization Format (OVF) used by virtualization applications such as VMware Workstation and Oracle VM Virtualbox. The OVA template can be used by a user to create multiple virtual machines (e.g., multiple VMs for agent functionality in multiple external environments). In some embodiments, the OVA template is sealed (preventing user remote access) and does not allow a user to execute arbitrary code with the virtual appliance **1132**. The user may only interact with the OVA template via the local console **1145** for registration and all other management operations are orchestrated with the virtual appliance **1132** from the

CSPI **1102**. The agent **1124** provides environmental/configuration information to plugins (e.g., CSPI **1102** region, external site identifier, agent identifier, agent type, etc.) and necessary information for plugins to obtain a resource principal session token(s) to communicate with CSPI **1102**. The agent **1124** also provides monitoring capabilities (metrics and logging) and reports plugin's status back to the agent control plane **1125**. The agent **1124** itself is not publicly reachable from the Internet and the agent **1124** and associated plugins initiate connection to CSPI **1102** API endpoints (via direct connectivity or corporate proxy).

As described with respect to FIG. **11A**, the discovery plugin **1127** facilitates learning about a user's resources in external environments (e.g., on-premises network **1104** and/or on-premises network **1106**) and creating/updating assets that represent the resources in a cloud bridge inventory **1126**. The replication plugin **1140** is deployed to the external environment **1104**, and responsible for replicating data from on-premise VMs to the user's CSPI **1102** tenancy (i.e., migration) using an asset replication task. The asset replication task is an atomic unit of work for the replication plugin **1140** to import the user's asset into CSPI **1102** object storage. This approach lowers the entry bar for migration by not requiring the user to establish on-premises connectivity to CSPI **1102** or open ports for communication back to CSPI **1102** for every asset source (e.g., vCenter). The replication plugin **1140** is published as a binary and then deployed by the CSPI **1102** service to corresponding appliances. Although only the discovery plugin **1127** and the replication plugin **1140** are shown and described herein with respect to virtual appliance **1132**, it should be understood that other types of plugins may integrate with the agent **1124** as part of the virtual appliance **1132** to facilitate providing one or more functionalities (e.g., cloud services).

For some functionalities, the agent **1124** requires third-party libraries/packages that can't be distributed as part of agent package and require consent and special actions by the user. For example, for cloud migration scope, a Virtual Disk Development Kit (VDDK) is required by the replication plugin **1140** to generate snapshots for VMware VM's disks. An agent dependencies process initiated by the user comprises the user downloading a third-party library/package from an associated third-party site, accepting terms of service and uploading the library/package to a preconfigured CSPI **1102** object storage bucket. In certain instances, public documentation provided by the CSP provides links and specific instructions to the user on how the downloading/uploading are performed and this may be performed once per dependency version. The user then logs into the console **1108**, navigates to agent dependencies set-up, creates a new agent dependency for the library/package, which comprises providing the object location in object storage (namespace, bucket, object name, etc.), and selects the type of library/package. This causes the agent control plane **1125** to start a dependency signature verification workflow that downloads and calculates a signature (checksum) for the library/package and compares the library/package signature against whitelist signatures in a database to identify and validate the library/package. Once the library/package is identified and validated, the user goes to the console **1108**, navigates to external sites set-up, and adds the agent dependency identifier to external site identifier. The agent **1124** then fetches this information from the agent control plane **1125** and locally installs the dependency and makes it available to plugins to consume.

The update service subsystem **1136** updates software and parameters of the agent service subsystem **1134** and virtual

51

appliance **1132**. The update service subsystem **1136** comprises an updater **1140**, which is a software component that runs on the virtual appliance **1132** and is responsible for keeping the virtual appliance **1132** up-to-the-date. Components of the agent **1124** are managed by cloud bridge engineers. Release of new code (e.g., new agent security update) results in creation of a new bundle for a given component (e.g., security service subsystem **1138**) of the agent **1124**. The updater **1140** maintains a list of currently used components, and versions of bundles that each component uses. A bundle is specific realization or instance of the component. A description of the bundle is stored in a database or embeddable key/value store of the agent control plane **1125**, while the data portion is a BLOB stored in object storage of the agent control plane **1125**. A BLOB is a binary file containing executables, configuration, and other data needed for the update of the component. The updater **1140** is periodically polling the agent control plane **1125**. When a new bundle is available, the agent control plane **1125** replies to the polling with an “update” command, which contains information about the new bundle to install. Upon receiving the “update” command, the updater **1140** downloads the BLOB, creates a rollback snapshot, prepares a new filesystem (e.g., a subvolume) with the update based on the BLOB, stops the agent **1124** processing, and reboots the agent **1124** into the new filesystem.

The security service subsystem **1138** implements security and compliance controls for the agent service subsystem **1134** and virtual appliance **1132** using firewalls, antivirus scanning tools, auditing tools, and system configurations.

The following use cases are provided to facilitate an understanding of the cloud bridge architecture; however, it should be understood that these use cases are nonlimiting and other use cases are contemplated and may be implemented by the cloud bridge within a CSPI.

Use Case 1: Private access to remote database for a database security cloud service.

1. A user uses the console **1108** to create an external environment: on-premises network **1104**.

2. The user downloads an OVA template for virtual appliance **1132** and deploys the agent **1124** in the on-premises network **1104**.

3. The user accesses a local console **1145** of the agent **1124** to register the agent **1124** with the cloud bridge.

4. The user initiates a discovery from the console **1108** to discover resources including on-premises database **1116** in on-premises network **1104**.

5. The agent **1124** collects host, database and database object metadata and creates or updates database assets in inventory **1126** using discovery plugin **1127**, discovery control plane **1128**, and inventory control plane **1130**.

6. The user views discovered database assets in the inventory **1126** via the console **1108**, identifies the asset associated with the on-premise database **1116**, and creates a database private endpoint (DB1) in the VCN **1110** to enable private network access to the asset associated with the on-premise database **1116**, which the user would like to use with a database security cloud service (cloud service A) (e.g., Data Safe).

7. The user creates a private endpoint (CSA) in VCN **1110** for database security cloud service (cloud service A) and configures it to protect the asset associated with the on-premise database **1116** based on the CloudID or IP address, just as the user would protect a cloud native database in their VCN **1110**.

52

8. Optionally, the user configures compute instances in VCN **1110** to connect to the on-premise database **1116** via the private endpoint (DB1) and communication channel **1122**.

5 Use Case 2: Private access to remote virtual machine hosting a data repository.

1. A user uses the console **1108** to create an external environment: on-premises network **1104**.

2. The user downloads an OVA template for virtual appliance **1132** and deploys the agent **1124** in the on-premises network **1104**.

3. The user accesses a local console **1145** of the agent **1124** to register the agent **1124** with the cloud bridge.

4. The user initiates a discovery from the console **1108** to discover resources including on-premises virtual machine **1118** running a self-hosted Bitbucket Server in on-premises network **1104**.

5. The agent **1124** collects host, database and database object metadata and creates or updates database assets in inventory **1126** using discovery plugin **1127**, discovery control plane **1128**, and inventory control plane **1130**.

6. The user views discovered VM assets in the inventory **1126** via the console **1108**, identifies the asset associated with the on-premises virtual machine **1118**, and creates a VM private endpoint (VM1) in the VCN **1110** to enable private network access to the asset associated with the on-premises virtual machine **1118**, which the user would like to use with a VM cloud service (cloud service B) (e.g., CSPI DevOps service).

7. The user creates a private endpoint (CSB) in VCN **1110** for CSPI DevOps service (e.g., a self-hosted Bitbucket Server) and selects the newly created VM private endpoint (VM1) to configure the connection URL and adds the authentication credentials.

8. The user configures the CSPI DevOps “Managed Build stage” to use the private endpoint (CSB) and VM private endpoint (VM1) as part of a DevOps CI/CD pipeline.

9. The user executes the pipeline which can access the data repository from the remote self-hosted Bitbucket Server through the private endpoint (CSB), VM private endpoint (VM1), and communication channel **1122**.

In some embodiments, the cloud integration service facilitates workload migration from external environments to the CSPI (e.g., CSPI **101** and **200** described with respect to FIGS. 1 and 2). The workload migration service assists users in all aspects of cloud migration: whether migrating a multi-tier application, a particular datacenter, or a specific class of infrastructure. A cloud migration service (e.g., the Oracle Cloud Migration (OCB)) works with the cloud bridge service (e.g., the Oracle Cloud Bridge (OCB)) for the management and configuration of remote resources that interact with other cloud services provided by the CSPI. The cloud bridge addresses key aspects of this integration including automatic discovery and cloud identity for remote resources, their representation as assets in the user tenancy, secure network connectivity between remote resources and the CSPI, and lifecycle management of agent technology running in the remote environment.

Discovery and Inventory Control for Remote Resource Integration with Cloud Services

Use of cloud services withing a CSPI can be facilitated by making it easier to move assets onto the CSPI, and specifically by making it easier for users to move assets into the CSPI. This can include, for example, migrating on-premise assets such as one or several VMs, databases, or the like into the CSPI.

A successful migration can include the use of a framework to discover and explore the existing assets such as on-premise assets, and then plan migration, replicate data and launch target environments. The steps of an exemplary migration are depicted in FIG. 12. As depicted, an exemplary migration can include managing migration assets, analyzing and migrating assets, and verifying success of the migration.

The managing of migration assets can include connecting to the source environment, deploying the virtual agent 1132, and discovering assets on the source environment. This discovery of migration assets can include deploying the virtual appliance in an on-premise environment, launching one or several plugins in the on-premise environment including, for example, a discovery plugin and a replication plugin, and using the replication plugin to manage the replication of source assets snapshots from the source environment.

Analyzing and migrating assets can include creating a migration projection, creating a migration plan, and/or replicating migration assets. The creating of the migration plan can include assessment and planning of the migration. This can include ongoing inventory analysis that contains statistics, summaries, and histograms about the virtual machines in your inventory. In some embodiments, for example, a virtual machine can contain metadata and metrics along with their history and how they are discovered or imported. Metadata history can be tracked to allow monitoring of a virtual machine's evolution over the course of a long-running migration. This can allow the identification and/or highlighting of any changes to the migrating assets, which changes can, in some embodiments, lead to one or several changes in a migration plan. In some embodiments, assessment and planning can include the creation of a migration project that contains the migration plans for replicating the virtual machines. In some embodiments, the migration plan can group interrelated and/or dependent virtual machines that can be migrated together. In some embodiments, a migration plan can be and/or include a mapping of one or several virtual machines to target resource types in the CPSI. In some embodiments, the migration plan provides the context to launch virtual machines, including compartment, subnets, and launch dependencies.

The replication of assets can include the creating of assets in the CPSI that replicate assets in the external customer environment such as, for example, in the on-premise environment. In some embodiments, this replication can be performed, in part by a replication plugin that can manage the replication of external assets snapshots. In some embodiments, this can include the management of one or several full-image and/or incremental virtual-machine snapshots.

A migration can be a long-running process, and can include one or several iterations of test launches with various configurations. In some embodiments, these iterations can include the continuous refreshing of source data from a source environment before a final migration and the activation of the migrated systems within the CSPI. For example, in an embodiment in which a customer is migrating information from a source environment into a CSPI, and specifically into the customer's tenancy in the CSPI, the customer may repeatedly discover and/or capture on-premise assets and a migration plan can be generated some or all of the times that on-premise assets are discovered. For example, in embodiments in which the customer changes one or several on-premise assets, these changes can be captured and/or discovered by repeated iteration of the discovery of those assets. In such an embodiment, a migra-

tion can be planned that can capture each of these versions of the customer assets. In some embodiments, only the final version of the assets is migrated, and in some embodiments, the customer can have control over which version of the assets are replicated in the CSPI.

In some embodiments, the planning of a migration can include evaluation of the capabilities of the CSPI as compared to the capabilities of the on-premise environment and/or on-premise assets. In some embodiments, this evaluation can include the determination of whether and to what extent on-premise capabilities differ and/or exceed CSPI capabilities. Based on this determination, the plan for the migration of assets can be modified.

Specifically, for example, during the planning step, the capabilities of customer on-premise VMs are identified, and one or several VM configurations (recommendations) within OCI are presented. This includes identifying attributes of each on-premise VM including, hardware, capabilities, and metrics of actual use. The attributes can include, number of CPUs, average CPU usage, max CPU usage, memory, average memory usage, max memory usage, number of VNICs, number of GPU, and/or network bandwidth.

Based on this information, a number of proposed OCI VM shapes are presented. These VM shapes can be generated and presented based in part on information relating to the customer, such as customer preference for optimum pricing or optimum performance.

The attributes of the on-premise VMs are compared to identified attributes of the OCI VM shapes, and a difference score is generated characterizing the difference between the attributes OCI VM shapes and the attributes of the on-premise VMs. An incompatibility error is generated when the difference score indicates too large a difference between the OCI VM shapes and the attributes of the on-premise VM.

User selected VM shapes are combined to form a migration plan that can be in a terraform representation of the migration plan.

Once the planning of the migration is complete, the replication step of the migration can be performed. In some embodiments, the replication step is performed once, and in other embodiments, the replication step can be repeatedly performed as the on-premise assets change and/or are modified. In some embodiments, the replication step can include the creation of a snapshot of one or several assets to be replicated and the storage of that snapshot. In some embodiments, that snapshot can be stored in a golden volume group (GVG). A copy of the snapshot stored in the GVG can be created, modified, and then used in the creation of an executable stack such as a terraform stack. In some embodiments, the customer can be provided with the executable stack and can modify the executable stack before executing the executable stack to thereby replicate the asset in CSPI, and specifically in the customer tenancy in CSPI.

An exemplary embodiment of an architecture 1300 that can be involved in a migration is depicted in FIG. 13. As seen, the architecture 1300 includes an external customer environment 1302, and a CPSI 1304. The external customer environment 1302 can include any environment that is distinct from the CPSI 1304 into which the assets are being migrated. In some embodiments, the external customer environment 1302 can include an on-premise environment and/or can include a CPSI separate from CPSI 1302.

The CPSI 1304 and the external customer environment 1302 can be communicatively coupled via a public network 1306. The public network 1306 can be a wired and/or wireless communications network and can include, for example, the public internet.

55

The customer environment **1302** can include the virtual appliance **1132** which can contain a replication plugin **1308**. The replication plugin **1308** can, in some embodiments, be a plugin that can perform asset replication.

The external customer environment **1302** can include a centralized management utility **1310** for the one or several assets to be replicated. This centralized management utility **1310** can be for one or several VMs. In some embodiments, the centralized management utility **1310** can be used to manage the assets being replicated and/or some or all dependent components to the assets from a single centralized location.

The CPSI **1304** can include a customer tenancy **1312** and a service tenancy **1314**. The customer tenancy **1312** can include an object storage **1314**, a replication server **1316**, and a block storage **1318**. The object storage **1314** can be an internet-scale, high-performance storage platform that offers reliable and cost-efficient data durability. In some embodiments, object storage **1314** can store, for example, large amounts of unstructured data. In some embodiments, including analytic data and rich content, like images and videos.

The replication server **1316** can be a virtual machine that can, in some embodiments, read one or several asset snapshots and can write those one or several asset snapshot into one or several volumes. In some embodiment, this can, in effect, save the one or several snapshots in one or several desired locations. In some embodiments, the one or several locations to which the one or several assets are written can be volumes corresponding to one or several attributes or all or portions of each of the assets. In some embodiments, and after some modifications, these volumes can then be used to launch one or several assets in the CPSI identical to the source assets.

The block storage **1318** can be structured storage and/or storage for structured data. In some embodiments, block storage can, for example, store data in one or several equally-sized blocks. These blocks can be stored on underlying physical storage in a manner that is optimized for fast access and retrieval.

As seen in FIG. **13**, the service tenancy can include an appliance control plane **1320**, a migration and replication control plane **1322**, a replication control plane **1324**, an inventory control plane **1326**, and a discovery control plane **1328**. In some embodiments, the control planes **1320**, **1322**, **1326**, and **1328** together for the control plane, which control plane can include the replication control plane **1324**. The replication control plane **1324** can, in some embodiments, manage replication of the one or several source assets. In some embodiments, the replication control plane **1324** can be and/or include a customer facing API that can be deployed in the overlay.

Secure Network Connectivity Between Secure Networks

Cloud computing provides many advantages, but may be disadvantageous to a user having assets in different secure networks. For example, a single user may include some assets located in one VCN, or more specifically located within a tenancy in one VCN, and other assets located in another VCN and/or located in an on-premise network. In some embodiments, it can be extremely difficult for that user to access assets between two or more secure networks.

The present relates to enabling users to access, from one secure network, assets in another secure network. This specifically includes accessing assets in an on-premise network from a VCN. In some embodiments, this can include the creation of an appliance and/or the creation of a user wallet in the on-premise network, which on-premise net-

56

work can include an asset. This asset, referred to herein as an external resource, can include, for example, a database and/or virtual machine (VM). This user wallet, which can be a user server wallet, can include one or several credentials that can be used to authenticate a user, a device, and/or a session. These one or several credentials can include, for example, one or several certificates, passwords, usernames, or the like. In some embodiments, the user wallet can be a user server wallet.

As used herein, a "wallet" refers to a container that stores information such as one or several certificates that can be used in establishing a secure connection. This information can include, for example, one or several certificate authority ("CA") certificates (trust store), one or several user certificates (key store), or the like.

The present further includes the creation of an endpoint, referred to herein as an external resource representation, in the VCN. Based on information contained in the user wallet, a VCN wallet can be created. This VCN wallet can include a VCN client wallet which can be positioned in the VCN and/or can be made accessible by the VCN.

The present further includes the creation of one or more intermediate containers. The intermediate container can be provided with and/or be provided with access to a VCN wallet and/or a user wallet. Specifically, the intermediate container can be provided with and/or be provided with access to one or both of a VCN server wallet and a user client wallet. The intermediate container can be configured to generate a session with the on-premise network, and specifically with external resource via the appliance in the on-premise network. This session can be authenticated by a combination of the user server wallet on the on-premise network and the user client wallet on or accessible by the intermediate container. This session established between the intermediate container and the on-premise network can include the creation of a secure tunnel across a public communications network such as the internet.

The intermediate container can further establish secure communication with the VCN. This secure communication can be established by the VCN server wallet and the VCN client wallet. In some embodiments, the intermediate container can function as a proxy server to route communications between the on-premise network and the VCN.

In some embodiments, the creation of one or more intermediate containers can include the creation of a network bridge. The network bridge can be a higher level container resource representing the environment being accessed. In some embodiments, the environment being accessed can be a customer on-premise environment/network. In some embodiments, a network bridge can be created for each environment being accessed. In some embodiments, each network bridge can be associated with an environment upon creation, and this association cannot be modified after the network bridge is created. In some embodiments, the network bridge can take a list of IPv4 CIDRs corresponding to the on-premise environment. These CIDRs can be consumed by a network service to provide private connectivity from the VCN to external resources in the customer on-premise environment in that CIDR range. The points of private connectivity are referred to herein as network bridge endpoints, or private endpoints. In some embodiments, network bridge endpoints can be attached to the associated network bridge upon creation.

In order to establish the TLS tunnels to the remote site, the network service can deploy the appropriate certificates and key material that can be used during the TLS phase of the tunnels. In some embodiments, each network bridge can be

considered as a security boundary. As such, in some embodiments, a private certificate authority (“CA”) can be created for each network bridge. In some embodiments, this private CA can be created in the network service tenancy and can be used to generate leaf certificates to sign certificates only for the entities that will be spun up for that remote site.

In some embodiments, and in order for a network bridge to have connectivity enabled to the VCN, a virtual appliance, with a networking plugin enabled, can be configured in the on-premise environment. In some embodiments, this virtual appliance can be installed by the customer in a section of the on-premise network that has internet connectivity. Upon installation, the networking plugin running on the agent, can register with a networking control plane as a result of which a NetworkBridgeConnection can be created. NetworkBridgeConnections can be attached to a network bridge upon creation and can be, in some embodiments, read only resources.

In some embodiments, a customer may install one or more appliances in their on-premise environment, each of which appliances can create a separate NetworkBridgeConnection. In some embodiments, the network control plane brings up secure tunnels on only one of the appliances by default, in other words, only one NetworkBridgeConnection moves to a CONNECTED state. Alternatively, if high availability is enabled, secure tunnels can be brought up on two or more appliances instead. In other words, two or more NetworkBridgeConnections can be in a CONNECTED state at any given point of time.

With reference now to FIG. 14, a schematic depiction of one embodiment of a system 1400 for secure network connectivity between secure networks is shown. The system 1400 can include the virtual appliance 1132, also referred to herein as a virtual agent 1132. The virtual appliance 1132 can be a networking plugin that can be a sealed VM providing no remote access. The virtual appliance 1132 can be manually created by a customer on the external environment and/or in the on-premise environment. In some embodiments, the creation of the virtual appliance can include the downloading of the virtual appliance 1132 to the external environment and/or the execution of a file installing the virtual appliance 1132 on the external environment.

The system includes an agent 1124 which can be a part of the virtual appliance 1132. The agent 1124 can directly communicate to the agents control plane 1125 via a public internet via, for example, public internet endpoints. The agent 1124 can manage the life-cycle of plugins and can grant an identifier to plugin. The agent control plane 1125 can, in some embodiments, manage the lifecycle of one or several agents 1124.

The agent 1124 can include a discovery plugin 1126 and a networking plugin 1402. The discovery plugin 1126 can be an application that is a part of a software package comprising the agent. In some embodiments, this software package and specifically the discovery plugin 1126 can directly communicate with the discovery control plane 1128 via public internet endpoints to orchestrate the discovery of remote resources. In some embodiments, the discovery plugin 1126 is also responsible of creation of assets in inventory, which inventory contains the metadata information of the created assets. The discovery control plane 1128 can orchestrate discovery operations via the discovery plugin 1126. Thus, in some embodiments, the discovery plugin 1126 can discover resources in an environment, such as the on-premise network 1104, and creates assets in inventory, which inventory can contain metadata information relating to each of the discovered assets.

The system can include an inventory control plane 1130. The inventory control plane 1130 can provide a common repository for assets. The inventory control plane 1130 can provide a single panel and/or interface for searching and/or organizing the assets.

The system 1400, and specifically the agent 1124 can include the networking plugin 1402. The networking plugin 1402 is an application portion of the software package comprising the agent 1124. The networking plugin 1402 communicates with network services 1404, and specifically with the network services control plane 1406 to orchestrate creation and/or establishment of secure connectivity to the VCN via public internet endpoints. The networking plugin 1402 can, in some embodiments, communicate with the network data plane 1410. In some embodiments, the communication with the network data plane 1410 can be via one or several public internet endpoints. In some embodiments, the communication with the network data plane 1410 can be via a secure connection, and specifically via one or several tunnels, such as one or several VPN tunnels. In some embodiments, these tunnels can be OpenVPN tunnels. In some embodiments, a single agent can be connected to the network data plane 1410 via redundant tunnels which can be TLS secured tunnels. In some embodiments, the networking plugin 1402 can orchestrate creation and establishment of secure connectivity to the VCN 1110 via public internet endpoints.

In some embodiments, the networking plugin 1302 can act as a bridge between a customer remote environment and the data plane 1410. In some embodiments, the networking plugin 1402 can be pre-packaged with the agent 11247. In some embodiments, the networking plugin 1402 initiates creation of OpenVPN tunnels to the data plane 1410 and is also responsible for packet forwarding in the customer remote-environment. A customer may choose to selectively disable the plugin 1402 on an appliance.

The system can include the network data plane 1410. The network data plane 1410 can be a set of network service VMs that can, in some embodiments, provide fault tolerant and redundant packet flow paths from the customer VCN to remote resources. In some embodiments, the data plane 1410 can handle packet forwarding. In some embodiments, the networking data plane 1410 provides secure connectivity to the databases in the remote site by using a combination of OpenVPN containers and CMAN containers.

The system 1400 can include a network management plane 1408. The network management plane 1408 can manage the data plane 1410, and specifically can manage data plane infrastructure resources. In some embodiments, the network management plane 1408 can manage the data plane infrastructure resources for secure connectivity with networking plugin 1402 instances. Thus, in some embodiments, the network management plane 1408 can allocate data plane resources based on customer configuration. In some embodiments, the network management plane 1408 can migrate data plane resources from failed hosts.

The system can further include an endpoint, also referred to herein as a private endpoint (PE) 1412. The PE 1412 can allow other services and/or assets in the VCN, and specifically in the customer tenancy to access the asset and/or data contained in the asset.

The system can include a DBendpoint 1413, also referred to herein as a database endpoint or as a remote resource representation. The DBendpoint 1413 provides a logical representation of a single asset within the customer’s VCN 1110 and specifically within the customer’s tenancy 1312 in the VCN 1110.

With reference to FIG. 15, a schematic depiction of another embodiment of a system 1500 for secure network connectivity between secure networks is shown. As seen, a customer has at least one asset 1502, such as a database 1504, in a first environment 1506, which first environment 1506 is a separate environment, and specifically is an on-premise environment. The customer further has a second environment 1508 which is separate from the first environment 1506. In the specific embodiment of FIG. 15, this second environment 1508 is a customer VCN 1110. This can include a customer tenancy and/or subnet. In some embodiments, the second environment 1508 can be an on-premise environment separated from the on-premise environment of the first environment 1506. In the embodiment depicted in FIG. 15, the first environment 1506 and the second environment 1508 are communicatively connected via a service tenancy 1510 which include a control/management plane VCN 1512 and a data plane VCN 1410.

The control/management plane VCN 1512 can include the control plane 1406 and the management plane 1408. The control plane 1406 can in some embodiments, manage the configuration of the customer facing resources and interact with the management plane in order to program the required resources on data plane. In some embodiments, the management plane 1408 can allocate data plane resources based on customer configuration and/or can migrate data plane resources from failed hosts. In some embodiments the control plane 1406 and/or the management plane 1408 can create the structure and/or feature to enable the customer to access the asset in the first environment from the second environment.

The data plane VCN 1410 can include the resource subnet 1520 and the tunnel subnet 1522, each of which can include a data plane node 1524. In some embodiments, the data plane can provide secure connectivity between the asset 1502 in the first environment 1506 to the second environment 1508. In some embodiments, the providing of secure connectivity between the at least one asset 1502 in the first environment 1506 and the second environment can include the forwarding of packets between these environments 1506, 1508. In some embodiments, the secure connectivity can be achieved via use of a combination of OpenVPN containers and connection manager ("CMAN") container. In some embodiments, the CMAN container can be a proxy server that forwards connection requests to databases or other proxy servers.

In some embodiments, a wallet can be accessible to each of the second environment 1508 and the service tenancy 1510. In some embodiments, the wallet can be located in and/or can be accessible by the data plane 1410. In some embodiments, this can include providing access to one or several containers in the data plane 1410 and/or by the resource subnet 1520 and/or the tunnel subnet 1522. In some embodiments, and as discussed below, this wallet can include one or several VCN wallets, including, for example, a VCN server wallet accessible by the data plane 1410 and/or a VCN client wallet accessible by the second environment 1508.

As seen FIG. 15, the data plane node 1524 in the resource subnet 1520 can include a resource shard 1526 which can contain a CMAN container 1528. In some embodiments, a CMAN container 1528 can be launched in the data plane for each database asset endpoint 1413 that is created. Thus, in some embodiments, a CMAN container 1528 can be in a 1-to-1 relationship with an asset endpoint 1413. In some embodiments, the CMAN container 1528 can serve as a proxy server to redirect packets received by the CMAN

container 1528 from the first environment 1506 to the second environment 1508 and to redirect packets received by the CMAN container 1528 from the second environment 1508 to the first environment 1506. In some embodiments, a CMAN container 1528 can be configured to handle IP based redirects in case of SCAN/RAC deployments. In some embodiments, a CMAN container 1528 can be further configured to allow application of database service-specific ACLs to thereby restrict connectivity from specific clients.

The tunnel subnet 1522 can include a tunnel shard 1530 which can contain a container 1532, and specifically can contain an OpenVPN container, with which a secure connection can be established to the first environment. In some embodiments, this secure connection can include the creation of one or several tunnels connecting the tunnel shard 1530 with the first environment 1506. These one or several tunnels can be one or several secure tunnels such as, for example, secure TLS tunnels which extend through a public internet. In some embodiments, the container 1532 on the tunnel shard 1530 can be an open VPN container.

As used herein, a shard can be a single tenanted environment responsible for providing specific functionality, in other words, a shard can be a single tenanted application-specific container. Each data plane node on the respective fleets can run multiple shards belonging to different Network Bridges. There can be, in some embodiments, 3 types of shards in the network data plane 1410: tunnel shards, resource shards and database shards.

In some embodiments, a tunnel shard can be configured to initiate a communicating connection with another resource, container, and/or environment. In some embodiments, a tunnel shard can be a docker container that runs tunneling specific software to provide connectivity to the remote customer site. The management plane can be responsible for deciding the placement of these shards.

In some embodiments, the VPN container on the tunnel shard 1530 can be communicatively coupled to the CMAN container 1528. As depicted in FIG. 15, this connection can be a communicating connection via a Border Gateway Protocol (BGP).

In some embodiments, for example, a customer and/or customer service may request access to the asset in the first environment 1506 from the endpoint 1413 in the second environment 1508. This packet can be sent from the SVNIC 1534 in the second environment 1508 to the worker VNIC 1536 in the data plane node 1524 and directed to the CMAN container 1528 associated with the second environment 1506. As discussed above, in some embodiments, a SVNIC 1534 can be created for every registered external endpoint, in other words, for every external asset, and can be associated with the endpoint 1413 in the VCN created for that asset.

The CMAN container 1528 can forward the packet to the OpenVPN 1532 container of the tunnel shard 1530, which can direct the packet to the asset 1502 in the first environment 1506 via the tunnel established between the tunnel shard and the virtual appliance (agent) 1538 in the first environment. A response packet can similarly traverse from the first environment 1506 to the second environment 1508.

With reference to FIG. 16, a flowchart illustrating one embodiment of a process 1600 for connecting the first 1506 and second environment 1508 is shown. The process 1600 can be performed by all or portions of the above-described systems. The process 1600 begins at block 1602, wherein an external resource is registered as an external endpoint. In some embodiments, this external resource can be an asset in a secure network such as, for example, the network of the

61

first environment. In some embodiments, the external resource can be registered as an external endpoint in the second environment, or in other words, in the customer VCN.

At block **1604**, a user wallet is received in the on-premise network. In some embodiments, the user wallet can be received from the user and can include security credentials. These security credentials can include, for example, one or several certificates such as trusted certificates, one or several tokens, keys, passwords, usernames, or the like. In some embodiments, the security credentials can be sufficient to authenticate a user, a tunnel, a session, a communication, or the like. In some embodiments, the user wallet can be stored in the on-premise network, and can be made available to the asset in the on-premise environment, to the VNIC of the on-premise environment, and/or to another component or module of the on-premise environment.

In some embodiments, a database wallet can be created for the user. The database wallet can be created based on security credentials and/or security information received from the user. This security information can be information contained in the user wallet and/or can be information separately provided by the user.

At block **1606**, an external resource representation **1413** is created. The external resource representation **1413** can be created to represent the external resource in the second environment **1508**, and specifically in the VCN. The external representation **1413** can be created in the second environment **1508** by the control plane **1406**. In some embodiments, the creating of the external resource representation **1413** can include the creation of an associated VNIC. In some embodiments, the associated VNIC can be SVNIC **1534**. In some embodiments, and as discussed above, a SVNIC **1534** can be created for each asset, and thus, in some embodiments, each endpoint **1413** can be associated with a unique SVNIC **1534**.

At block **1608**, a connection is established between a logical interface provisioned for the external resource and the VNIC associated with the external resource representation. In some embodiments, the connection can be established via at least one intermediate container. In some embodiments, the at least one intermediate container resides on the data plane. In some embodiments, the at least one intermediate container includes a first container and second container. The first container can be configured for communicative coupling with the first environment via, for example, a first tunnel across a first communication network. The second container can be configured for communicative coupling with the second environment, and in some embodiments for communicative coupling with the customer VCN. Further, the first and second container can be communicatively coupled with each other.

At block **1610** additional wallets are created. In some embodiments, these additional wallets can be created based at least partially on the user wallet and/or based at least partially on the database wallet. These wallets can, in some embodiments, be created in the at least one intermediate container and in the second environment including in the customer VCN. These wallets can include one or several VCN wallets, including, for example, a VCN server wallet and/or a VCN client wallet. In some embodiments, the VCN server wallet is accessible by the at least one intermediate container and the VCN client wallet is accessible by the VNIC in the second environment.

With reference now to FIG. 17, a schematic depiction of one embodiment of the system **1700** for secure network communication between secure networks is shown. As seen

62

the system includes the first environment **1506** including the external resource **1502** (sale-DB) and the user wallet, and specifically a user server wallet **1702**. The system further includes the second environment **1508** including the endpoint **1413** (Datasafe PE) and the VCN client wallet **1704**. The first environment **1506** and the second environment **1508** are linked via the at least one intermediate container **1706** that include the CMAN container **1528** and both a user client wallet **1708** and a VCN server wallet **1710**. In some embodiments, the user server wallet **1702** and the user client wallet **1708** can be provided by the customer, and in some embodiments, the VCN client wallet **1704** and the VCN server wallet **1710** can be created by the control plane. The VCN client wallet **1704** and the VCN server wallet **1710** can be used to authenticate communications and/or to create a secure session between the at least one intermediate container **1706** and the second environment **1508**, and the user server wallet **1702** and the user client wallet **1708** can be used to authenticate communications and/or to create a secure session and/or a secure tunnel between the at least one intermediate container **1706** and the first environment **1506**.

With reference now to FIG. 18, a flowchart illustrating one embodiment of a process **1800** for accessing an asset in the first environment **1506** via the second environment **1508** is shown. The process begins at block **1802** wherein VCN wallets **1704**, **1710** are created. In some embodiments, these VCN wallets **1704**, **1710** can be wallets in a VCN through use of which communication with another environment can be established. In the context of FIG. 17, these wallets **1704**, **1710** include the VCN client wallet **1704** located in the second environment **1508** and the VCN server wallet **1710** located in and/or accessible to the intermediate container **1706**. In some embodiments, and via these wallets **1704**, **1710**, communication with the first environment **1506** can be established from the second environment **1508**.

In some embodiments, the VCN wallets **1704**, **1710** can be created by the control plane. The VCN wallets **1704**, **1710** can, in some embodiments be created by the control plane based on information received from the user. In some embodiments, this information can be received from the user in the first environment **1506** and/or in the second environment **1508**. The VCN wallets **1704**, **1710** can be positioned and/or made accessible to the second environment **1508** and/or to the at least one intermediate container **1706**.

In some embodiments, each network bridge can have a separate CA. In some embodiments, such a separate CA can allow a single network service **1404** and/or network data plane **1410** to run multiple containers, and specifically multiple CMAN containers **1528** running for different customers. Thus, in some embodiments, a private CA is stood up for each network bridge and certificates are generated for the OpenVPN tunnels. In some embodiments, the same private CA can be used, and client/server certificates can be generated per network bridge endpoint signed by this root CA. In some embodiments, these can be maintained in the network service tenancy **1404**. In some embodiments, one or several client wallets and an auto-login server wallets can be generated for the CMAN container **1528**. The client wallet will be made available via an API to the local SQL clients or to partner services to connect to the remote database endpoint via a PE in the user VCN.

At block **1804**, a session is created between an external resource **1502** and at least one intermediate container **1706**. In some embodiments, this can include creating a session between the at least one intermediate container **1706** and an external appliance, such as agent **1124**, located in the first

environment **1506** and associated with the external resource **1502**. In some embodiments, the creation of the session can include the establishment of a tunnel from the first environment **1506** to the at least one intermediate container **1706**. In some embodiments, this tunnel can be created via use of a user client wallet **1708** accessible by the at least one intermediate container **1706** and a user server wallet **1702** accessible in the first environment **1506**.

At block **1806**, a second session is created between the at least one intermediate container **1706** and the external resource representation that can be endpoint **1413**. The external resource representation can, in some embodiments, be located in the second environment **1508**. In some embodiments, the creation of the second session can include the creation of a communicating coupling between the at least one intermediate container **1706** and the second environment **1508**. In some embodiments, this second session is created via the use of the VCN wallets **1704**, **1710**, and specifically via a VCN client wallet **1704** accessible to the second environment **1508** and a VCN server wallet **1710** accessible to the at least one intermediate container **1706**. In some embodiments, the second session is created via authentication with the VCN client wallet **1704** and the VCN server wallet **1710**.

At block **1808**, a request is transmitted from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource. The request can be to access data contained in the external resource and/or to access the external resource. In some embodiments, the request can comprise a query for data contained in the external resource **1502**. The request can traverse, from the second environment **1508** to the first environment **1506** via the at least one intermediate container **1706**. In some embodiments, the request is transmitted from the external resource **1502** to the external resource representation **1413** via the intermediate container **1706**, which intermediate container **1706** can, in some embodiments, comprise a proxy server.

At block **1810**, a result is received at the at least one intermediate container **1706**. In some embodiments, the result can correspond to the request. In some embodiments, the result can be received via the established connection.

At block **1812**, the result is transmitted to the external resource representation **1413**. In some embodiments, the result is transmitted via the at least one intermediate container **1706**. In some embodiments, the result is transmitted via the VNIC using the established connection.

EXAMPLE IMPLEMENTATION

As noted above, infrastructure as a service (IaaS) is one particular type of cloud computing. IaaS can be configured to provide virtualized computing resources over a public network (e.g., the Internet). In an IaaS model, a cloud computing provider can host the infrastructure components (e.g., servers, storage devices, network nodes (e.g., hardware), deployment software, platform virtualization (e.g., a hypervisor layer), or the like). In some cases, an IaaS provider may also supply a variety of services to accompany those infrastructure components (example services include billing software, monitoring software, logging software, load balancing software, clustering software, etc.). Thus, as these services may be policy-driven, IaaS users may be able to implement policies to drive load balancing to maintain application availability and performance.

In some instances, IaaS customers may access resources and services through a wide area network (WAN), such as

the Internet, and can use the cloud provider's services to install the remaining elements of an application stack. For example, the user can log in to the IaaS platform to create virtual machines (VMs), install operating systems (OSs) on each VM, deploy middleware such as databases, create storage buckets for workloads and backups, and even install enterprise software into that VM. Customers can then use the provider's services to perform various functions, including balancing network traffic, troubleshooting application issues, monitoring performance, managing disaster recovery, etc.

In most cases, a cloud computing model will require the participation of a cloud provider. The cloud provider may, but need not be, a third-party service that specializes in providing (e.g., offering, renting, selling) IaaS. An entity might also opt to deploy a private cloud, becoming its own provider of infrastructure services.

In some examples, IaaS deployment is the process of putting a new application, or a new version of an application, onto a prepared application server or the like. It may also include the process of preparing the server (e.g., installing libraries, daemons, etc.). This is often managed by the cloud provider, below the hypervisor layer (e.g., the servers, storage, network hardware, and virtualization). Thus, the customer may be responsible for handling (OS), middleware, and/or application deployment (e.g., on self-service virtual machines (e.g., that can be spun up on demand) or the like).

In some examples, IaaS provisioning may refer to acquiring computers or virtual hosts for use, and even installing needed libraries or services on them. In most cases, deployment does not include provisioning, and the provisioning may need to be performed first.

In some cases, there are two different challenges for IaaS provisioning. First, there is the initial challenge of provisioning the initial set of infrastructure before anything is running. Second, there is the challenge of evolving the existing infrastructure (e.g., adding new services, changing services, removing services, etc.) once everything has been provisioned. In some cases, these two challenges may be addressed by enabling the configuration of the infrastructure to be defined declaratively. In other words, the infrastructure (e.g., what components are needed and how they interact) can be defined by one or more configuration files. Thus, the overall topology of the infrastructure (e.g., what resources depend on which, and how they each work together) can be described declaratively. In some instances, once the topology is defined, a workflow can be generated that creates and/or manages the different components described in the configuration files.

In some examples, an infrastructure may have many interconnected elements. For example, there may be one or more virtual private clouds (VPCs) (e.g., a potentially on-demand pool of configurable and/or shared computing resources), also known as a core network. In some examples, there may also be one or more inbound/outbound traffic group rules provisioned to define how the inbound and/or outbound traffic of the network will be set up and one or more virtual machines (VMs). Other infrastructure elements may also be provisioned, such as a load balancer, a database, or the like. As more and more infrastructure elements are desired and/or added, the infrastructure may incrementally evolve.

In some instances, continuous deployment techniques may be employed to enable deployment of infrastructure code across various virtual computing environments. Additionally, the described techniques can enable infrastructure

management within these environments. In some examples, service teams can write code that is desired to be deployed to one or more, but often many, different production environments (e.g., across various different geographic locations, sometimes spanning the entire world). However, in some examples, the infrastructure on which the code will be deployed must first be set up. In some instances, the provisioning can be done manually, a provisioning tool may be utilized to provision the resources, and/or deployment tools may be utilized to deploy the code once the infrastructure is provisioned.

FIG. 19 is a block diagram 1900 illustrating an example pattern of an IaaS architecture, according to at least one embodiment. Service operators 1902 can be communicatively coupled to a secure host tenancy 1904 that can include a virtual cloud network (VCN) 1906 and a secure host subnet 1908. In some examples, the service operators 1902 may be using one or more client computing devices, which may be portable handheld devices (e.g., an iPhone®, cellular telephone, an iPad®, computing tablet, a personal digital assistant (PDA)) or wearable devices (e.g., a Google Glass® head mounted display), running software such as Microsoft Windows Mobile®, and/or a variety of mobile operating systems such as iOS, Windows Phone, Android, BlackBerry 8, Palm OS, and the like, and being Internet, e-mail, short message service (SMS), BlackBerry®, or other communication protocol enabled. Alternatively, the client computing devices can be general purpose personal computers including, by way of example, personal computers and/or laptop computers running various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems. The client computing devices can be workstation computers running any of a variety of commercially-available UNIX® or UNIX-like operating systems, including without limitation the variety of GNU/Linux operating systems, such as for example, Google Chrome OS. Alternatively, or in addition, client computing devices may be any other electronic device, such as a thin-client computer, an Internet-enabled gaming system (e.g., a Microsoft Xbox gaming console with or without a Kinect® gesture input device), and/or a personal messaging device, capable of communicating over a network that can access the VCN 1906 and/or the Internet.

The VCN 1906 can include a local peering gateway (LPG) 1910 that can be communicatively coupled to a secure shell (SSH) VCN 1912 via an LPG 1910 contained in the SSH VCN 1912. The SSH VCN 1912 can include an SSH subnet 1914, and the SSH VCN 1912 can be communicatively coupled to a control plane VCN 1916 via the LPG 1910 contained in the control plane VCN 1916. Also, the SSH VCN 1912 can be communicatively coupled to a data plane VCN 1918 via an LPG 1910. The control plane VCN 1916 and the data plane VCN 1918 can be contained in a service tenancy 1919 that can be owned and/or operated by the IaaS provider.

The control plane VCN 1916 can include a control plane demilitarized zone (DMZ) tier 1920 that acts as a perimeter network (e.g., portions of a corporate network between the corporate intranet and external networks). The DMZ-based servers may have restricted responsibilities and help keep breaches contained. Additionally, the DMZ tier 1920 can include one or more load balancer (LB) subnet(s) 1922, a control plane app tier 1924 that can include app subnet(s) 1926, a control plane data tier 1928 that can include database (DB) subnet(s) 1930 (e.g., frontend DB subnet(s) and/or backend DB subnet(s)). The LB subnet(s) 1922 contained in the control plane DMZ tier 1920 can be communicatively

coupled to the app subnet(s) 1926 contained in the control plane app tier 1924 and an Internet gateway 1934 that can be contained in the control plane VCN 1916, and the app subnet(s) 1926 can be communicatively coupled to the DB subnet(s) 1930 contained in the control plane data tier 1928 and a service gateway 1936 and a network address translation (NAT) gateway 1938. The control plane VCN 1916 can include the service gateway 1936 and the NAT gateway 1938.

The control plane VCN 1916 can include a data plane mirror app tier 1940 that can include app subnet(s) 1926. The app subnet(s) 1926 contained in the data plane mirror app tier 1940 can include a virtual network interface controller (VNIC) 1942 that can execute a compute instance 1944. The compute instance 1944 can communicatively couple the app subnet(s) 1926 of the data plane mirror app tier 1940 to app subnet(s) 1926 that can be contained in a data plane app tier 1946.

The data plane VCN 1918 can include the data plane app tier 1946, a data plane DMZ tier 1948, and a data plane data tier 1950. The data plane DMZ tier 1948 can include LB subnet(s) 1922 that can be communicatively coupled to the app subnet(s) 1926 of the data plane app tier 1946 and the Internet gateway 1934 of the data plane VCN 1918. The app subnet(s) 1926 can be communicatively coupled to the service gateway 1936 of the data plane VCN 1918 and the NAT gateway 1938 of the data plane VCN 1918. The data plane data tier 1950 can also include the DB subnet(s) 1930 that can be communicatively coupled to the app subnet(s) 1926 of the data plane app tier 1946.

The Internet gateway 1934 of the control plane VCN 1916 and of the data plane VCN 1918 can be communicatively coupled to a metadata management service 1952 that can be communicatively coupled to public Internet 1954. Public Internet 1954 can be communicatively coupled to the NAT gateway 1938 of the control plane VCN 1916 and of the data plane VCN 1918. The service gateway 1936 of the control plane VCN 1916 and of the data plane VCN 1918 can be communicatively coupled to cloud services 1956.

In some examples, the service gateway 1936 of the control plane VCN 1916 or of the data plane VCN 1918 can make application programming interface (API) calls to cloud services 1956 without going through public Internet 1954. The API calls to cloud services 1956 from the service gateway 1936 can be one-way: the service gateway 1936 can make API calls to cloud services 1956, and cloud services 1956 can send requested data to the service gateway 1936. But, cloud services 1956 may not initiate API calls to the service gateway 1936.

In some examples, the secure host tenancy 1904 can be directly connected to the service tenancy 1919, which may be otherwise isolated. The secure host subnet 1908 can communicate with the SSH subnet 1914 through an LPG 1910 that may enable two-way communication over an otherwise isolated system. Connecting the secure host subnet 1908 to the SSH subnet 1914 may give the secure host subnet 1908 access to other entities within the service tenancy 1919.

The control plane VCN 1916 may allow users of the service tenancy 1919 to set up or otherwise provision desired resources. Desired resources provisioned in the control plane VCN 1916 may be deployed or otherwise used in the data plane VCN 1918. In some examples, the control plane VCN 1916 can be isolated from the data plane VCN 1918, and the data plane mirror app tier 1940 of the control plane VCN 1916 can communicate with the data plane app

67

tier **1946** of the data plane VCN **1918** via VNICs **1942** that can be contained in the data plane mirror app tier **1940** and the data plane app tier **1946**.

In some examples, users of the system, or customers, can make requests, for example create, read, update, or delete (CRUD) operations, through public Internet **1954** that can communicate the requests to the metadata management service **1952**. The metadata management service **1952** can communicate the request to the control plane VCN **1916** through the Internet gateway **1934**. The request can be received by the LB subnet(s) **1922** contained in the control plane DMZ tier **1920**. The LB subnet(s) **1922** may determine that the request is valid, and in response to this determination, the LB subnet(s) **1922** can transmit the request to app subnet(s) **1926** contained in the control plane app tier **1924**. If the request is validated and requires a call to public Internet **1954**, the call to public Internet **1954** may be transmitted to the NAT gateway **1938** that can make the call to public Internet **1954**. Metadata that may be desired to be stored by the request can be stored in the DB subnet(s) **1930**.

In some examples, the data plane mirror app tier **1940** can facilitate direct communication between the control plane VCN **1916** and the data plane VCN **1918**. For example, changes, updates, or other suitable modifications to configuration may be desired to be applied to the resources contained in the data plane VCN **1918**. Via a VNIC **1942**, the control plane VCN **1916** can directly communicate with, and can thereby execute the changes, updates, or other suitable modifications to configuration to, resources contained in the data plane VCN **1918**.

In some embodiments, the control plane VCN **1916** and the data plane VCN **1918** can be contained in the service tenancy **1919**. In this case, the user, or the customer, of the system may not own or operate either the control plane VCN **1916** or the data plane VCN **1918**. Instead, the IaaS provider may own or operate the control plane VCN **1916** and the data plane VCN **1918**, both of which may be contained in the service tenancy **1919**. This embodiment can enable isolation of networks that may prevent users or customers from interacting with other users', or other customers', resources. Also, this embodiment may allow users or customers of the system to store databases privately without needing to rely on public Internet **1954**, which may not have a desired level of threat prevention, for storage.

In other embodiments, the LB subnet(s) **1922** contained in the control plane VCN **1916** can be configured to receive a signal from the service gateway **1936**. In this embodiment, the control plane VCN **1916** and the data plane VCN **1918** may be configured to be called by a customer of the IaaS provider without calling public Internet **1954**. Customers of the IaaS provider may desire this embodiment since database(s) that the customers use may be controlled by the IaaS provider and may be stored on the service tenancy **1919**, which may be isolated from public Internet **1954**.

FIG. **20** is a block diagram **2000** illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators **2002** (e.g., service operators **1902** of FIG. **19**) can be communicatively coupled to a secure host tenancy **2004** (e.g., the secure host tenancy **1904** of FIG. **19**) that can include a virtual cloud network (VCN) **2006** (e.g., the VCN **1906** of FIG. **19**) and a secure host subnet **2008** (e.g., the secure host subnet **1908** of FIG. **19**). The VCN **2006** can include a local peering gateway (LPG) **2010** (e.g., the LPG **1910** of FIG. **19**) that can be communicatively coupled to a secure shell (SSH) VCN **2012** (e.g., the SSH VCN **1912** of FIG. **19**) via an LPG **2010** contained in the SSH VCN **2012**. The SSH VCN **2012** can

68

include an SSH subnet **2014** (e.g., the SSH subnet **1914** of FIG. **19**), and the SSH VCN **2012** can be communicatively coupled to a control plane VCN **2016** (e.g., the control plane VCN **1916** of FIG. **19**) via an LPG **2010** contained in the control plane VCN **2016**. The control plane VCN **2016** can be contained in a service tenancy **2019** (e.g., the service tenancy **1919** of FIG. **19**), and the data plane VCN **2018** (e.g., the data plane VCN **1918** of FIG. **19**) can be contained in a customer tenancy **2021** that may be owned or operated by users, or customers, of the system.

The control plane VCN **2016** can include a control plane DMZ tier **2020** (e.g., the control plane DMZ tier **1920** of FIG. **19**) that can include LB subnet(s) **2022** (e.g., LB subnet(s) **1922** of FIG. **19**), a control plane app tier **2024** (e.g., the control plane app tier **1924** of FIG. **19**) that can include app subnet(s) **2026** (e.g., app subnet(s) **1926** of FIG. **19**), a control plane data tier **2028** (e.g., the control plane data tier **1928** of FIG. **19**) that can include database (DB) subnet(s) **2030** (e.g., similar to DB subnet(s) **1930** of FIG. **19**). The LB subnet(s) **2022** contained in the control plane DMZ tier **2020** can be communicatively coupled to the app subnet(s) **2026** contained in the control plane app tier **2024** and an Internet gateway **2034** (e.g., the Internet gateway **1934** of FIG. **19**) that can be contained in the control plane VCN **2016**, and the app subnet(s) **2026** can be communicatively coupled to the DB subnet(s) **2030** contained in the control plane data tier **2028** and a service gateway **2036** (e.g., the service gateway **1936** of FIG. **19**) and a network address translation (NAT) gateway **2038** (e.g., the NAT gateway **1938** of FIG. **19**). The control plane VCN **2016** can include the service gateway **2036** and the NAT gateway **2038**.

The control plane VCN **2016** can include a data plane mirror app tier **2040** (e.g., the data plane mirror app tier **1940** of FIG. **19**) that can include app subnet(s) **2026**. The app subnet(s) **2026** contained in the data plane mirror app tier **2040** can include a virtual network interface controller (VNIC) **2042** (e.g., the VNIC of **1942**) that can execute a compute instance **2044** (e.g., similar to the compute instance **1944** of FIG. **19**). The compute instance **2044** can facilitate communication between the app subnet(s) **2026** of the data plane mirror app tier **2040** and the app subnet(s) **2026** that can be contained in a data plane app tier **2046** (e.g., the data plane app tier **1946** of FIG. **19**) via the VNIC **2042** contained in the data plane mirror app tier **2040** and the VNIC **2042** contained in the data plane app tier **2046**.

The Internet gateway **2034** contained in the control plane VCN **2016** can be communicatively coupled to a metadata management service **2052** (e.g., the metadata management service **1952** of FIG. **19**) that can be communicatively coupled to public Internet **2054** (e.g., public Internet **1954** of FIG. **19**). Public Internet **2054** can be communicatively coupled to the NAT gateway **2038** contained in the control plane VCN **2016**. The service gateway **2036** contained in the control plane VCN **2016** can be communicatively coupled to cloud services **2056** (e.g., cloud services **1956** of FIG. **19**).

In some examples, the data plane VCN **2018** can be contained in the customer tenancy **2021**. In this case, the IaaS provider may provide the control plane VCN **2016** for each customer, and the IaaS provider may, for each customer, set up a unique compute instance **2044** that is contained in the service tenancy **2019**. Each compute instance **2044** may allow communication between the control plane VCN **2016**, contained in the service tenancy **2019**, and the data plane VCN **2018** that is contained in the customer tenancy **2021**. The compute instance **2044** may allow resources, that are provisioned in the control plane

VCN **2016** that is contained in the service tenancy **2019**, to be deployed or otherwise used in the data plane VCN **2018** that is contained in the customer tenancy **2021**.

In other examples, the customer of the IaaS provider may have databases that live in the customer tenancy **2021**. In this example, the control plane VCN **2016** can include the data plane mirror app tier **2040** that can include app subnet(s) **2026**. The data plane mirror app tier **2040** can reside in the data plane VCN **2018**, but the data plane mirror app tier **2040** may not live in the data plane VCN **2018**. That is, the data plane mirror app tier **2040** may have access to the customer tenancy **2021**, but the data plane mirror app tier **2040** may not exist in the data plane VCN **2018** or be owned or operated by the customer of the IaaS provider. The data plane mirror app tier **2040** may be configured to make calls to the data plane VCN **2018** but may not be configured to make calls to any entity contained in the control plane VCN **2016**. The customer may desire to deploy or otherwise use resources in the data plane VCN **2018** that are provisioned in the control plane VCN **2016**, and the data plane mirror app tier **2040** can facilitate the desired deployment, or other usage of resources, of the customer.

In some embodiments, the customer of the IaaS provider can apply filters to the data plane VCN **2018**. In this embodiment, the customer can determine what the data plane VCN **2018** can access, and the customer may restrict access to public Internet **2054** from the data plane VCN **2018**. The IaaS provider may not be able to apply filters or otherwise control access of the data plane VCN **2018** to any outside networks or databases. Applying filters and controls by the customer onto the data plane VCN **2018**, contained in the customer tenancy **2021**, can help isolate the data plane VCN **2018** from other customers and from public Internet **2054**.

In some embodiments, cloud services **2056** can be called by the service gateway **2036** to access services that may not exist on public Internet **2054**, on the control plane VCN **2016**, or on the data plane VCN **2018**. The connection between cloud services **2056** and the control plane VCN **2016** or the data plane VCN **2018** may not be live or continuous. Cloud services **2056** may exist on a different network owned or operated by the IaaS provider. Cloud services **2056** may be configured to receive calls from the service gateway **2036** and may be configured to not receive calls from public Internet **2054**. Some cloud services **2056** may be isolated from other cloud services **2056**, and the control plane VCN **2016** may be isolated from cloud services **2056** that may not be in the same region as the control plane VCN **2016**. For example, the control plane VCN **2016** may be located in "Region 1," and cloud service "Deployment **19**," may be located in Region **1** and in "Region 2." If a call to Deployment **19** is made by the service gateway **2036** contained in the control plane VCN **2016** located in Region **1**, the call may be transmitted to Deployment **19** in Region **1**. In this example, the control plane VCN **2016**, or Deployment **19** in Region **1**, may not be communicatively coupled to, or otherwise in communication with, Deployment **19** in Region **2**.

FIG. **21** is a block diagram **2100** illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators **2102** (e.g., service operators **1902** of FIG. **19**) can be communicatively coupled to a secure host tenancy **2104** (e.g., the secure host tenancy **1904** of FIG. **19**) that can include a virtual cloud network (VCN) **2106** (e.g., the VCN **1906** of FIG. **19**) and a secure host subnet **2108** (e.g., the secure host subnet **1908** of FIG. **19**). The VCN **2106** can include an LPG **2110** (e.g., the LPG

1910 of FIG. **19**) that can be communicatively coupled to an SSH VCN **2112** (e.g., the SSH VCN **1912** of FIG. **19**) via an LPG **2110** contained in the SSH VCN **2112**. The SSH VCN **2112** can include an SSH subnet **2114** (e.g., the SSH subnet **1914** of FIG. **19**), and the SSH VCN **2112** can be communicatively coupled to a control plane VCN **2116** (e.g., the control plane VCN **1916** of FIG. **19**) via an LPG **2110** contained in the control plane VCN **2116** and to a data plane VCN **2118** (e.g., the data plane **1918** of FIG. **19**) via an LPG **2110** contained in the data plane VCN **2118**. The control plane VCN **2116** and the data plane VCN **2118** can be contained in a service tenancy **2119** (e.g., the service tenancy **1919** of FIG. **19**).

The control plane VCN **2116** can include a control plane DMZ tier **2120** (e.g., the control plane DMZ tier **1920** of FIG. **19**) that can include load balancer (LB) subnet(s) **2122** (e.g., LB subnet(s) **1922** of FIG. **19**), a control plane app tier **2124** (e.g., the control plane app tier **1924** of FIG. **19**) that can include app subnet(s) **2126** (e.g., similar to app subnet(s) **1926** of FIG. **19**), a control plane data tier **2128** (e.g., the control plane data tier **1928** of FIG. **19**) that can include DB subnet(s) **2130**. The LB subnet(s) **2122** contained in the control plane DMZ tier **2120** can be communicatively coupled to the app subnet(s) **2126** contained in the control plane app tier **2124** and to an Internet gateway **2134** (e.g., the Internet gateway **1934** of FIG. **19**) that can be contained in the control plane VCN **2116**, and the app subnet(s) **2126** can be communicatively coupled to the DB subnet(s) **2130** contained in the control plane data tier **2128** and to a service gateway **2136** (e.g., the service gateway of FIG. **19**) and a network address translation (NAT) gateway **2138** (e.g., the NAT gateway **1938** of FIG. **19**). The control plane VCN **2116** can include the service gateway **2136** and the NAT gateway **2138**.

The data plane VCN **2118** can include a data plane app tier **2146** (e.g., the data plane app tier **1946** of FIG. **19**), a data plane DMZ tier **2148** (e.g., the data plane DMZ tier **1948** of FIG. **19**), and a data plane data tier **2150** (e.g., the data plane data tier **1950** of FIG. **19**). The data plane DMZ tier **2148** can include LB subnet(s) **2122** that can be communicatively coupled to trusted app subnet(s) **2160** and untrusted app subnet(s) **2162** of the data plane app tier **2146** and the Internet gateway **2134** contained in the data plane VCN **2118**. The trusted app subnet(s) **2160** can be communicatively coupled to the service gateway **2136** contained in the data plane VCN **2118**, the NAT gateway **2138** contained in the data plane VCN **2118**, and DB subnet(s) **2130** contained in the data plane data tier **2150**. The untrusted app subnet(s) **2162** can be communicatively coupled to the service gateway **2136** contained in the data plane VCN **2118** and DB subnet(s) **2130** contained in the data plane data tier **2150**. The data plane data tier **2150** can include DB subnet(s) **2130** that can be communicatively coupled to the service gateway **2136** contained in the data plane VCN **2118**.

The untrusted app subnet(s) **2162** can include one or more primary VNICS **2164(1)-(N)** that can be communicatively coupled to tenant virtual machines (VMs) **2166(1)-(N)**. Each tenant VM **2166(1)-(N)** can be communicatively coupled to a respective app subnet **2167(1)-(N)** that can be contained in respective container egress VCNs **2168(1)-(N)** that can be contained in respective customer tenancies **2170(1)-(N)**. Respective secondary VNICS **2172(1)-(N)** can facilitate communication between the untrusted app subnet(s) **2162** contained in the data plane VCN **2118** and the app subnet contained in the container egress VCNs **2168(1)-(N)**. Each container egress VCNs **2168(1)-(N)** can include a NAT

gateway **2138** that can be communicatively coupled to public Internet **2154** (e.g., public Internet **1954** of FIG. **19**).

The Internet gateway **2134** contained in the control plane VCN **2116** and contained in the data plane VCN **2118** can be communicatively coupled to a metadata management service **2152** (e.g., the metadata management system **1952** of FIG. **19**) that can be communicatively coupled to public Internet **2154**. Public Internet **2154** can be communicatively coupled to the NAT gateway **2138** contained in the control plane VCN **2116** and contained in the data plane VCN **2118**. The service gateway **2136** contained in the control plane VCN **2116** and contained in the data plane VCN **2118** can be communicatively couple to cloud services **2156**.

In some embodiments, the data plane VCN **2118** can be integrated with customer tenancies **2170**. This integration can be useful or desirable for customers of the IaaS provider in some cases such as a case that may desire support when executing code. The customer may provide code to run that may be destructive, may communicate with other customer resources, or may otherwise cause undesirable effects. In response to this, the IaaS provider may determine whether to run code given to the IaaS provider by the customer.

In some examples, the customer of the IaaS provider may grant temporary network access to the IaaS provider and request a function to be attached to the data plane app tier **2146**. Code to run the function may be executed in the VMs **2166(1)-(N)**, and the code may not be configured to run anywhere else on the data plane VCN **2118**. Each VM **2166(1)-(N)** may be connected to one customer tenancy **2170**. Respective containers **2171(1)-(N)** contained in the VMs **2166(1)-(N)** may be configured to run the code. In this case, there can be a dual isolation (e.g., the containers **2171(1)-(N)** running code, where the containers **2171(1)-(N)** may be contained in at least the VM **2166(1)-(N)** that are contained in the untrusted app subnet(s) **2162**), which may help prevent incorrect or otherwise undesirable code from damaging the network of the IaaS provider or from damaging a network of a different customer. The containers **2171(1)-(N)** may be communicatively coupled to the customer tenancy **2170** and may be configured to transmit or receive data from the customer tenancy **2170**. The containers **2171(1)-(N)** may not be configured to transmit or receive data from any other entity in the data plane VCN **2118**. Upon completion of running the code, the IaaS provider may kill or otherwise dispose of the containers **2171(1)-(N)**.

In some embodiments, the trusted app subnet(s) **2160** may run code that may be owned or operated by the IaaS provider. In this embodiment, the trusted app subnet(s) **2160** may be communicatively coupled to the DB subnet(s) **2130** and be configured to execute CRUD operations in the DB subnet(s) **2130**. The untrusted app subnet(s) **2162** may be communicatively coupled to the DB subnet(s) **2130**, but in this embodiment, the untrusted app subnet(s) may be configured to execute read operations in the DB subnet(s) **2130**. The containers **2171(1)-(N)** that can be contained in the VM **2166(1)-(N)** of each customer and that may run code from the customer may not be communicatively coupled with the DB subnet(s) **2130**.

In other embodiments, the control plane VCN **2116** and the data plane VCN **2118** may not be directly communicatively coupled. In this embodiment, there may be no direct communication between the control plane VCN **2116** and the data plane VCN **2118**. However, communication can occur indirectly through at least one method. An LPG **2110** may be established by the IaaS provider that can facilitate communication between the control plane VCN **2116** and the data plane VCN **2118**. In another example, the control

plane VCN **2116** or the data plane VCN **2118** can make a call to cloud services **2156** via the service gateway **2136**. For example, a call to cloud services **2156** from the control plane VCN **2116** can include a request for a service that can communicate with the data plane VCN **2118**.

FIG. **22** is a block diagram **2200** illustrating another example pattern of an IaaS architecture, according to at least one embodiment. Service operators **2202** (e.g., service operators **1902** of FIG. **19**) can be communicatively coupled to a secure host tenancy **2204** (e.g., the secure host tenancy **1904** of FIG. **19**) that can include a virtual cloud network (VCN) **2206** (e.g., the VCN **1906** of FIG. **19**) and a secure host subnet **2208** (e.g., the secure host subnet **1908** of FIG. **19**). The VCN **2206** can include an LPG **2210** (e.g., the LPG **1910** of FIG. **19**) that can be communicatively coupled to an SSH VCN **2212** (e.g., the SSH VCN **1912** of FIG. **19**) via an LPG **2210** contained in the SSH VCN **2212**. The SSH VCN **2212** can include an SSH subnet **2214** (e.g., the SSH subnet **1914** of FIG. **19**), and the SSH VCN **2212** can be communicatively coupled to a control plane VCN **2216** (e.g., the control plane VCN **1916** of FIG. **19**) via an LPG **2210** contained in the control plane VCN **2216** and to a data plane VCN **2218** (e.g., the data plane **1918** of FIG. **19**) via an LPG **2210** contained in the data plane VCN **2218**. The control plane VCN **2216** and the data plane VCN **2218** can be contained in a service tenancy **2219** (e.g., the service tenancy **1919** of FIG. **19**).

The control plane VCN **2216** can include a control plane DMZ tier **2220** (e.g., the control plane DMZ tier **1920** of FIG. **19**) that can include LB subnet(s) **2222** (e.g., LB subnet(s) **1922** of FIG. **19**), a control plane app tier **2224** (e.g., the control plane app tier **1924** of FIG. **19**) that can include app subnet(s) **2226** (e.g., app subnet(s) **1926** of FIG. **19**), a control plane data tier **2228** (e.g., the control plane data tier **1928** of FIG. **19**) that can include DB subnet(s) **2230** (e.g., DB subnet(s) **2130** of FIG. **21**). The LB subnet(s) **2222** contained in the control plane DMZ tier **2220** can be communicatively coupled to the app subnet(s) **2226** contained in the control plane app tier **2224** and to an Internet gateway **2234** (e.g., the Internet gateway **1934** of FIG. **19**) that can be contained in the control plane VCN **2216**, and the app subnet(s) **2226** can be communicatively coupled to the DB subnet(s) **2230** contained in the control plane data tier **2228** and to a service gateway **2236** (e.g., the service gateway of FIG. **19**) and a network address translation (NAT) gateway **2238** (e.g., the NAT gateway **1938** of FIG. **19**). The control plane VCN **2216** can include the service gateway **2236** and the NAT gateway **2238**.

The data plane VCN **2218** can include a data plane app tier **2246** (e.g., the data plane app tier **1946** of FIG. **19**), a data plane DMZ tier **2248** (e.g., the data plane DMZ tier **1948** of FIG. **19**), and a data plane data tier **2250** (e.g., the data plane data tier **1950** of FIG. **19**). The data plane DMZ tier **2248** can include LB subnet(s) **2222** that can be communicatively coupled to trusted app subnet(s) **2260** (e.g., trusted app subnet(s) **2160** of FIG. **21**) and untrusted app subnet(s) **2262** (e.g., untrusted app subnet(s) **2162** of FIG. **21**) of the data plane app tier **2246** and the Internet gateway **2234** contained in the data plane VCN **2218**. The trusted app subnet(s) **2260** can be communicatively coupled to the service gateway **2236** contained in the data plane VCN **2218**, the NAT gateway **2238** contained in the data plane VCN **2218**, and DB subnet(s) **2230** contained in the data plane data tier **2250**. The untrusted app subnet(s) **2262** can be communicatively coupled to the service gateway **2236** contained in the data plane VCN **2218** and DB subnet(s) **2230** contained in the data plane data tier **2250**. The data

plane data tier **2250** can include DB subnet(s) **2230** that can be communicatively coupled to the service gateway **2236** contained in the data plane VCN **2218**.

The untrusted app subnet(s) **2262** can include primary VNICs **2264(1)-(N)** that can be communicatively coupled to tenant virtual machines (VMs) **2266(1)-(N)** residing within the untrusted app subnet(s) **2262**. Each tenant VM **2266(1)-(N)** can run code in a respective container **2267(1)-(N)**, and be communicatively coupled to an app subnet **2226** that can be contained in a data plane app tier **2246** that can be contained in a container egress VCN **2268**. Respective secondary VNICs **2272(1)-(N)** can facilitate communication between the untrusted app subnet(s) **2262** contained in the data plane VCN **2218** and the app subnet contained in the container egress VCN **2268**. The container egress VCN can include a NAT gateway **2238** that can be communicatively coupled to public Internet **2254** (e.g., public Internet **1954** of FIG. **19**).

The Internet gateway **2234** contained in the control plane VCN **2216** and contained in the data plane VCN **2218** can be communicatively coupled to a metadata management service **2252** (e.g., the metadata management system **1952** of FIG. **19**) that can be communicatively coupled to public Internet **2254**. Public Internet **2254** can be communicatively coupled to the NAT gateway **2238** contained in the control plane VCN **2216** and contained in the data plane VCN **2218**. The service gateway **2236** contained in the control plane VCN **2216** and contained in the data plane VCN **2218** can be communicatively couple to cloud services **2256**.

In some examples, the pattern illustrated by the architecture of block diagram **2200** of FIG. **22** may be considered an exception to the pattern illustrated by the architecture of block diagram **2100** of FIG. **21** and may be desirable for a customer of the IaaS provider if the IaaS provider cannot directly communicate with the customer (e.g., a disconnected region). The respective containers **2267(1)-(N)** that are contained in the VMs **2266(1)-(N)** for each customer can be accessed in real-time by the customer. The containers **2267(1)-(N)** may be configured to make calls to respective secondary VNICs **2272(1)-(N)** contained in app subnet(s) **2226** of the data plane app tier **2246** that can be contained in the container egress VCN **2268**. The secondary VNICs **2272(1)-(N)** can transmit the calls to the NAT gateway **2238** that may transmit the calls to public Internet **2254**. In this example, the containers **2267(1)-(N)** that can be accessed in real-time by the customer can be isolated from the control plane VCN **2216** and can be isolated from other entities contained in the data plane VCN **2218**. The containers **2267(1)-(N)** may also be isolated from resources from other customers.

In other examples, the customer can use the containers **2267(1)-(N)** to call cloud services **2256**. In this example, the customer may run code in the containers **2267(1)-(N)** that requests a service from cloud services **2256**. The containers **2267(1)-(N)** can transmit this request to the secondary VNICs **2272(1)-(N)** that can transmit the request to the NAT gateway that can transmit the request to public Internet **2254**. Public Internet **2254** can transmit the request to LB subnet(s) **2222** contained in the control plane VCN **2216** via the Internet gateway **2234**. In response to determining the request is valid, the LB subnet(s) can transmit the request to app subnet(s) **2226** that can transmit the request to cloud services **2256** via the service gateway **2236**.

It should be appreciated that IaaS architectures **1900**, **2000**, **2100**, **2200** depicted in the figures may have other components than those depicted. Further, the embodiments shown in the figures are only some examples of a cloud

infrastructure system that may incorporate an embodiment of the disclosure. In some other embodiments, the IaaS systems may have more or fewer components than shown in the figures, may combine two or more components, or may have a different configuration or arrangement of components.

In certain embodiments, the IaaS systems described herein may include a suite of applications, middleware, and database service offerings that are delivered to a customer in a self-service, subscription-based, elastically scalable, reliable, highly available, and secure manner. An example of such an IaaS system is the Oracle Cloud Infrastructure (OCI) provided by the present assignee.

FIG. **23** illustrates an example computer system **2300**, in which various embodiments may be implemented. The system **2300** may be used to implement any of the computer systems described above. As shown in the figure, computer system **2300** includes a processing unit **2304** that communicates with a number of peripheral subsystems via a bus subsystem **2302**. These peripheral subsystems may include a processing acceleration unit **2306**, an I/O subsystem **2308**, a storage subsystem **2318** and a communications subsystem **2324**. Storage subsystem **2318** includes tangible computer-readable storage media **2322** and a system memory **2310**.

Bus subsystem **2302** provides a mechanism for letting the various components and subsystems of computer system **2300** communicate with each other as intended. Although bus subsystem **2302** is shown schematically as a single bus, alternative embodiments of the bus subsystem may utilize multiple buses. Bus subsystem **2302** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, and a local bus using any of a variety of bus architectures. For example, such architectures may include an Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnect (PCI) bus, which can be implemented as a Mezzanine bus manufactured to the IEEE P1386.1 standard.

Processing unit **2304**, which can be implemented as one or more integrated circuits (e.g., a conventional microprocessor or microcontroller), controls the operation of computer system **2300**. One or more processors may be included in processing unit **2304**. These processors may include single core or multicore processors. In certain embodiments, processing unit **2304** may be implemented as one or more independent processing units **2332** and/or **2334** with single or multicore processors included in each processing unit. In other embodiments, processing unit **2304** may also be implemented as a quad-core processing unit formed by integrating two dual-core processors into a single chip.

In various embodiments, processing unit **2304** can execute a variety of programs in response to program code and can maintain multiple concurrently executing programs or processes. At any given time, some or all of the program code to be executed can be resident in processor(s) **2304** and/or in storage subsystem **2318**. Through suitable programming, processor(s) **2304** can provide various functionalities described above. Computer system **2300** may additionally include a processing acceleration unit **2306**, which can include a digital signal processor (DSP), a special-purpose processor, and/or the like.

I/O subsystem **2308** may include user interface input devices and user interface output devices. User interface input devices may include a keyboard, pointing devices such as a mouse or trackball, a touchpad or touch screen incorporated into a display, a scroll wheel, a click wheel, a dial,

75

a button, a switch, a keypad, audio input devices with voice command recognition systems, microphones, and other types of input devices. User interface input devices may include, for example, motion sensing and/or gesture recognition devices such as the Microsoft Kinect® motion sensor that enables users to control and interact with an input device, such as the Microsoft Xbox® 360 game controller, through a natural user interface using gestures and spoken commands. User interface input devices may also include eye gesture recognition devices such as the Google Glass® blink detector that detects eye activity (e.g., ‘blinking’ while taking pictures and/or making a menu selection) from users and transforms the eye gestures as input into an input device (e.g., Google Glass®). Additionally, user interface input devices may include voice recognition sensing devices that enable users to interact with voice recognition systems (e.g., Siri® navigator), through voice commands.

User interface input devices may also include, without limitation, three dimensional (3D) mice, joysticks or pointing sticks, gamepads and graphic tablets, and audio/visual devices such as speakers, digital cameras, digital camcorders, portable media players, webcams, image scanners, fingerprint scanners, barcode reader 3D scanners, 3D printers, laser rangefinders, and eye gaze tracking devices. Additionally, user interface input devices may include, for example, medical imaging input devices such as computed tomography, magnetic resonance imaging, position emission tomography, medical ultrasonography devices. User interface input devices may also include, for example, audio input devices such as MIDI keyboards, digital musical instruments and the like.

User interface output devices may include a display subsystem, indicator lights, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device, such as that using a liquid crystal display (LCD) or plasma display, a projection device, a touch screen, and the like. In general, use of the term “output device” is intended to include all possible types of devices and mechanisms for outputting information from computer system 2300 to a user or other computer. For example, user interface output devices may include, without limitation, a variety of display devices that visually convey text, graphics and audio/video information such as monitors, printers, speakers, headphones, automotive navigation systems, plotters, voice output devices, and modems.

Computer system 2300 may comprise a storage subsystem 2318 that provides a tangible non-transitory computer-readable storage medium for storing software and data constructs that provide the functionality of the embodiments described in this disclosure. The software can include programs, code modules, instructions, scripts, etc., that when executed by one or more cores or processors of processing unit 2304 provide the functionality described above. Storage subsystem 2318 may also provide a repository for storing data used in accordance with the present disclosure.

As depicted in the example in FIG. 23, storage subsystem 2318 can include various components including a system memory 2310, computer-readable storage media 2322, and a computer readable storage media reader 2320. System memory 2310 may store program instructions that are loadable and executable by processing unit 2304. System memory 2310 may also store data that is used during the execution of the instructions and/or data that is generated during the execution of the program instructions. Various different kinds of programs may be loaded into system memory 2310 including but not limited to client applica-

76

tions, Web browsers, mid-tier applications, relational database management systems (RDBMS), virtual machines, containers, etc.

System memory 2310 may also store an operating system 2316. Examples of operating system 2316 may include various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems, a variety of commercially-available UNIX® or UNIX-like operating systems (including without limitation the variety of GNU/Linux operating systems, the Google Chrome® OS, and the like) and/or mobile operating systems such as iOS, Windows® Phone, Android® OS, BlackBerry® OS, and Palm® OS operating systems. In certain implementations where computer system 2300 executes one or more virtual machines, the virtual machines along with their guest operating systems (GOSs) may be loaded into system memory 2310 and executed by one or more processors or cores of processing unit 2304.

System memory 2310 can come in different configurations depending upon the type of computer system 2300. For example, system memory 2310 may be volatile memory (such as random access memory (RAM)) and/or non-volatile memory (such as read-only memory (ROM), flash memory, etc.) Different types of RAM configurations may be provided including a static random access memory (SRAM), a dynamic random access memory (DRAM), and others. In some implementations, system memory 2310 may include a basic input/output system (BIOS) containing basic routines that help to transfer information between elements within computer system 2300, such as during start-up.

Computer-readable storage media 2322 may represent remote, local, fixed, and/or removable storage devices plus storage media for temporarily and/or more permanently containing, storing, computer-readable information for use by computer system 2300 including instructions executable by processing unit 2304 of computer system 2300.

Computer-readable storage media 2322 can include any appropriate media known or used in the art, including storage media and communication media, such as but not limited to, volatile and non-volatile, removable and non-removable media implemented in any method or technology for storage and/or transmission of information. This can include tangible computer-readable storage media such as RAM, ROM, electronically erasable programmable ROM (EEPROM), flash memory or other memory technology, CD-ROM, digital versatile disk (DVD), or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or other tangible computer readable media.

By way of example, computer-readable storage media 2322 may include a hard disk drive that reads from or writes to non-removable, nonvolatile magnetic media, a magnetic disk drive that reads from or writes to a removable, non-volatile magnetic disk, and an optical disk drive that reads from or writes to a removable, nonvolatile optical disk such as a CD ROM, DVD, and Blu-Ray® disk, or other optical media. Computer-readable storage media 2322 may include, but is not limited to, Zip® drives, flash memory cards, universal serial bus (USB) flash drives, secure digital (SD) cards, DVD disks, digital video tape, and the like. Computer-readable storage media 2322 may also include, solid-state drives (SSD) based on non-volatile memory such as flash-memory based SSDs, enterprise flash drives, solid state ROM, and the like, SSDs based on volatile memory such as solid state RAM, dynamic RAM, static RAM, DRAM-based SSDs, magnetoresistive RAM (MRAM) SSDs, and hybrid SSDs that use a combination of DRAM and flash memory

based SSDs. The disk drives and their associated computer-readable media may provide non-volatile storage of computer-readable instructions, data structures, program modules, and other data for computer system **2300**.

Machine-readable instructions executable by one or more processors or cores of processing unit **2304** may be stored on a non-transitory computer-readable storage medium. A non-transitory computer-readable storage medium can include physically tangible memory or storage devices that include volatile memory storage devices and/or non-volatile storage devices. Examples of non-transitory computer-readable storage medium include magnetic storage media (e.g., disk or tapes), optical storage media (e.g., DVDs, CDs), various types of RAM, ROM, or flash memory, hard drives, floppy drives, detachable memory drives (e.g., USB drives), or other type of storage device.

Communications subsystem **2324** provides an interface to other computer systems and networks. Communications subsystem **2324** serves as an interface for receiving data from and transmitting data to other systems from computer system **2300**. For example, communications subsystem **2324** may enable computer system **2300** to connect to one or more devices via the Internet. In some embodiments communications subsystem **2324** can include radio frequency (RF) transceiver components for accessing wireless voice and/or data networks (e.g., using cellular telephone technology, advanced data network technology, such as 3G, 4G or EDGE (enhanced data rates for global evolution), WiFi (IEEE 802.11 family standards, or other mobile communication technologies, or any combination thereof), global positioning system (GPS) receiver components, and/or other components. In some embodiments communications subsystem **2324** can provide wired network connectivity (e.g., Ethernet) in addition to or instead of a wireless interface.

In some embodiments, communications subsystem **2324** may also receive input communication in the form of structured and/or unstructured data feeds **2326**, event streams **2328**, event updates **2330**, and the like on behalf of one or more users who may use computer system **2300**.

By way of example, communications subsystem **2324** may be configured to receive data feeds **2326** in real-time from users of social networks and/or other communication services such as Twitter® feeds, Facebook® updates, web feeds such as Rich Site Summary (RSS) feeds, and/or real-time updates from one or more third party information sources.

Additionally, communications subsystem **2324** may also be configured to receive data in the form of continuous data streams, which may include event streams **2328** of real-time events and/or event updates **2330**, that may be continuous or unbounded in nature with no explicit end. Examples of applications that generate continuous data may include, for example, sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), clickstream analysis tools, automobile traffic monitoring, and the like.

Communications subsystem **2324** may also be configured to output the structured and/or unstructured data feeds **2326**, event streams **2328**, event updates **2330**, and the like to one or more databases that may be in communication with one or more streaming data source computers coupled to computer system **2300**.

Computer system **2300** can be one of various types, including a handheld portable device (e.g., an iPhone® cellular phone, an iPad® computing tablet, a PDA), a wearable device (e.g., a Google Glass® head mounted

display), a PC, a workstation, a mainframe, a kiosk, a server rack, or any other data processing system.

Due to the ever-changing nature of computers and networks, the description of computer system **2300** depicted in the figure is intended only as a specific example. Many other configurations having more or fewer components than the system depicted in the figure are possible. For example, customized hardware might also be used and/or particular elements might be implemented in hardware, firmware, software (including applets), or a combination. Further, connection to other computing devices, such as network input/output devices, may be employed. Based on the disclosure and teachings provided herein, a person of ordinary skill in the art will appreciate other ways and/or methods to implement the various embodiments.

Although specific embodiments have been described, various modifications, alterations, alternative constructions, and equivalents are also encompassed within the scope of the disclosure. Embodiments are not restricted to operation within certain specific data processing environments, but are free to operate within a plurality of data processing environments. Additionally, although embodiments have been described using a particular series of transactions and steps, it should be apparent to those skilled in the art that the scope of the present disclosure is not limited to the described series of transactions and steps. Various features and aspects of the above-described embodiments may be used individually or jointly.

Further, while embodiments have been described using a particular combination of hardware and software, it should be recognized that other combinations of hardware and software are also within the scope of the present disclosure. Embodiments may be implemented only in hardware, or only in software, or using combinations thereof. The various processes described herein can be implemented on the same processor or different processors in any combination. Accordingly, where components or services are described as being configured to perform certain operations, such configuration can be accomplished, e.g., by designing electronic circuits to perform the operation, by programming programmable electronic circuits (such as microprocessors) to perform the operation, or any combination thereof. Processes can communicate using a variety of techniques including but not limited to conventional techniques for inter process communication, and different pairs of processes may use different techniques, or the same pair of processes may use different techniques at different times.

The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense. It will, however, be evident that additions, subtractions, deletions, and other modifications and changes may be made thereunto without departing from the broader spirit and scope as set forth in the claims. Thus, although specific disclosure embodiments have been described, these are not intended to be limiting. Various modifications and equivalents are within the scope of the following claims.

The use of the terms “a” and “an” and “the” and similar referents in the context of describing the disclosed embodiments (especially in the context of the following claims) are to be construed to cover both the singular and the plural, unless otherwise indicated herein or clearly contradicted by context. The terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (i.e., meaning “including, but not limited to,”) unless otherwise noted. The term “connected” is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation

of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within the range, unless otherwise indicated herein and each separate value is incorporated into the specification as if it were individually recited herein. All methods described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. The use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments and does not pose a limitation on the scope of the disclosure unless otherwise claimed. No language in the specification should be construed as indicating any non-claimed element as essential to the practice of the disclosure.

Disjunctive language such as the phrase “at least one of X, Y, or Z,” unless specifically stated otherwise, is intended to be understood within the context as used in general to present that an item, term, etc., may be either X, Y, or Z, or any combination thereof (e.g., X, Y, and/or Z). Thus, such disjunctive language is not generally intended to, and should not, imply that certain embodiments require at least one of X, at least one of Y, or at least one of Z to each be present.

Preferred embodiments of this disclosure are described herein, including the best mode known for carrying out the disclosure. Variations of those preferred embodiments may become apparent to those of ordinary skill in the art upon reading the foregoing description. Those of ordinary skill should be able to employ such variations as appropriate and the disclosure may be practiced otherwise than as specifically described herein. Accordingly, this disclosure includes all modifications and equivalents of the subject matter recited in the claims appended hereto as permitted by applicable law. Moreover, any combination of the above-described elements in all possible variations thereof is encompassed by the disclosure unless otherwise indicated herein.

All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to the same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

In the foregoing specification, aspects of the disclosure are described with reference to specific embodiments thereof, but those skilled in the art will recognize that the disclosure is not limited thereto. Various features and aspects of the above-described disclosure may be used individually or jointly. Further, embodiments can be utilized in any number of environments and applications beyond those described herein without departing from the broader spirit and scope of the specification. The specification and drawings are, accordingly, to be regarded as illustrative rather than restrictive.

What is claimed is:

1. A method comprising:

creating a connection between an external resource residing in an on-premise network and at least one intermediate container, the on-premise network comprising a user wallet and the intermediate container comprising a VCN wallet, wherein creating the connection includes authentication with the user wallet and the VCN wallet, and wherein the external resource is registered as an external endpoint in a virtual cloud network (VCN); transmitting a request from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource;

receiving at the at least one intermediate container a result corresponding to the request via the created connection; and

transmitting, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection.

2. The method of claim 1, further comprising creating a second connection between the at least one intermediate container and the external resource representation.

3. The method of claim 2, wherein the VCN wallet comprises a VCN client wallet, and wherein the at least one intermediate container comprises a VCN server wallet.

4. The method of claim 3, wherein creating the second connection includes authentication with the VCN client wallet and the VCN server wallet.

5. The method of claim 4, further comprising creating each of the VCN client wallet and the VCN server wallet based on the user wallet.

6. The method of claim 4, wherein the request is transmitted from the external resource to the external resource representation via the intermediate container.

7. The method of claim 6, wherein the intermediate container resides on a data plane.

8. The method of claim 6, wherein the intermediate container comprises a proxy server.

9. The method of claim 7, wherein the intermediate container comprises a first container and a second container.

10. The method of claim 9, wherein the first container is configured for communicative coupling with the on-premise network via a first tunnel shard.

11. The method of claim 10, wherein the first container is configured for communicative coupling with the on-premise network via the first tunnel shard and a public communication network.

12. The method of claim 10, wherein the at least one intermediate container comprises a second container configured for communicative coupling with the VCN.

13. The method of claim 12, wherein the second container comprises a connection manager (“CMAN”) container configured to serve as a proxy server to redirect packets received by the CMAN container from the VCN to the on-premise network.

14. The method of claim 12, wherein the first container is communicatively coupled with the second container.

15. The method of claim 14, further comprising registering the external resource residing as the external endpoint in the VCN.

16. The method of claim 15, further comprising creating the external resource representation for the external endpoint in the VCN, the creating of the external resource representation comprising creating the VNIC.

17. The method of claim 16, further comprising creating a private endpoint in the VCN.

18. The method of claim 17, wherein the private endpoint is communicatively coupled with the external resource via the external resource representation.

19. A system comprising:

memory comprising processor-executable stored instructions; and

a processor configured to execute the stored instructions to:

create a connection between an external resource residing in an on-premise network and at least one intermediate container, the on-premise network comprising a user wallet and the intermediate container comprising a VCN wallet, wherein creating the connection includes authentication with the user wallet

81

and the VCN wallet, and wherein the external resource is registered as an external endpoint in a virtual cloud network (VCN);

transmit a request from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource;

receive at the at least one intermediate container a result corresponding to the request via the created connection; and

transmit, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection.

20. A non-transitory computer-readable storage medium storing a plurality of instructions executable by one or more processors, the plurality of instructions when executed by the one or more processors cause the one or more processors to:

82

create a connection between an external resource residing in an on-premise network and at least one intermediate container, the on-premise network comprising a user wallet and the intermediate container comprising a VCN wallet, wherein creating the connection includes authentication with the user wallet and the VCN wallet, and wherein the external resource is registered as an external endpoint in a virtual cloud network (VCN);

transmit a request from an external resource representation residing in the VCN via a virtual network interface card (VNIC) to the external resource;

receive at the at least one intermediate container a result corresponding to the request via the created connection; and

transmit, by the at least one intermediate container, the result to the external resource representation via the VNIC using the created connection.

* * * * *